ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA

BÁO CÁO BÀI TẬP LỚN

MÔN: ĐÁNH GIÁ VÀ KIỂM ĐỊNH CHẤT LƯỢNG PHẦN MỀM

ĐỀ TÀI: Đánh giá chất lượng phần mềm

Quản lý phòng khám thú cưng

Nhóm sinh viên thực hiện :

STT	Họ và tên	MSV	Lớp
1	Lữ Trung Anh	23010339	K17-CNTTVJ
2	Đỗ Quỳnh Chi	23010618	K17-CNTTVJ
3	Đỗ Vân Khánh	23013929	K17-CNTTVJ

Lớp tín chỉ: CSE703010-1-1-26(COUR02.LT1)

Giảng viên hướng dẫn : **TS. Trịnh Thanh Bình**

Hà Nội, tháng 10 năm 2026

MỤC LỤC

BẢNG PHÂN CÔNG NHIỆM VỤ 5
DANH MỤC HÌNH ẢNH 6
DANH MỤC BẢNG BIỂU 8
LỜI MỞ ĐẦU 9
1. Chương 1: Phần mềm quản lý phòng khám thú cưng 10
1.1. Xác định yêu cầu 10
1.1.1. Đặt vấn đề 10
1.1.2. Yêu cầu chung 10
1.1.3. Yêu cầu cụ thể 11
1.2. Các yêu cầu chức năng 11
1.2.1. Tác nhân hệ thống 11
1.2.2. Các chức năng hệ thống 12
1.2.3. Sơ đồ Use-Case tổng quát 12
1.2.4. Đặc tả Use-Case 12
1.3. Các yêu cầu phi chức năng 18
1.3.1. Giao diện người dùng 18
1.3.2. Hiệu suất và bảo mật 18
2. Chương 2: Kế hoạch kiểm thử 19
2.1. Giới thiệu 19
2.1.1. Mục đích 19
2.1.2. Tổng quan 19
2.1.3. Phạm vi 19
2.1.4. Các định nghĩa và các từ viết tắt được dùng 19
2.1.5. Những người sử dụng tài liệu này 20
2.2. Những yêu cầu về tài nguyên 20
2.2.1. Phần cứng 20
2.2.2. Phần mềm 21
2.2.3. Công cụ kiểm thử 21
2.2.4. Môi trường kiểm thử 21
2.3. Phạm vi kiểm thử 22
2.3.1. Những chức năng được kiểm thử 22
2.3.2. Những chức năng không được kiểm thử 23
2.4. Chiến lược kiểm thử 23
2.4.1. Các hình thức kiểm thử và các bước kiểm thử 23
2.4.2. Các loại kiểm thử 24
2.5. Điều kiện chấp thuận 27
2.6. Defect Tracking (Lỗi) 27
2.6.1. Phân loại lỗi 27
2.6.2. Quy trình xử lý lỗi 27
3. Chương 3: Xây dựng các test case 28
3.1. Chức năng của HỆ THỐNG 28
3.1.1. Truy cập hệ thống & trang Tổng quan 28
3.1.2. Đăng nhập & phân quyền (RBAC – R6.2) 29
3.2. Chức năng của LỄ TÂN 29
3.2.1. UC1.1 Tạo mới hồ sơ khách hàng 29
3.2.2. UC1.3 Chỉnh sửa hồ sơ khách hàng 32

3.2.3. UC1.4 Tạm khóa / Duyệt hồ sơ khách hàng 33
3.2.4. UC1.5 Tìm kiếm khách hàng / thú cưng 33
3.2.5. UC1.2 Tạo mới hồ sơ thú cưng 34
3.2.6. UC2.1 Đặt lịch khám 35
3.2.7. UC2.2 Xác nhận / Hủy / Hoàn thành lịch hẹn 37
3.3. Chức năng của BÁC SĨ 37
3.3.1. UC3.1 Tạo hồ sơ khám bệnh 37
3.3.2. UC3.3 Tra cứu lịch sử khám 38
3.3.3. UC2.4 Quản lý ca làm việc bác sĩ 39
3.3.4. UC8.2 Theo dõi thú cưng qua camera 40
3.4. Chức năng của QUẢN TRỊ VIÊN / KẾ TOÁN 41
3.4.1. UC6.1 Quản lý thông tin bác sĩ 41
3.4.2. UC4.1 Nhập kho thuốc / vật tư 42
3.4.3. UC4.3 Cảnh báo tồn kho thấp 43
3.4.4. UC5.1 Tạo hóa đơn bán thuốc 44
3.4.5. UC5.2 Thanh toán hóa đơn 44
3.4.6. UC7.2 Báo cáo doanh thu 45
3.5. Test case phi chức năng 45
3.5.1. Kiểm thử tải và hiệu năng 45
3.5.2. Kiểm thử chịu tải 46
3.5.3. Kiểm thử toàn vẹn dữ liệu 47
3.5.4. Kiểm thử độ tin cậy và phục hồi 48
3.6. Biểu đồ so sánh 48
3.7. Danh sách lỗi phát hiện 48
4. Chương 4: Cài đặt phần mềm 51
4.1. Giới thiệu 51
4.2. Yêu cầu hệ thống 51
4.2.1. Yêu cầu phần cứng 51
4.2.2. Yêu cầu phần mềm 51
4.3. Quy trình cài đặt 51
4.4. Hướng dẫn sử dụng cơ bản 51
5. Chương 5: Kết quả kiểm thử và kết luận 52
5.1. Giới thiệu trang web quản lý phòng khám thú cưng 52
5.1.1. Giao diện Tổng quan vận hành phòng khám 52
5.1.2. Giao diện Quản lý khách hàng 52
5.1.3. Giao diện Quản lý lịch hẹn 53
5.1.4. Giao diện Quản lý bác sĩ 53
5.1.5. Giao diện Quản lý hồ sơ khám bệnh 54
5.1.6. Giao diện Ca làm việc bác sĩ 54
5.1.7. Giao diện Quản lý kho thuốc và vật tư 55
5.1.8. Giao diện Hóa đơn 55
5.1.9. Giao diện Camera giám sát 56
5.1.10. Một số màn hình ghi nhận trong quá trình kiểm thử 56
5.2. Kiểm thử đơn vị (Unit Test) bằng Mocha 59
5.2.1. Mục tiêu và phạm vi 59
5.2.2. Cài đặt và chạy 60
5.2.3. Mã nguồn unit test 60
5.2.4. Kết quả unit test 64

5.3. Kiểm thử tự động bằng Mocha và Selenium WebDriver 65
5.3.1. Giới thiệu về Mocha 65
5.3.2. Giới thiệu về Selenium WebDriver 65
5.3.3. Cấu trúc bộ kiểm thử tự động 66
5.3.4. Mã nguồn kiểm thử tự động 66
5.3.5. Kết quả kiểm thử tự động 69
5.3.6. Mục tiêu và phạm vi kiểm thử 72
5.4. Kiểm thử hiệu năng: tải và chịu tải 72
5.4.1. Giới thiệu công cụ autocannon 72
5.4.2. Cài đặt và chạy 72
5.4.3. Mã nguồn kiểm thử hiệu năng 72
5.4.4. Kết quả kiểm thử tải 73
5.4.5. Kết quả kiểm thử chịu tải 74
5.5. Kiểm thử toàn vẹn dữ liệu 76
5.5.1. Cài đặt và chạy 76
5.5.2. Mã nguồn kiểm thử toàn vẹn dữ liệu 76
5.5.3. Kết quả kiểm thử toàn vẹn dữ liệu 77
5.6. Kiểm thử độ tin cậy và phục hồi 77
5.6.1. Cài đặt và chạy 77
5.6.2. Mã nguồn kiểm thử độ tin cậy và phục hồi 78
5.6.3. Kết quả kiểm thử độ tin cậy và phục hồi 78
5.7. Tổng hợp kết quả 79
5.8. KẾT LUẬN 80
TÀI LIỆU THAM KHẢO 82

BẢNG PHÂN CÔNG NHIỆM VỤ

STT	Họ tên	MSSV	Nhiệm vụ	Đánh giá
1	Đỗ Quỳnh Chi	23010618	- Phân tích hệ thống, xác định yêu cầu chức năng & phi chức năng - Lập kế hoạch kiểm thử (Test Plan), xác định phạm vi kiểm thử - Kiểm thử thủ công (Manual Testing) - Tổng hợp báo cáo, làm slide	33.3%
2	Lữ Trung Anh	23010339	- Vẽ sơ đồ Use Case, đặc tả Use Case - Thiết kế test case cho các tác nhân: Hệ thống, Lễ tân, Bác sĩ, Quản trị viên/Kế toán - Viết Unit Test (Mocha + tsx) cho schema và lớp storage - Hỗ trợ hoàn thiện báo cáo	33.3%
3	Đỗ Vân Khánh	23013929	- Cấu hình môi trường kiểm thử tự động Mocha & Selenium WebDriver - Viết script và thực thi Automation Test - Thu thập kết quả, vẽ biểu đồ so sánh, viết kết luận	33.3%

DANH MỤC HÌNH ẢNH

Hình 1: Sơ đồ Use-Case tổng quát của hệ thống Pet Clinic

Hình 2: Biểu đồ so sánh các test case chức năng theo nhóm

Hình 3: Giao diện Tổng quan vận hành phòng khám

Hình 4: Giao diện Quản lý khách hàng

Hình 5: Giao diện Quản lý lịch hẹn

Hình 6: Giao diện Quản lý bác sĩ

Hình 7: Giao diện Quản lý hồ sơ khám bệnh

Hình 8: Giao diện Ca làm việc bác sĩ

Hình 9: Giao diện Quản lý kho thuốc và vật tư

Hình 10: Giao diện Hóa đơn

Hình 11: Giao diện Camera giám sát

Hình 12: UC1.1-TC001 – Thông báo lỗi khi bỏ trống form thêm khách hàng

Hình 13: UC1.1-TC007 – Thêm khách hàng thành công

Hình 14: UC1.1-TC010 (Fail) – Bỏ trống Địa chỉ bị từ chối kèm lỗi JSON

Hình 15: UC2.1-TC002 – Đặt lịch hẹn thành công

Hình 16: UC3.1-TC001 – Thông báo lỗi khi bỏ trống hồ sơ khám

Hình 17: UC4.3-TC001 – Cảnh báo tồn kho thấp

Hình 18: UC5.1-TC002 – Tạo hóa đơn thành công, tồn kho tự trừ

Hình 19: Kết quả chạy unit test bằng Mocha

Hình 20: Kết quả chạy kiểm thử tự động (phần 1)

Hình 21: Kết quả chạy kiểm thử tự động (phần 2)

Hình 22: Chi tiết các test case Fail trong lần chạy tự động

Hình 23: Thời gian phản hồi p50/p99 của các kịch bản tải

Hình 24: Kết quả chạy kiểm thử tải

Hình 25: Kết quả kiểm thử tìm kiếm trên 10.000 bản ghi

Hình 26: Thời gian phản hồi và tỷ lệ lỗi khi tăng dần tải

Hình 27: Kết quả chạy kiểm thử chịu tải

Hình 28: Kết quả chạy kiểm thử toàn vẹn dữ liệu

Hình 29: Bộ nhớ ứng dụng trong thời gian chạy tải liên tục

Hình 30: Kết quả chạy kiểm thử độ tin cậy và phục hồi

Hình 31: Tỷ lệ Pass/Fail của kiểm thử chức năng và kiểm thử đơn vị

DANH MỤC BẢNG BIỂU

Bảng 1: Bảng tổng hợp các nhóm chức năng của hệ thống

Bảng 2: Bảng liệt kê các định nghĩa được sử dụng

Bảng 3: Bảng liệt kê các từ viết tắt được sử dụng

Bảng 4: Bảng yêu cầu về tài nguyên phần cứng

Bảng 5: Bảng yêu cầu về tài nguyên phần mềm

Bảng 6: Bảng liệt kê các công cụ kiểm thử

Bảng 7: Bảng dữ liệu mẫu dùng cho kiểm thử

Bảng 8: Bảng phân loại mức độ nghiêm trọng của lỗi

Bảng 9: Bảng tổng hợp số lượng test case theo tác nhân

Bảng 10: Bảng test case kiểm thử tải và hiệu năng

Bảng 11: Bảng test case kiểm thử chịu tải

Bảng 12: Bảng test case kiểm thử toàn vẹn dữ liệu

Bảng 13: Bảng test case kiểm thử độ tin cậy và phục hồi

Bảng 14: Bảng danh sách lỗi phát hiện qua kiểm thử chức năng và phi chức năng

Bảng 15: Bảng kết quả kiểm thử đơn vị

Bảng 16: Bảng cấu trúc mã nguồn kiểm thử tự động

Bảng 17: Bảng kết quả kiểm thử tải với 200 người dùng đồng thời

Bảng 18: Bảng kết quả kiểm thử chịu tải theo từng mức kết nối

Bảng 19: Bảng kết quả kiểm thử độ tin cậy và phục hồi

Bảng 20: Bảng tổng hợp kết quả kiểm thử

LỜI MỞ ĐẦU

Trong thời đại chuyển đổi số, phần mềm ngày càng giữ vai trò trung tâm trong vận hành của các cơ sở dịch vụ, trong đó có phòng khám thú y. Cùng với đó, kiểm thử và đánh giá chất lượng phần mềm là giai đoạn không thể thiếu trong vòng đời phát triển, giúp sản phẩm vận hành ổn định, chính xác và an toàn. Với mục tiêu áp dụng kiến thức môn học vào thực tiễn, nhóm lựa chọn đề tài **"Đánh giá chất lượng phần mềm quản lý phòng khám thú cưng (Pet Clinic)"**.

Pet Clinic Pro là ứng dụng web (React + Express + MySQL) hỗ trợ phòng khám quản lý khách hàng và thú cưng, lịch hẹn, ca làm việc của bác sĩ, hồ sơ khám bệnh, kho thuốc, hóa đơn và camera giám sát. Phần mềm phục vụ các nhóm người dùng chính:

- **Lễ tân:** tiếp nhận khách hàng, tạo hồ sơ khách hàng và thú cưng, đặt và xử lý lịch hẹn.
- **Bác sĩ thú y:** lập hồ sơ khám, tra cứu lịch sử điều trị, theo dõi ca trực và camera.
- **Quản trị viên / Kế toán:** quản lý bác sĩ, kho thuốc, hóa đơn, thanh toán và báo cáo doanh thu.
- **Khách hàng:** đặt lịch, nhận nhắc lịch và theo dõi thú cưng qua camera.

Đề tài không chỉ dừng ở việc tìm hiểu phần mềm mà tập trung kiểm thử toàn diện: xây dựng kế hoạch kiểm thử, thiết kế 74 test case chức năng, 30 unit test và 28 test case phi chức năng (tải, chịu tải, toàn vẹn dữ liệu, độ tin cậy và phục hồi), tự động hóa toàn bộ bằng Mocha, Selenium WebDriver và autocannon. Qua đó nhóm rèn luyện kỹ năng phân tích, thiết kế và thực thi kiểm thử, đồng thời chỉ ra các lỗi cụ thể để nhóm phát triển khắc phục.

1. Chương 1: Phần mềm quản lý phòng khám thú cưng

1.1. Xác định yêu cầu

1.1.1. Đặt vấn đề

Số hộ gia đình nuôi thú cưng ở các thành phố tăng nhanh, nhưng nhiều phòng khám thú y vẫn quản lý bằng sổ sách hoặc Excel. Cách làm này bộc lộ nhiều bất cập:

- **Trùng lặp lịch hẹn, tiếp nhận chậm:** lễ tân khó biết bác sĩ nào còn trống, thú cưng nào đã có lịch.
- **Hồ sơ rời rạc:** hồ sơ khám, đơn thuốc và lịch sử điều trị nằm ở nhiều nơi, khó truy xuất khi tái khám.
- **Quản lý kho kém hiệu quả:** dễ thất thoát, khó kiểm soát hạn dùng và lô thuốc.
- **Hóa đơn, báo cáo thủ công:** làm chậm việc quyết toán và đánh giá doanh thu.
- **Giám sát thú cưng chưa tích hợp:** khách hàng phải xin đường dẫn camera thủ công, tiềm ẩn rủi ro bảo mật.

Vì vậy cần một hệ thống số hóa giúp quản lý khách hàng và thú cưng, đặt lịch hẹn, lưu hồ sơ khám, quản lý kho thuốc, đồng thời phát sinh hóa đơn và báo cáo tự động. Đề tài "Đánh giá chất lượng phần mềm quản lý phòng khám thú cưng" được lựa chọn để kiểm chứng phần mềm Pet Clinic Pro đáp ứng các yêu cầu trên đến đâu.

1.1.2. Yêu cầu chung

Để giải quyết các vấn đề nêu trên, hệ thống cần đáp ứng các yêu cầu chung sau:

- **Quản lý toàn diện:** từ tiếp nhận khách hàng đến khám, kê đơn và thanh toán.
- **Đặt lịch và nhắc lịch tự động:** kiểm tra lịch trống của bác sĩ, gửi nhắc lịch qua Email/SMS.
- **Tập trung hồ sơ khám:** lưu triệu chứng, chẩn đoán, phác đồ điều trị theo từng thú cưng.
- **Kiểm soát kho:** nhập kho theo lô, tự trừ tồn khi bán, cảnh báo tồn thấp và sắp hết hạn.
- **Bảo mật và phân quyền:** phân quyền theo vai trò (RBAC) cho Admin, Bác sĩ, Lễ tân, Khách hàng.
- **Dễ sử dụng:** giao diện trực quan, thông báo và kiểm tra dữ liệu rõ ràng.

Phạm vi đề tài:

- **Đối tượng sử dụng:** phòng khám thú y và cửa hàng thú cưng quy mô vừa và nhỏ.
- **Kiến trúc:** ứng dụng web nhiều lớp – giao diện React (Vite, TanStack Query), API Express/TypeScript, CSDL MySQL qua Drizzle ORM.
- **Ngoài phạm vi:** liên thông dữ liệu y tế quốc gia, chatbot, khám từ xa (telemedicine).

1.1.3. Yêu cầu cụ thể

- **Quản lý khách hàng & thú cưng (R1):** tạo, sửa, khóa/duyệt hồ sơ; số điện thoại 10 chữ số và không trùng; microchip không trùng; tìm kiếm theo tên, email, số điện thoại.
- **Quản lý lịch khám (R2):** đặt lịch theo lịch trống của bác sĩ; chỉ được hủy/đổi trước giờ hẹn ít nhất 1 tiếng; nhắc lịch tự động; quản lý ca làm việc tránh trùng.
- **Hồ sơ khám (R3):** tạo hồ sơ khám với triệu chứng, chẩn đoán, điều trị; tra cứu lịch sử.
- **Kho thuốc (R4):** nhập kho theo lô kèm hạn dùng; trừ tồn khi bán; cảnh báo tồn thấp và sắp hết hạn.
- **Thanh toán (R5):** tạo hóa đơn, thanh toán nhiều hình thức (tiền mặt, thẻ, chuyển khoản, ví điện tử), báo cáo doanh thu.
- **Nhân sự & phân quyền (R6), báo cáo (R7), camera (R8):** quản lý bác sĩ, phân quyền RBAC, thống kê, xem camera theo phòng.

1.2. Các yêu cầu chức năng

1.2.1. Tác nhân hệ thống

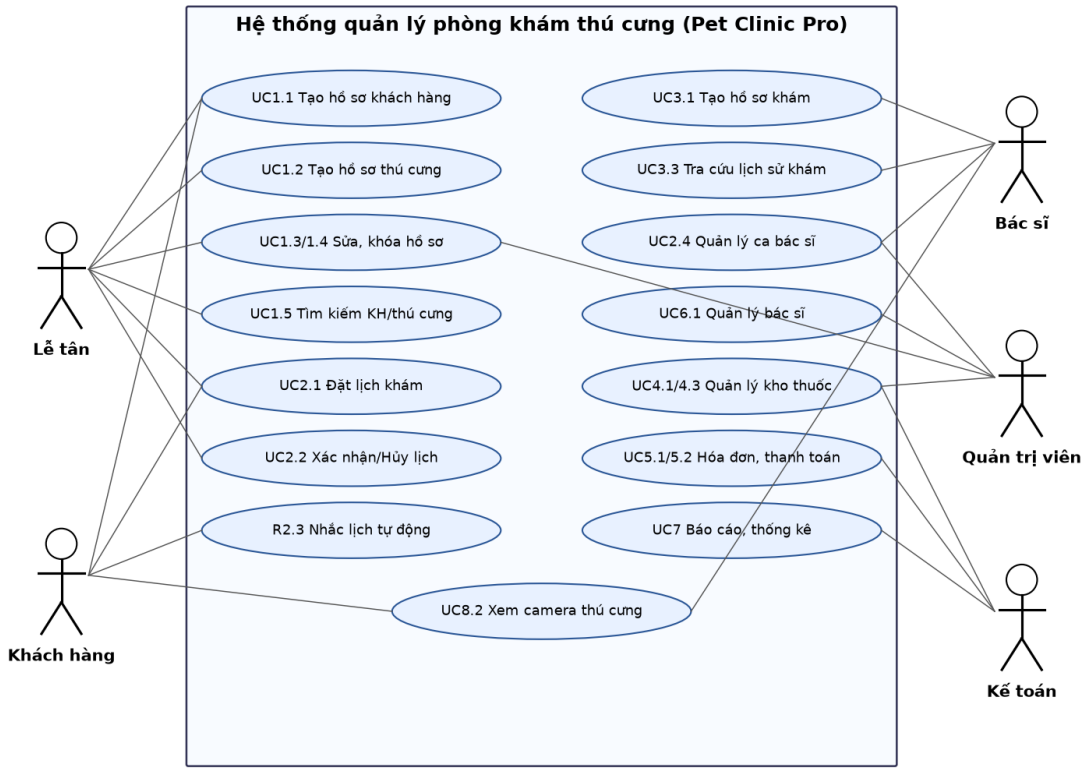
- **Lễ tân (Receptionist):** tiếp nhận khách hàng, quản lý hồ sơ khách hàng và thú cưng, đặt và xử lý lịch hẹn.
- **Bác sĩ thú y (Veterinarian):** tạo hồ sơ khám, kê đơn, tra cứu lịch sử khám, xem ca trực và camera.
- **Quản trị viên (Admin):** quản lý bác sĩ, ca trực, kho thuốc, phân quyền người dùng.
- **Kế toán:** lập hóa đơn, xác nhận thanh toán, xem báo cáo doanh thu.
- **Khách hàng (Customer):** đặt lịch, nhận nhắc lịch, xem camera thú cưng lưu trú.

1.2.2. Các chức năng hệ thống

Nhóm chức năng	Chức năng	Màn hình
R1 – Khách hàng & thú cưng	UC1.1 Tạo hồ sơ khách hàng; UC1.2 Tạo hồ sơ thú cưng; UC1.3 Chỉnh sửa; UC1.4 Tạm khóa/Duyệt; UC1.5 Tìm kiếm	Khách hàng
R2 – Lịch khám	UC2.1 Đặt lịch; UC2.2 Xác nhận/Hủy/Hoàn thành; R2.3 Nhắc lịch tự động; UC2.4 Ca làm việc bác sĩ	Lịch hẹn, Ca bác sĩ
R3 – Hồ sơ khám	UC3.1 Tạo hồ sơ khám; UC3.3 Tra cứu lịch sử khám	Hồ sơ khám
R4 – Kho thuốc	UC4.1 Nhập kho theo lô; UC4.2 Trừ tồn khi bán; UC4.3 Cảnh báo tồn thấp/hết hạn	Kho thuốc
R5 – Thanh toán	UC5.1 Tạo hóa đơn; UC5.2 Thanh toán	Hóa đơn
R6 – Nhân sự	UC6.1 Quản lý bác sĩ; UC6.2 Phân quyền RBAC	Bác sĩ
R7 – Báo cáo	UC7.1 Số ca khám; UC7.2 Doanh thu; UC7.3 Thuốc/dịch vụ	Tổng quan
R8 – Camera	UC8.1 Quản lý kết nối camera; UC8.2 Theo dõi trực tuyến	Camera giám sát

Bảng 1: Bảng tổng hợp các nhóm chức năng của hệ thống

1.2.3. Sơ đồ Use-Case tổng quát



Hình 1: Sơ đồ Use-Case tổng quát của hệ thống Pet Clinic

1.2.4. Đặc tả Use-Case

UC1.1 Tạo mới hồ sơ khách hàng

- **Mô tả ngắn gọn:** Cho phép lễ tân tạo hồ sơ khách hàng mới với thông tin cơ bản.

- **Tác nhân thực hiện:** Lễ tân, Khách hàng (tự đăng ký)
- **Luồng sự kiện:**
  - Luồng chính:
    - Tác nhân chọn "Thêm khách hàng"
    - Hệ thống hiển thị form: họ tên, email, số điện thoại, địa chỉ
    - Tác nhân nhập thông tin và chọn "Lưu khách hàng"
    - Hệ thống kiểm tra hợp lệ, lưu và thông báo "Thêm khách hàng thành công"
  - Luồng phát sinh:
    - Bỏ trống trường bắt buộc, email sai định dạng hoặc SĐT không đủ 10 số: hệ thống báo lỗi và yêu cầu nhập lại
    - SĐT/email đã tồn tại: hệ thống báo lỗi trùng
- **Tiền điều kiện:** Người dùng có quyền Lễ tân
- **Hậu điều kiện:** Hồ sơ khách hàng được lưu với mã định danh duy nhất

UC1.2 Tạo mới hồ sơ thú cưng

- **Mô tả ngắn gọn:** Tạo hồ sơ thú cưng và liên kết với khách hàng đã có.
- **Tác nhân thực hiện:** Lễ tân, Quản trị viên
- **Luồng sự kiện:**
  - Luồng chính:
    - Tác nhân chọn "Thêm thú cưng" tại dòng khách hàng
    - Hệ thống hiển thị form: tên, loài, giống, giới tính, tuổi, cân nặng, microchip
    - Tác nhân nhập và chọn "Lưu thú cưng"
    - Hệ thống lưu và thông báo "Thêm thú cưng thành công"
  - Luồng phát sinh:
    - Bỏ trống tên hoặc loài: hệ thống báo lỗi
    - Microchip đã tồn tại: hệ thống báo "Microchip đã tồn tại, vui lòng kiểm tra lại."
- **Tiền điều kiện:** Khách hàng đã tồn tại
- **Hậu điều kiện:** Thú cưng được lưu, liên kết đúng khách hàng

UC1.5 Tìm kiếm khách hàng / thú cưng

- **Mô tả ngắn gọn:** Tìm khách hàng theo tên, email, số điện thoại; tìm thú cưng theo tên chủ.
- **Tác nhân thực hiện:** Lễ tân
- **Luồng sự kiện:**
  - Luồng chính:
    - Tác nhân nhập từ khóa vào ô tìm kiếm
    - Hệ thống lọc và hiển thị danh sách phù hợp
  - Luồng phát sinh:
    - Không có kết quả: hiển thị "Không có khách hàng phù hợp."
- **Tiền điều kiện:** Người dùng đã vào trang Khách hàng
- **Hậu điều kiện:** Danh sách được lọc theo từ khóa

UC2.1 Đặt lịch khám

- **Mô tả ngắn gọn:** Tạo lịch hẹn khám/chăm sóc cho thú cưng.
- **Tác nhân thực hiện:** Lễ tân, Khách hàng
- **Luồng sự kiện:**
  - Luồng chính:
    - Tác nhân chọn "Tạo lịch hẹn"
    - Chọn khách hàng, thú cưng, bác sĩ (không bắt buộc), ngày, giờ, dịch vụ
    - Chọn lưu
    - Hệ thống kiểm tra trùng lịch và thông báo "Đặt lịch thành công"
  - Luồng phát sinh:
    - Bác sĩ hoặc thú cưng đã có lịch cùng khung giờ: báo "Không thể đặt lịch"
    - Thiếu thông tin: báo lỗi từng trường
- **Tiền điều kiện:** Khách hàng và thú cưng đã tồn tại
- **Hậu điều kiện:** Lịch hẹn được lưu ở trạng thái "Chờ xác nhận"

UC2.2 Xác nhận / Hủy / Hoàn thành lịch hẹn

- **Mô tả ngắn gọn:** Cập nhật trạng thái lịch hẹn.
- **Tác nhân thực hiện:** Lễ tân, Bác sĩ
- **Luồng sự kiện:**

- Luồng chính:
  - Tác nhân mở tab "Quản lý toàn bộ"
  - Chọn "Xác nhận", "Hoàn thành" hoặc "Hủy lịch"
  - Hệ thống cập nhật trạng thái và thông báo
- Luồng phát sinh:
  - Hủy khi còn dưới 1 tiếng trước giờ hẹn: hệ thống từ chối (R2.2)

- **Tiền điều kiện:** Lịch hẹn đã tồn tại
- **Hậu điều kiện:** Trạng thái lịch hẹn được cập nhật

UC2.4 Quản lý ca làm việc bác sĩ

- **Mô tả ngắn gọn:** Tạo, xóa ca trực của bác sĩ, tránh trùng lịch.
- **Tác nhân thực hiện:** Quản trị viên
- **Luồng sự kiện:**
  - Luồng chính:
    - Chọn "Tạo ca trực"
    - Chọn bác sĩ, ngày trực, giờ bắt đầu, giờ kết thúc
    - Chọn "Lưu ca trực"
    - Hệ thống thông báo "Đã tạo ca trực"
  - Luồng phát sinh:
    - Không chọn bác sĩ/ngày: báo "Vui lòng chọn bác sĩ và ngày trực"
    - Giờ không hợp lệ hoặc trùng ca: hệ thống từ chối
- **Tiền điều kiện:** Bác sĩ đã tồn tại
- **Hậu điều kiện:** Ca trực được lưu

UC3.1 Tạo hồ sơ khám

- **Mô tả ngắn gọn:** Ghi nhận buổi khám: triệu chứng, chẩn đoán, điều trị.
- **Tác nhân thực hiện:** Bác sĩ
- **Luồng sự kiện:**
  - Luồng chính:
    - Mở tab "Tạo hồ sơ mới"
    - Chọn thú cưng, bác sĩ; nhập triệu chứng, chẩn đoán, phác đồ điều trị
    - Chọn "Lưu hồ sơ"
    - Hệ thống thông báo "Lưu hồ sơ khám thành công"

- Luồng phát sinh:
    - Thiếu trường bắt buộc: hiển thị lỗi tương ứng
- **Tiền điều kiện:** Thú cưng đã tồn tại
- **Hậu điều kiện:** Hồ sơ khám được lưu và hiển thị trong danh sách

UC4.1 / UC4.3 Nhập kho và cảnh báo tồn kho

- **Mô tả ngắn gọn:** Thêm thuốc/vật tư kèm lô và hạn dùng; cảnh báo tồn thấp.
- **Tác nhân thực hiện:** Quản trị viên
- **Luồng sự kiện:**
    - Luồng chính:
        - Chọn "Thêm vật tư"
        - Nhập mã, tên, danh mục, đơn vị, tồn kho, tồn tối thiểu, giá, hạn dùng
        - Chọn "Lưu vật tư"
        - Hệ thống lưu vật tư và lô hàng; nếu tồn ≤ tối thiểu hiển thị "Cảnh báo tồn kho thấp"
    - Luồng phát sinh:
        - Thiếu thông tin hoặc tồn kho âm: báo lỗi
        - Mã vật tư trùng: hệ thống từ chối
- **Tiền điều kiện:** Người dùng có quyền quản lý kho
- **Hậu điều kiện:** Vật tư được lưu vào kho

UC5.1 / UC5.2 Tạo hóa đơn và thanh toán

- **Mô tả ngắn gọn:** Lập hóa đơn bán thuốc, tự trừ tồn kho, xác nhận thu tiền.
- **Tác nhân thực hiện:** Kế toán, Lễ tân
- **Luồng sự kiện:**
    - Luồng chính:
        - Chọn "Tạo hóa đơn"
        - Chọn khách hàng, thuốc, số lượng
        - Chọn "Tạo hóa đơn": hệ thống tạo hóa đơn và trừ tồn
        - Chọn "Xác nhận đã thu tiền": hóa đơn chuyển "Đã thanh toán"
    - Luồng phát sinh:
        - Chưa chọn khách hàng/thuốc: báo lỗi

        - Số lượng vượt tồn kho: báo "Không thể tạo hóa đơn", không phát sinh hóa đơn
- **Tiền điều kiện:** Khách hàng và thuốc đã tồn tại
- **Hậu điều kiện:** Hóa đơn được lưu, tồn kho giảm tương ứng

UC6.1 Quản lý thông tin bác sĩ

- **Mô tả ngắn gọn:** Thêm, cập nhật bác sĩ và trạng thái nhận lịch.
- **Tác nhân thực hiện:** Quản trị viên
- **Luồng sự kiện:**
    - Luồng chính:
        - Chọn "Thêm bác sĩ"
        - Nhập họ tên, email, SĐT, chuyên khoa, mã hành nghề, kinh nghiệm, phí khám
        - Chọn "Tạo bác sĩ"
        - Hệ thống thông báo "Đã thêm bác sĩ mới"
    - Luồng phát sinh:
        - Thiếu thông tin hoặc email sai: báo "Không thể thêm bác sĩ"
- **Tiền điều kiện:** Người dùng có quyền Quản trị
- **Hậu điều kiện:** Bác sĩ được lưu, xuất hiện trong danh sách

UC8.2 Theo dõi thú cưng qua camera

- **Mô tả ngắn gọn:** Xem trực tiếp camera của phòng lưu trú.
- **Tác nhân thực hiện:** Khách hàng, Bác sĩ, Lễ tân
- **Luồng sự kiện:**
    - Luồng chính:
        - Chọn "Camera giám sát"
        - Hệ thống hiển thị danh sách camera theo phòng và trạng thái
        - Chọn camera để xem trực tiếp
    - Luồng phát sinh:
        - Camera ngoại tuyến: hiển thị "Camera đang ngoại tuyến"
- **Tiền điều kiện:** Thú cưng đang lưu trú có camera
- **Hậu điều kiện:** Người dùng xem được luồng video

1.3. Các yêu cầu phi chức năng

1.3.1. Giao diện người dùng

- **Thân thiện, dễ sử dụng:** menu điều hướng cố định bên trái, mỗi chức năng một trang riêng.
- **Responsive:** giao diện desktop cho lễ tân, giao diện co giãn cho thiết bị di động.
- **Đồng nhất:** dùng chung bộ thành phần giao diện (shadcn/ui), hỗ trợ chế độ sáng/tối.
- **Thông báo rõ ràng:** kiểm tra dữ liệu tại form và thông báo (toast) sau mỗi thao tác.

1.3.2. Hiệu suất và bảo mật

- **Thời gian phản hồi:** các thao tác thông thường ≤ 3 giây khi có 200 người dùng đồng thời.
- **Tìm kiếm:** tìm khách hàng/thú cưng trên khoảng 10.000 bản ghi trả về ≤ 2 giây.
- **Bảo mật:** mật khẩu băm bcrypt/argon2, giao tiếp HTTPS (TLS 1.2+), xác thực JWT/OAuth2, phân quyền RBAC.
- **Độ tin cậy:** uptime ≥ 99%, sao lưu CSDL hằng ngày.

2. Chương 2: Kế hoạch kiểm thử

2.1. Giới thiệu

2.1.1. Mục đích

Tài liệu kế hoạch kiểm thử này đưa ra các mục đích sau:

- Xác định thông tin cơ bản về dự án và các thành phần chức năng được kiểm thử và không được kiểm thử.
- Liệt kê những yêu cầu cho việc kiểm thử (Test Requirements).
- Những chiến lược kiểm thử nên được sử dụng.
- Ước lượng những yêu cầu về tài nguyên cho việc kiểm thử.
- Những tài liệu được lập sau khi hoàn thành việc kiểm thử.

2.1.2. Tổng quan

Báo cáo cung cấp cái nhìn tổng quan về phạm vi dự án, các yêu cầu, chức năng và công việc cần kiểm thử. Dự án phát triển hệ thống quản lý phòng khám thú cưng gồm các nhóm chức năng: quản lý khách hàng và thú cưng, lịch hẹn, ca bác sĩ, hồ sơ khám, kho thuốc, hóa đơn – thanh toán, báo cáo và camera. Mỗi nhóm chức năng được kiểm thử để đảm bảo hệ thống hoạt động đúng yêu cầu.

2.1.3. Phạm vi

Tài liệu áp dụng cho việc kiểm thử phiên bản Pet Clinic Pro (MySQL Edition) chạy cục bộ tại http://localhost:5000. Các dịch vụ và chức năng cần kiểm tra bao gồm:

- Quản lý khách hàng và thú cưng
- Đặt lịch và xử lý lịch hẹn, ca làm việc bác sĩ
- Hồ sơ khám bệnh
- Kho thuốc, hóa đơn và thanh toán
- Quản lý bác sĩ, báo cáo doanh thu, camera giám sát

2.1.4. Các định nghĩa và các từ viết tắt được dùng

2.1.4.1. Các định nghĩa

Thuật ngữ	Định nghĩa
Pet Clinic	Hệ thống quản lý phòng khám thú cưng được kiểm thử
Khách hàng (Customer)	Chủ sở hữu thú cưng
Lịch hẹn (Appointment)	Thời gian đặt khám/chăm sóc cho một thú cưng
Hồ sơ khám (Medical Record)	Thông tin một buổi khám: triệu chứng, chẩn đoán, điều trị

Thuật ngữ	Định nghĩa
Lô hàng (Stock Batch)	Lô nhập thuốc kèm số lượng và hạn dùng
Ca trực (Doctor Shift)	Khoảng thời gian làm việc của bác sĩ trong một ngày

Bảng 2: Bảng liệt kê các định nghĩa được sử dụng

2.1.4.2. Các từ viết tắt

Từ viết tắt	Mô tả
TC	Test Case (Trường hợp kiểm thử)
UT	Unit Test (Kiểm thử đơn vị)
UC	Use Case (Ca sử dụng)
GUI	Graphical User Interface (Giao diện người dùng)
API	Application Programming Interface
RBAC	Role-Based Access Control (Phân quyền theo vai trò)
CSDL	Cơ sở dữ liệu
KH	Khách hàng
SĐT	Số điện thoại

Bảng 3: Bảng liệt kê các từ viết tắt được sử dụng

2.1.5. Những người sử dụng tài liệu này

- **Test Manager:** quản lý toàn bộ quy trình kiểm thử, đảm bảo kế hoạch được thực hiện đúng tiến độ.
- **Test Designer:** thiết kế các test case, xác định chiến lược và phương pháp kiểm thử.
- **Tester:** thực hiện kiểm thử theo test case, báo cáo lỗi và theo dõi quá trình sửa lỗi.
- **Nhóm phát triển:** tiếp nhận danh sách lỗi để khắc phục.

2.2. Những yêu cầu về tài nguyên

2.2.1. Phần cứng

Để đảm bảo quá trình kiểm thử hệ thống diễn ra hiệu quả, máy kiểm thử cần đáp ứng cấu hình tối thiểu sau (máy chạy đồng thời máy chủ ứng dụng, CSDL và trình duyệt):

Thành phần	Cấu hình tối thiểu	Cấu hình khuyến nghị
CPU	Intel Core i3 / AMD Ryzen 3, 2.0 GHz, 2 nhân	Intel Core i5 / AMD Ryzen 5, 4 nhân trở lên
RAM	8 GB	16 GB
Ổ cứng trống	10 GB	20 GB (SSD)

Thành phần	Cấu hình tối thiểu	Cấu hình khuyến nghị
Kiến trúc	64 bit	64 bit
Màn hình	1366 × 768	1920 × 1080
Mạng	Kết nối Internet để tải thư viện và driver trình duyệt	Băng thông ≥ 10 Mbps

Bảng 4: Bảng yêu cầu về tài nguyên phần cứng

2.2.2. Phần mềm

Các phần mềm cần thiết để hỗ trợ quá trình kiểm thử bao gồm:

Tên phần mềm	Phiên bản	Loại
Microsoft Windows / macOS / Ubuntu	Windows 10 trở lên, macOS 12 trở lên, Ubuntu 20.04 trở lên	Hệ điều hành
Google Chrome	Phiên bản ổn định mới nhất (≥ 120)	Trình duyệt
Microsoft Edge	Phiên bản ổn định mới nhất (≥ 120)	Trình duyệt
Mozilla Firefox	Phiên bản ổn định mới nhất (≥ 115)	Trình duyệt
Node.js	20 LTS trở lên	Môi trường chạy ứng dụng và test
MySQL Community Server	8.0 trở lên	Hệ quản trị CSDL
Pet Clinic Pro	1.0.0 (MySQL Edition)	Phần mềm được kiểm thử

Bảng 5: Bảng yêu cầu về tài nguyên phần mềm

2.2.3. Công cụ kiểm thử

Các công cụ được sử dụng để quản lý, thực hiện và theo dõi quá trình kiểm thử:

Hoạt động	Công cụ	Nhà cung cấp	Phiên bản
Kiểm thử đơn vị (Unit Test)	Mocha + tsx + node:assert	OpenJS Foundation	Mocha 10.x
Kiểm thử tự động giao diện	Mocha + Selenium WebDriver (chromedriver, msedgedriver, geckodriver)	OpenJS / SeleniumHQ	selenium-webdriver 4.x
Quản lý Test Case	Microsoft Word / Excel	Microsoft	2019 trở lên
Defect Tracking	Microsoft Excel	Microsoft	2019 trở lên
Kiểm thử hiệu năng: tải, chịu tải, chạy liên tục	autocannon (bộ sinh tải HTTP)	Matteo Collina / OSS	8.x
Kiểm thử toàn vẹn dữ liệu, độ tin cậy	Mocha + mysql2 (đối chiếu trực tiếp CSDL)	OpenJS / sidorares	mysql2 3.x
Biểu đồ kết quả	Python Matplotlib	Matplotlib	3.x

Bảng 6: Bảng liệt kê các công cụ kiểm thử

2.2.4. Môi trường kiểm thử

- Máy tính cá nhân đáp ứng cấu hình ở mục 2.2.1, cài một trong các hệ điều hành ở mục 2.2.2.

- Trình duyệt kiểm thử: Google Chrome, Microsoft Edge, Mozilla Firefox bản mới nhất. Kiểm thử tự động chạy ở chế độ headless, cửa sổ 1440 × 900; chọn trình duyệt bằng biến môi trường BROWSER.
- Ứng dụng chạy tại http://localhost:5000, tắt nhắc lịch tự động để không gửi Email/SMS thật.
- CSDL MySQL riêng cho kiểm thử (không dùng CSDL thật), khởi tạo bằng npm run db:setup và nạp lại dữ liệu mẫu trước mỗi lượt chạy.
- Dữ liệu mẫu được sinh bởi server/seed.ts (npm run db:seed), có quy luật cố định để kết quả kiểm thử lặp lại được:

Bảng dữ liệu	Số bản ghi	Đặc điểm phục vụ kiểm thử
Bác sĩ	12	3 bác sĩ đang ở trạng thái bận (không nhận lịch)
Khách hàng	30	27 đang hoạt động, 3 chờ duyệt (Huỳnh Hoàng Nam, Ngô Mỹ Duyên, Huỳnh Bích Ngọc)
Thú cưng	50	Chó, mèo, thỏ, chim, sóc; mỗi con có mã microchip riêng
Thuốc / vật tư	30	4 mặt hàng dưới tồn tối thiểu: Doxycycline, Thuốc nhỏ mắt, Vaccine mèo 4 bệnh, Ống truyền dịch
Lô hàng	30	Một số lô hết hạn trong 30 ngày tới
Lịch hẹn	60	Từ 10 ngày trước đến 14 ngày tới, đủ trạng thái; để trống khung 09:00, 11:00, 15:00 cho test case
Hồ sơ khám	43	10 loại chẩn đoán; "Viêm da dị ứng" có 4 hồ sơ
Camera	15	12 trực tuyến; Phòng 04, 09, 13 ngoại tuyến
Ca trực	31	Ca sáng 08:00–12:00, ca chiều 13:30–17:30 trong 7 ngày tới
Hóa đơn	60	45 đã thanh toán, 5 chờ thanh toán, 5 quá hạn, 5 đã hủy

Bảng 7: Bảng dữ liệu mẫu dùng cho kiểm thử

2.3. Phạm vi kiểm thử

2.3.1. Những chức năng được kiểm thử

- **Hệ thống:** truy cập, điều hướng menu, trang 404, giao diện sáng/tối, đăng nhập và phân quyền.
- **Khách hàng & thú cưng:** kiểm tra dữ liệu nhập (bắt buộc, định dạng email, giá trị biên SĐT), trùng SĐT/email/microchip, chỉnh sửa, khóa/duyệt, tìm kiếm.
- **Lịch hẹn:** đặt lịch hợp lệ, trùng giờ bác sĩ/thú cưng, ngày quá khứ, xác nhận/hoàn thành/hủy, quy tắc hủy trước 1 tiếng.
- **Ca bác sĩ, hồ sơ khám, camera:** tạo/xóa ca trực, giờ không hợp lệ, trùng ca; tạo và tra cứu hồ sơ; lọc camera.

- **Quản trị / Kế toán:** quản lý bác sĩ; nhập kho, tồn kho âm, trùng mã, cảnh báo tồn thấp; tạo hóa đơn, trừ tồn, bán vượt tồn, thanh toán; doanh thu trên trang Tổng quan.
- **Phi chức năng:** hiệu năng (tải 200 người dùng, tìm kiếm trên 10.000 bản ghi), chịu tải (tìm ngưỡng quá tải), toàn vẹn dữ liệu (thao tác đồng thời, nhất quán giữa các bảng, dữ liệu đặc biệt), độ tin cậy và phục hồi (mất CSDL, tắt đột ngột, chạy liên tục).
- **Mức đơn vị:** các schema kiểm tra dữ liệu (shared/schema.ts) và lớp nghiệp vụ DatabaseStorage (server/storage.ts).

2.3.2. Những chức năng không được kiểm thử

- Gửi nhắc lịch thực qua Email/SMS (cần cấu hình SMTP, Twilio).
- Phát luồng video camera thực (luồng HLS bên ngoài).
- Trang cửa hàng (Shop, Cart, Product Detail) – chưa được gắn vào điều hướng của ứng dụng.
- Kiểm thử hiệu năng tải lớn và kiểm thử xâm nhập chuyên sâu – đề xuất cho giai đoạn sau.

2.4. Chiến lược kiểm thử

2.4.1. Các hình thức kiểm thử và các bước kiểm thử

2.4.1.1. Kiểm thử hộp đen

Định nghĩa: kiểm thử hộp đen không quan tâm đến mã nguồn, chỉ tập trung vào đầu vào và đầu ra để kiểm tra phần mềm có hoạt động đúng đặc tả hay không. Phương pháp này phù hợp với các chức năng như tạo khách hàng, đặt lịch, tạo hóa đơn dưới góc độ người dùng. Nhóm áp dụng kỹ thuật phân vùng tương đương và phân tích giá trị biên (SĐT 9/10/11 chữ số, tuổi âm, tồn kho âm, số lượng vượt tồn).

Các bước kiểm thử:

- **Bước 1 – Lập kế hoạch:** xây dựng tài liệu yêu cầu, chọn công cụ, tìm hiểu chức năng cần kiểm tra.
- **Bước 2 – Thiết kế test case:** xác định dữ liệu đầu vào, các bước và kết quả mong đợi.
- **Bước 3 – Thực thi:** chạy test case, ghi nhận Pass/Fail và lỗi nếu có.
- **Bước 4 – Báo cáo:** tổng hợp kết quả, tỷ lệ lỗi, chuyển cho nhóm phát triển.
- **Bước 5 – Kiểm thử lại:** sau khi lỗi được sửa, chạy lại để xác nhận.

2.4.1.2. Kiểm thử hộp trắng

Định nghĩa: kiểm thử hộp trắng dựa trên cấu trúc bên trong và mã nguồn. Nhóm đọc mã server/storage.ts và shared/schema.ts, viết unit test cho từng nhánh điều kiện: kiểm tra trùng lịch của bác sĩ và thú cưng, trừ tồn kho trong transaction, chặn tồn kho âm, xóa mềm lịch hẹn.

- **Bước 1:** phân tích mã nguồn và các nhánh điều kiện.
- **Bước 2:** thiết kế test case bao phủ từng nhánh (hợp lệ / ném lỗi).
- **Bước 3:** thực thi bằng Mocha, ghi nhận kết quả.
- **Bước 4:** báo cáo và kiểm thử lại sau khi sửa.

2.4.1.3. Kiểm thử thủ công

Định nghĩa: người kiểm thử thao tác trực tiếp trên giao diện (nhấn nút, nhập liệu) để quan sát kết quả. Nhóm dùng kiểm thử thủ công để khám phá giao diện, xác định các bước và thông báo thực tế trước khi tự động hóa.

- **Bước 1:** chuẩn bị môi trường và dữ liệu.
- **Bước 2:** thực thi thủ công, quan sát và ghi nhận.
- **Bước 3:** báo cáo lỗi.
- **Bước 4:** kiểm thử lại.

2.4.1.4. Kiểm thử tự động

Định nghĩa: kiểm thử bằng công cụ phần mềm. Toàn bộ 74 test case chức năng được tự động hóa bằng Mocha và Selenium WebDriver, chạy được trên Chrome, Edge và Firefox, cho phép chạy lại nhanh sau mỗi lần sửa lỗi.

- **Bước 1:** chọn công cụ Mocha, Selenium WebDriver.
- **Bước 2:** viết kịch bản cho từng test case.
- **Bước 3:** thực thi kịch bản, so sánh với kết quả mong đợi.
- **Bước 4:** ghi nhận và báo cáo.
- **Bước 5:** kiểm thử lại (hồi quy).

2.4.2. Các loại kiểm thử

2.4.2.1. Kiểm thử chức năng

Đảm bảo các chức năng hoạt động đúng đặc tả; dựa trên kiểm thử hộp đen.

- **Kỹ thuật:** bao phủ cả dữ liệu hợp lệ và không hợp lệ; với dữ liệu không hợp lệ hệ thống phải đưa ra thông báo rõ ràng.

- **Điều kiện dừng:** thực thi hết test case đã thiết kế, mọi lỗi được ghi nhận chi tiết.
- **Chịu trách nhiệm:** Test Designer, Tester.

2.4.2.2. Kiểm thử hiệu năng

Mục đích: đo thời gian phản hồi và thông lượng của hệ thống, đối chiếu với yêu cầu phi chức năng ở mục 1.3.2: thao tác thông thường ≤ 3 giây khi có 200 người dùng đồng thời; tìm kiếm trên khoảng 10.000 bản ghi ≤ 2 giây.

Công cụ đo: autocannon – bộ sinh tải HTTP viết bằng Node.js, giả lập nhiều kết nối đồng thời gửi yêu cầu liên tục tới API và ghi nhận phân phối thời gian phản hồi. Có thể thay bằng các công cụ tương đương: Apache JMeter, k6, Locust.

Các chỉ số đo:

- **Thời gian phản hồi (latency):** trung bình, p50 (trung vị), p90, p99 (99% yêu cầu nhanh hơn giá trị này), lớn nhất.
- **Thông lượng (throughput):** số yêu cầu xử lý mỗi giây (req/s).
- **Tỷ lệ lỗi:** phần trăm yêu cầu bị lỗi kết nối, quá thời gian chờ (10 giây) hoặc trả mã khác 2xx.
- **Tài nguyên:** bộ nhớ (RAM) của tiến trình ứng dụng khi chạy tải liên tục.

Lưu ý môi trường đo: máy sinh tải, máy chủ ứng dụng và CSDL chạy trên cùng một máy kiểm thử theo cấu hình ở mục 2.2. Số liệu vì vậy mang tính tham khảo và cần đo lại trên hạ tầng triển khai thật.

2.4.2.3. Kiểm thử tải (Load Testing)

- **Mục đích:** kiểm tra hệ thống đáp ứng đúng yêu cầu ở mức tải dự kiến – 200 người dùng đồng thời.
- **Kỹ thuật:** autocannon giữ 200 kết nối đồng thời trong 30 giây cho từng kịch bản: trang Tổng quan, danh sách khách hàng, lịch hẹn, kho thuốc, kịch bản đọc hỗn hợp nhiều màn hình và kịch bản ghi (tạo hồ sơ khách hàng). Riêng tìm kiếm được đo trên 10.000 khách hàng.
- **Tiêu chí đạt:** p99 ≤ 3.000 ms (tìm kiếm ≤ 2.000 ms) và tỷ lệ lỗi 0%.
- **Chịu trách nhiệm:** Test Designer, Tester.

2.4.2.4. Kiểm thử chịu tải (Stress Testing)

- **Mục đích:** xác định giới hạn tối đa hệ thống chịu được, cách hệ thống phản ứng khi quá tải và khả năng hồi phục khi tải giảm.

- **Kỹ thuật:** tăng dần số kết nối 50 → 100 → 200 → 400 → 800 → 1.200 → 1.600, mỗi mức 15 giây với kịch bản hỗn hợp; sau đó giảm về 50 kết nối để kiểm tra hồi phục.
- **Ngưỡng quá tải:** mức đầu tiên có p99 > 3 giây hoặc tỷ lệ lỗi > 1%.
- **Tiêu chí đạt:** không quá tải ở mức ≤ 200 kết nối; máy chủ không sập; hồi phục hoàn toàn khi tải giảm.

2.4.2.5. Kiểm thử toàn vẹn dữ liệu (Data Integrity Testing)

- **Mục đích:** đảm bảo dữ liệu luôn chính xác và nhất quán: khi nhiều người thao tác cùng lúc, giữa các bảng liên quan (hóa đơn – dòng hóa đơn – tồn kho, thú cưng – khách hàng), với dữ liệu đặc biệt và dữ liệu độc hại.
- **Kỹ thuật:** gửi yêu cầu đồng thời để phát hiện race condition (bán thuốc, đặt lịch); đối chiếu trực tiếp trong CSDL bằng truy vấn SQL (tổng tiền, tồn kho âm, bản ghi mồ côi); lưu rồi đọc lại tiếng Việt, emoji, ký tự HTML, chuỗi rất dài, chuỗi SQL Injection.
- **Công cụ:** Mocha + mysql2 (kết nối CSDL để đối chiếu).
- **Tiêu chí đạt:** không bán vượt tồn kho, không trùng lịch, không có bản ghi lệch hoặc mồ côi, dữ liệu đọc lại giống hệt dữ liệu đã ghi.

2.4.2.6. Kiểm thử độ tin cậy và phục hồi (Reliability & Recovery Testing)

- **Mục đích:** đánh giá hệ thống hoạt động ổn định lâu dài và tự phục hồi sau sự cố, gắn với yêu cầu uptime ≥ 99%.
- **Kỹ thuật:** bộ test tự khởi chạy một tiến trình ứng dụng riêng rồi chủ động gây sự cố: gửi yêu cầu sai định dạng; dừng MySQL khi ứng dụng đang chạy rồi bật lại; tắt đột ngột ứng dụng (kill) rồi khởi động lại; chạy tải liên tục 180 giây và theo dõi bộ nhớ.
- **Tiêu chí đạt:** ứng dụng không sập khi mất CSDL; tự phục hồi ≤ 30 giây khi CSDL trở lại; không mất dữ liệu đã ghi; khởi động lại ≤ 10 giây; chạy liên tục không lỗi, bộ nhớ không tăng liên tục.

2.4.2.7. Kiểm thử bảo mật

Đảm bảo dữ liệu khách hàng và hồ sơ khám được bảo vệ. Nhóm kiểm tra việc truy cập chức năng khi chưa đăng nhập (SYS-TC004). Kết quả cho thấy hệ thống chưa có xác thực và phân quyền, đây là lỗi mức Cao. Đề xuất bổ sung kiểm thử SQL Injection, XSS bằng Burp Suite/OWASP ZAP sau khi có cơ chế đăng nhập.

2.5. Điều kiện chấp thuận

- **Hoàn thành test case:** 100% test case được thực thi; không còn lỗi mức Cao; tỷ lệ Pass ≥ 90%.
- **Ổn định:** hệ thống chạy ổn định trên Chrome, Edge và Firefox, không treo khi thao tác liên tục.
- **Bảo mật:** có đăng nhập và phân quyền theo vai trò trước khi triển khai thực tế.
- **Hiệu năng:** p99 ≤ 3 giây với 200 người dùng đồng thời; tìm kiếm ≤ 2 giây trên 10.000 bản ghi; không lỗi ở mức tải dự kiến.
- **Toàn vẹn và tin cậy:** không mất, không sai lệch dữ liệu khi thao tác đồng thời; tự phục hồi sau sự cố CSDL.
- **Khả năng bảo trì:** bộ test tự động chạy lại được bằng một lệnh để kiểm thử hồi quy.

2.6. Defect Tracking (Lỗi)

2.6.1. Phân loại lỗi

Mức độ nghiêm trọng	Đặc tả lỗi
Cao	Chức năng chính không sử dụng được (không thêm được khách hàng, không tìm kiếm được); ô nhập liệu không nhập được; lỗ hổng bảo mật (không có đăng nhập).
Trung bình	Vi phạm quy tắc nghiệp vụ nhưng vẫn thao tác được: dữ liệu trùng, lịch hẹn quá khứ, ca trực sai giờ, hóa đơn rỗng.
Thấp	Thông báo lỗi thiếu hoặc khó hiểu, bộ lọc không có kết quả, lỗi hiển thị giao diện.

Bảng 8: Bảng phân loại mức độ nghiêm trọng của lỗi

2.6.2. Quy trình xử lý lỗi

Bước 1: **Ghi nhận lỗi:** mô tả lỗi, test case gây lỗi, các bước tái hiện và kết quả thực tế.

Bước 2: **Phân loại lỗi:** theo mức độ Cao, Trung bình, Thấp.

Bước 3: **Phân công xử lý:** chuyển cho thành viên phụ trách chức năng.

Bước 4: **Sửa lỗi:** nhóm phát triển sửa và chuyển lại cho Tester.

Bước 5: **Kiểm thử lại:** chạy lại test case liên quan và toàn bộ bộ test tự động để phát hiện lỗi hồi quy.

Bước 6: **Báo cáo lỗi:** cập nhật số lượng, mức độ và trạng thái lỗi cho các bên.

Bước 7: **Đánh giá và tổng kết:** rút kinh nghiệm cho các phiên bản sau.

3. Chương 3: Xây dựng các test case

Nhóm xây dựng 74 test case chức năng, chia theo tác nhân như bảng dưới đây. Cột Status là kết quả thực thi thực tế bằng bộ kiểm thử tự động ở Chương 5 (lần chạy ngày 02/10/2026).

Nhóm	Số test case	Pass	Fail
Chức năng của HỆ THỐNG	5	4	1
Chức năng của LỄ TÂN	34	25	9
Chức năng của BÁC SĨ	15	12	3
Chức năng của QUẢN TRỊ VIÊN / KẾ TOÁN	20	15	5
Tổng cộng	74	56	18

Bảng 9: Bảng tổng hợp số lượng test case theo tác nhân

3.1. Chức năng của HỆ THỐNG

3.1.1. Truy cập hệ thống & trang Tổng quan

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
SYS-TC001	Truy cập trang chủ của hệ thống	1. Mở trình duyệt, truy cập địa chỉ hệ thống	Trang web được truy cập	http://localhost:5000	Trang "Tổng quan vận hành phòng khám" hiển thị đủ 6 thẻ thống kê	Pass
		2. Quan sát trang Tổng quan	Hiển thị tiêu đề và 6 thẻ thống kê			
SYS-TC002	Điều hướng tới các chức năng qua thanh menu bên trái	1. Truy cập trang chủ	Trang Tổng quan được hiển thị		Mỗi mục menu mở đúng trang chức năng tương ứng	Pass
		2. Lần lượt chọn từng mục trên menu	Trang tương ứng được hiển thị	Bác sĩ, Khách hàng, Lịch hẹn, Hồ sơ khám, Ca bác sĩ, Kho thuốc, Hóa đơn, Camera giám sát		
SYS-TC003	Truy cập đường dẫn không tồn tại	1. Nhập đường dẫn không tồn tại trên thanh địa chỉ	Hệ thống xử lý đường dẫn	/trang-khong-ton-tai	Hiển thị trang "404 - Không tìm thấy trang"	Pass

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
SYS-TC005	Chuyển đổi giao diện sáng/tối	1. Truy cập trang chủ	Giao diện sáng		Giao diện chuyển đổi qua lại giữa sáng và tối	Pass
		2. Nhấn nút đổi giao diện ở góc phải	Giao diện chuyển sang tối			
		3. Nhấn lại nút	Giao diện trở lại sáng			

3.1.2. Đăng nhập & phân quyền (RBAC – R6.2)

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
SYS-TC004	Truy cập chức năng quản lý khi chưa đăng nhập	1. Mở trình duyệt mới (chưa đăng nhập)	Không có phiên đăng nhập		Hệ thống yêu cầu đăng nhập, không cho xem dữ liệu khách hàng	Fail
		2. Truy cập trực tiếp trang Khách hàng	Hệ thống chuyển tới trang đăng nhập	/customers		

3.2. Chức năng của LỄ TÂN

3.2.1. UC1.1 Tạo mới hồ sơ khách hàng

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC1.1-TC001	Bỏ trống tất cả các trường và nhấn "Lưu khách hàng"	1. Chọn "Thêm khách hàng"	Form thêm khách hàng hiển thị		Hiển thị lỗi "Họ tên là bắt buộc", "Email không hợp lệ", "Số điện thoại phải gồm đúng 10 chữ số"	Pass
		2. Bỏ trống Họ tên, Email, SĐT				
		3. Nhấn "Lưu khách hàng"	Hiển thị thông báo lỗi			
UC1.1-TC002	Bỏ trống Họ tên, các trường khác hợp lệ	1. Chọn "Thêm khách hàng"	Form hiển thị		Hiển thị lỗi "Họ tên là bắt buộc"	Pass
		2. Bỏ trống Họ tên				

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	**Step Expected Result**			
		3. Nhập Email, SĐT hợp lệ		a@test.vn / 0987654321		
		4. Nhấn "Lưu khách hàng"	Hiển thị thông báo lỗi			
UC1.1-TC003	Email sai định dạng	1. Chọn "Thêm khách hàng"	Form hiển thị		Hiển thị thông báo email không hợp lệ, không lưu khách hàng	**Pass**
		2. Nhập Họ tên, SĐT hợp lệ		Lê Minh / 0987654321		
		3. Nhập Email sai định dạng		leminh.gmail.com		
		4. Nhấn "Lưu khách hàng"	Hiển thị thông báo lỗi			
UC1.1-TC004	Kiểm thử giá trị biên: SĐT 9 chữ số	1. Chọn "Thêm khách hàng"	Form hiển thị		Hiển thị lỗi "Số điện thoại phải gồm đúng 10 chữ số"	**Pass**
		2. Nhập Họ tên, Email hợp lệ		Lê Minh / leminh@test.vn		
		3. Nhập SĐT 9 chữ số		098765432		
		4. Nhấn "Lưu khách hàng"	Hiển thị thông báo lỗi			
UC1.1-TC005	Kiểm thử giá trị biên: SĐT 11 chữ số	1. Chọn "Thêm khách hàng"	Form hiển thị		Hiển thị lỗi "Số điện thoại phải gồm đúng 10 chữ số"	**Pass**
		2. Nhập Họ tên, Email hợp lệ		Lê Minh / leminh@test.vn		
		3. Nhập SĐT 11 chữ số		09876543210		
		4. Nhấn "Lưu khách hàng"	Hiển thị thông báo lỗi			
UC1.1-TC006	SĐT chứa ký tự chữ	1. Chọn "Thêm khách hàng"	Form hiển thị		Hiển thị lỗi "Số điện thoại phải gồm đúng 10 chữ số"	**Pass**
		2. Nhập Họ tên, Email hợp lệ		Lê Minh / leminh@test.vn		

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	**Step Expected Result**			
		3. Nhập SĐT có chữ		09ab123456		
		4. Nhấn "Lưu khách hàng"	Hiển thị thông báo lỗi			
UC1.1-TC007	Tất cả các trường nhập hợp lệ	1. Chọn "Thêm khách hàng"	Form hiển thị		Hiển thị "Thêm khách hàng thành công", khách hàng xuất hiện trong danh sách	**Pass**
		2. Nhập Họ tên		Khách Kiểm Thử 475837		
		3. Nhập Email		kt475837@test.vn		
		4. Nhập SĐT		0947583711		
		5. Nhập Địa chỉ		Hà Đông, Hà Nội		
		6. Nhấn "Lưu khách hàng"	Hiển thị thông báo thành công			
UC1.1-TC008	SĐT trùng với khách hàng đã có	1. Chọn "Thêm khách hàng"	Form hiển thị		Hiển thị lỗi số điện thoại đã tồn tại, không tạo khách hàng (quy tắc nghiệp vụ UC1.1)	**Fail**
		2. Nhập Họ tên, Email mới, Địa chỉ		Khách Trùng SĐT / trungsdt@test.vn / Hà Nội		
		3. Nhập SĐT đã tồn tại (của Nguyễn Huy)		0911000001		
		4. Nhấn "Lưu khách hàng"	Hiển thị thông báo lỗi			
UC1.1-TC009	Email trùng với khách hàng đã có	1. Chọn "Thêm khách hàng"	Form hiển thị		Hiển thị lỗi email đã tồn tại, không tạo khách hàng	**Fail**
		2. Nhập Họ tên, SĐT mới, Địa chỉ		Khách Trùng Email / 0977000999 / Hà Nội		

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	**Step Expected Result**			
		3. Nhập Email đã tồn tại		owner.huy@example.com		
		4. Nhấn "Lưu khách hàng"	Hiển thị thông báo lỗi			
UC1.1-TC010	Bỏ trống Địa chỉ (trường không bắt buộc), các trường khác hợp lệ	1. Chọn "Thêm khách hàng"	Form hiển thị		Hiển thị "Thêm khách hàng thành công" (Địa chỉ không có dấu * nên không bắt buộc)	**Fail**
		2. Nhập Họ tên, Email, SĐT hợp lệ		Khách Không Địa Chỉ / kdc475837@test.vn / 0947583722		
		3. Bỏ trống Địa chỉ				
		4. Nhấn "Lưu khách hàng"	Hiển thị thông báo thành công			

3.2.2. UC1.3 Chỉnh sửa hồ sơ khách hàng

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	**Step Expected Result**			
UC1.3-TC001	Cập nhật địa chỉ khách hàng hợp lệ	1. Chọn "Chỉnh sửa" tại khách hàng vừa tạo	Form cập nhật hiển thị dữ liệu cũ		Hiển thị "Cập nhật khách hàng thành công", địa chỉ mới hiển thị trong danh sách	**Pass**
		2. Sửa Địa chỉ		Cầu Giấy, Hà Nội		
		3. Nhấn "Lưu"	Hiển thị thông báo thành công			
UC1.3-TC002	Cập nhật SĐT không hợp lệ	1. Chọn "Chỉnh sửa" tại khách hàng	Form cập nhật hiển thị		Hiển thị lỗi "Số điện thoại phải gồm đúng 10 chữ số", không lưu	**Pass**
		2. Sửa SĐT thành 3 chữ số		123		
		3. Nhấn "Lưu"	Hiển thị thông báo lỗi			

3.2.3. UC1.4 Tạm khóa / Duyệt hồ sơ khách hàng

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC1.4-TC001	Tạm khóa khách hàng đang hoạt động	1. Truy cập Quản lý khách hàng	Danh sách hiển thị		Hiển thị "Đã cập nhật trạng thái khách hàng", trạng thái đổi thành "Chờ duyệt"	Pass
		2. Nhấn "Tạm khóa" tại khách hàng vừa tạo	Trạng thái chuyển "Chờ duyệt"	Khách Kiểm Thử 475837		
UC1.4-TC002	Duyệt lại khách hàng đang bị khóa	1. Nhấn "Duyệt" tại khách hàng đang "Chờ duyệt"	Trạng thái chuyển "Đang hoạt động"	Khách Kiểm Thử 475837	Hiển thị "Đã duyệt khách hàng", trạng thái đổi thành "Đang hoạt động"	Pass

3.2.4. UC1.5 Tìm kiếm khách hàng / thú cưng

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC1.5-TC001	Tìm khách hàng theo tên	1. Nhập tên vào ô tìm kiếm	Danh sách được lọc	Trần Lan	Chỉ hiển thị khách hàng "Trần Lan"	Fail
UC1.5-TC002	Tìm khách hàng theo số điện thoại	1. Nhập SĐT vào ô tìm kiếm	Danh sách được lọc	0911000003	Hiển thị khách hàng "Phạm Tuấn"	Fail
UC1.5-TC003	Tìm với từ khóa không tồn tại	1. Nhập từ khóa không có trong dữ liệu	Danh sách rỗng	zzzkhongco	Hiển thị "Không có khách hàng phù hợp."	Pass
UC1.5-TC004	Tìm thú cưng theo tên chủ nuôi	1. Nhập tên khách hàng vào ô "Tìm thú cưng theo tên khách hàng"	Hiển thị thú cưng của khách	Phạm Tuấn	Hiển thị đúng 2 thú cưng của khách hàng Phạm Tuấn (Bun, Nau)	Pass
UC1.5-TC005	Lọc khách hàng theo trạng thái "Chờ duyệt"	1. Truy cập Quản lý khách hàng	Danh sách 30 khách hàng mẫu hiển thị		Hiển thị đúng 3 khách hàng chờ duyệt: Huỳnh Hoàng Nam, Ngô Mỹ Duyên, Huỳnh Bích Ngọc	Pass
		2. Chọn bộ lọc trạng thái "Chờ duyệt"	Danh sách được lọc	Chờ duyệt		
UC1.5-TC006	Lọc khách hàng theo trạng thái	1. Truy cập Quản lý khách hàng	Danh sách hiển thị		Hiển thị toàn bộ khách hàng đang hoạt động (tổng	Pass

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
	"Đang hoạt động"	2. Chọn bộ lọc trạng thái "Đang hoạt động"	Danh sách được lọc	Đang hoạt động	số khách hàng trừ 3 khách chờ duyệt), không có khách chờ duyệt	

3.2.5. UC1.2 Tạo mới hồ sơ thú cưng

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC1.2-TC001	Bỏ trống Tên và Loài thú cưng	1. Nhấn "Thêm thú cưng" tại khách hàng Nguyễn Huy	Form thêm thú cưng hiển thị		Hiển thị lỗi "Tên thú cưng là bắt buộc", "Loài là bắt buộc"	**Pass**
		2. Bỏ trống Tên, Loài				
		3. Nhấn "Lưu thú cưng"	Hiển thị thông báo lỗi			
UC1.2-TC002	Nhập Tên, bỏ trống Loài	1. Nhấn "Thêm thú cưng"	Form hiển thị		Hiển thị lỗi "Loài là bắt buộc"	**Pass**
		2. Nhập Tên thú cưng		Mướp		
		3. Bỏ trống Loài, nhấn "Lưu thú cưng"	Hiển thị thông báo lỗi			
UC1.2-TC003	Thông tin thú cưng hợp lệ	1. Nhấn "Thêm thú cưng" tại Nguyễn Huy	Form hiển thị, khách hàng tự điền		Hiển thị "Thêm thú cưng thành công", số thú cưng của khách hàng tăng 1	**Pass**
		2. Nhập Tên		Bông 475837		
		3. Chọn Loài		Mèo		
		4. Nhập Tuổi, Cân nặng, Microchip		2 / 3.5 / MC-475837		
		5. Nhấn "Lưu thú cưng"	Hiển thị thông báo thành công			

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC1.2-TC004	Mã microchip trùng với thú cưng đã có	1. Nhấn "Thêm thú cưng"	Form hiển thị		Hiển thị "Microchip đã tồn tại, vui lòng kiểm tra lại." (luồng thay thế UC1.2)	**Fail**
		2. Nhập Tên, chọn Loài		Mực / Chó		
		3. Nhập microchip đã dùng ở TC003		MC-475837		
		4. Nhấn "Lưu thú cưng"	Hiển thị thông báo lỗi			
UC1.2-TC005	Kiểm thử giá trị biên: Tuổi âm	1. Nhấn "Thêm thú cưng"	Form hiển thị		Không lưu thú cưng, hiển thị thông báo lỗi	**Pass**
		2. Nhập Tên, chọn Loài		Na / Thỏ		
		3. Nhập Tuổi âm		-1		
		4. Nhấn "Lưu thú cưng"	Hiển thị thông báo lỗi			

3.2.6. UC2.1 Đặt lịch khám

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC2.1-TC001	Bỏ trống tất cả các trường và nhấn "Lưu"	1. Chọn "Tạo lịch hẹn"	Form tạo lịch hẹn hiển thị		Hiển thị thông báo "Vui lòng chọn khách hàng", "Vui lòng chọn thú cưng"...	**Fail**
		2. Không chọn khách hàng, thú cưng, ngày, dịch vụ				
		3. Nhấn nút lưu	Hiển thị thông báo lỗi			
UC2.1-TC002	Đặt lịch hợp lệ	1. Chọn "Tạo lịch hẹn"	Form hiển thị		Hiển thị "Đặt lịch thành công"	**Pass**
		2. Chọn Khách hàng, Thú cưng		Nguyễn Huy / Bông 475837		
		3. Chọn Bác sĩ		Bác sĩ đầu tiên trong danh sách		

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	**Step Expected Result**			
		4. Chọn Ngày, Giờ khám		2026-10-04 15:00		
		5. Chọn Dịch vụ		Khám tổng quát		
		6. Nhấn nút lưu	Hiển thị thông báo thành công			
UC2.1-TC003	Đặt lịch trùng khung giờ của bác sĩ	1. Chọn "Tạo lịch hẹn"	Form hiển thị		Hiển thị "Không thể đặt lịch" – bác sĩ đã có lịch trong khung giờ	**Pass**
		2. Chọn khách hàng, thú cưng khác		Trần Lan		
		3. Chọn cùng Bác sĩ, Ngày, Giờ với TC002		2026-10-04 15:00		
		4. Chọn dịch vụ, nhấn lưu	Hiển thị thông báo lỗi	Tiêm phòng		
UC2.1-TC004	Đặt lịch trùng khung giờ của thú cưng	1. Chọn "Tạo lịch hẹn"	Form hiển thị		Hiển thị "Không thể đặt lịch" – thú cưng đã có lịch trong khung giờ	**Pass**
		2. Chọn cùng khách hàng, thú cưng với TC002, không chọn bác sĩ		Bông 475837		
		3. Chọn cùng Ngày, Giờ		2026-10-04 15:00		
		4. Chọn dịch vụ, nhấn lưu	Hiển thị thông báo lỗi	Tái khám		
UC2.1-TC005	Đặt lịch vào ngày trong quá khứ	1. Chọn "Tạo lịch hẹn"	Form hiển thị		Hệ thống từ chối lịch hẹn trong quá khứ	**Fail**
		2. Chọn khách hàng, thú cưng, dịch vụ hợp lệ		Nguyễn Huy / Bông 475837		
		3. Chọn ngày hôm qua		2026-10-02 10:00		
		4. Nhấn lưu	Hiển thị thông báo lỗi			

3.2.7. UC2.2 Xác nhận / Hủy / Hoàn thành lịch hẹn

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC2.2-TC001	Xác nhận lịch hẹn đang chờ	1. Có lịch hẹn "Chờ xác nhận" ngày mai	Lịch hẹn hiển thị ở tab "Quản lý toàn bộ"	Cún Xác Nhận / 2026-10-04 11:00	Hiển thị "Đã cập nhật lịch hẹn", lịch hẹn chuyển trạng thái "Đã xác nhận"	**Pass**
		2. Nhấn "Xác nhận" tại lịch hẹn	Trạng thái "Đã xác nhận"			
UC2.2-TC002	Hoàn thành lịch hẹn đã xác nhận	1. Nhấn "Hoàn thành" tại lịch hẹn đã xác nhận ở TC001	Trạng thái "Hoàn thành"	Cún Xác Nhận	Hiển thị "Đã cập nhật lịch hẹn", nút "Hủy lịch" bị vô hiệu	**Pass**
UC2.2-TC003	Hủy lịch hẹn còn trên 1 tiếng	1. Tạo lịch hẹn ngày mai cho thú cưng mới	Lịch hẹn "Chờ xác nhận"	2026-10-04 09:00	Hiển thị "Đã hủy lịch hẹn"	**Pass**
		2. Nhấn "Hủy lịch"	Trạng thái "Đã hủy"			
UC2.2-TC004	Hủy lịch hẹn khi còn dưới 1 tiếng	1. Có lịch hẹn bắt đầu sau 30 phút	Lịch hẹn "Chờ xác nhận"	Giờ hiện tại + 30 phút	Hệ thống từ chối – chỉ được hủy trước giờ hẹn ít nhất 1 tiếng (R2.2)	**Fail**
		2. Nhấn "Hủy lịch"	Hiển thị thông báo lỗi			

3.3. Chức năng của BÁC SĨ

3.3.1. UC3.1 Tạo hồ sơ khám bệnh

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC3.1-TC001	Bỏ trống tất cả các trường và nhấn "Lưu hồ sơ"	1. Mở tab "Tạo hồ sơ mới"	Form hồ sơ khám hiển thị		Hiển thị lỗi "Vui lòng chọn thú cưng", "Vui lòng chọn bác sĩ", "Triệu chứng là bắt buộc", "Chẩn đoán là bắt buộc", "Phác đồ điều trị là bắt buộc"	**Pass**
		2. Không nhập dữ liệu, nhấn "Lưu hồ sơ"	Hiển thị thông báo lỗi			
UC3.1-TC002	Bỏ trống Chẩn đoán	1. Mở tab "Tạo hồ sơ mới"	Form hiển thị		Hiển thị lỗi "Chẩn đoán là bắt buộc"	**Pass**
		2. Chọn Thú cưng, Bác sĩ		Bông 475837		

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	**Step Expected Result**			
		3. Nhập Triệu chứng, Điều trị; bỏ trống Chẩn đoán		Bỏ ăn 2 ngày / Truyền dịch		
		4. Nhấn "Lưu hồ sơ"	Hiển thị thông báo lỗi			
UC3.1-TC003	Nhập đầy đủ thông tin hồ sơ khám hợp lệ	1. Mở tab "Tạo hồ sơ mới"	Form hiển thị		Hiển thị "Lưu hồ sơ khám thành công", hồ sơ xuất hiện trong danh sách	**Pass**
		2. Chọn Thú cưng, Bác sĩ		Bông 475837		
		3. Nhập Triệu chứng		Bỏ ăn 2 ngày, nôn		
		4. Nhập Chẩn đoán		Viêm dạ dày 475837		
		5. Nhập Phác đồ điều trị		Truyền dịch, kháng sinh 5 ngày		
		6. Nhấn "Lưu hồ sơ"	Hiển thị thông báo thành công			

3.3.2. UC3.3 Tra cứu lịch sử khám

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	**Step Expected Result**			
UC3.3-TC001	Tìm hồ sơ theo chẩn đoán	1. Mở tab "Danh sách hồ sơ"	Danh sách hồ sơ hiển thị		Hiển thị "1 hồ sơ" có chẩn đoán tương ứng	**Pass**
		2. Nhập chẩn đoán vào ô tìm kiếm	Danh sách được lọc	Viêm dạ dày 475837		
UC3.3-TC002	Tìm hồ sơ với từ khóa không tồn tại	1. Nhập từ khóa không có trong dữ liệu	Danh sách rỗng	zzzkhong co	Hiển thị "Không tìm thấy hồ sơ theo điều kiện tìm kiếm."	**Pass**
UC3.3-TC003	Tìm hồ sơ theo chẩn	1. Mở tab "Danh sách hồ sơ"	Danh sách hồ sơ mẫu hiển thị		Hiển thị "4 hồ sơ", tất cả có chẩn	**Pass**

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
	đoán có nhiều kết quả	2. Nhập chẩn đoán vào ô tìm kiếm	Danh sách được lọc	Viêm da dị ứng	đoán "Viêm da dị ứng"	
UC3.3-TC004	Hiển thị toàn bộ lịch sử khám	1. Mở tab "Danh sách hồ sơ", không nhập từ khóa	Danh sách đầy đủ		Số hồ sơ hiển thị bằng tổng số hồ sơ trong CSDL (43 hồ sơ mẫu + hồ sơ tạo ở UC3.1-TC003)	**Pass**

3.3.3. UC2.4 Quản lý ca làm việc bác sĩ

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC2.4-TC001	Không chọn bác sĩ và ngày trực	1. Chọn "Tạo ca trực"	Form tạo ca trực hiển thị		Hiển thị "Vui lòng chọn bác sĩ và ngày trực"	**Pass**
		2. Không chọn bác sĩ, ngày; nhấn "Lưu ca trực"	Hiển thị thông báo lỗi			
UC2.4-TC002	Tạo ca trực hợp lệ	1. Chọn "Tạo ca trực"	Form hiển thị		Hiển thị "Đã tạo ca trực"	**Pass**
		2. Chọn Bác sĩ, Ngày trực		Bác sĩ đầu tiên / 2026-10-06		
		3. Nhập giờ bắt đầu – kết thúc		08:00 – 12:00		
		4. Nhấn "Lưu ca trực"	Hiển thị thông báo thành công			
UC2.4-TC003	Giờ kết thúc sớm hơn giờ bắt đầu	1. Chọn "Tạo ca trực"	Form hiển thị		Hệ thống từ chối ca trực có giờ kết thúc trước giờ bắt đầu	**Fail**
		2. Chọn Bác sĩ, Ngày trực		2026-10-06		
		3. Nhập giờ bắt đầu 17:00, kết thúc 13:00		17:00 – 13:00		
		4. Nhấn "Lưu ca trực"	Hiển thị thông báo lỗi			

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC2.4-TC004	Tạo ca trực trùng với ca đã có của bác sĩ	1. Chọn "Tạo ca trực"	Form hiển thị		Hệ thống báo trùng ca, không tạo ca trực (R2.4 – tránh trùng lịch)	Fail
		2. Chọn cùng Bác sĩ, Ngày với TC002		2026-10-06		
		3. Nhập giờ 09:00 – 11:00 (giao với 08:00 – 12:00)		09:00 – 11:00		
		4. Nhấn "Lưu ca trực"	Hiển thị thông báo lỗi			
UC2.4-TC005	Xóa ca trực	1. Nhấn "Xóa ca" tại một ca trực	Ca trực bị xóa khỏi danh sách		Hiển thị "Đã xóa ca trực"	Pass

3.3.4. UC8.2 Theo dõi thú cưng qua camera

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC8.2-TC001	Xem danh sách camera đang trực tuyến	1. Chọn menu "Camera giám sát"	Danh sách camera hiển thị		Hiển thị 12 camera trực tuyến (Phòng 01–03, 05–08, 10–12, 14, 15); không hiển thị 3 camera tắt (04, 09, 13)	Pass
UC8.2-TC002	Lọc camera theo trạng thái "Ngoại tuyến"	1. Chọn menu "Camera giám sát"	Danh sách camera hiển thị		Hiển thị 3 camera Phòng 04, 09, 13 với nhãn "Ngoại tuyến"	Fail
		2. Chọn bộ lọc trạng thái "Ngoại tuyến"	Hiển thị camera ngoại tuyến	Phòng 04, 09, 13 (camera đang tắt)		
UC8.2-TC003	Tìm camera theo số phòng	1. Chọn menu "Camera giám sát"	Danh sách camera hiển thị		Chỉ hiển thị camera Phòng 12	Pass
		2. Nhập số phòng vào ô tìm kiếm	Danh sách được lọc	Phòng 12		

3.4. Chức năng của QUẢN TRỊ VIÊN / KẾ TOÁN

3.4.1. UC6.1 Quản lý thông tin bác sĩ

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC6.1-TC001	Bỏ trống tất cả các trường và nhấn "Tạo bác sĩ"	1. Chọn "Thêm bác sĩ"	Form thêm bác sĩ hiển thị		Hiển thị thông báo lỗi rõ ràng cho từng trường bắt buộc	Fail
		2. Không nhập dữ liệu, nhấn "Tạo bác sĩ"	Hiển thị thông báo lỗi			
UC6.1-TC002	Email bác sĩ sai định dạng	1. Chọn "Thêm bác sĩ"	Form hiển thị		Hiển thị "Không thể thêm bác sĩ", không tạo bác sĩ	Pass
		2. Nhập Họ tên, Chuyên khoa, Mã hành nghề hợp lệ		BS. Hà An / Nội khoa / VET-475837		
		3. Nhập Email sai		haan.vet		
		4. Nhấn "Tạo bác sĩ"	Hiển thị thông báo lỗi			
UC6.1-TC003	Thêm bác sĩ hợp lệ	1. Chọn "Thêm bác sĩ"	Form hiển thị		Hiển thị "Đã thêm bác sĩ mới", bác sĩ xuất hiện trong danh sách	Pass
		2. Nhập Họ tên, Email		BS. Kiểm Thử 475837 / bs475837@petclinic.vn		
		3. Nhập SĐT, Chuyên khoa, Mã hành nghề		0901234567 / Da liễu / VET-475837		
		4. Nhấn "Tạo bác sĩ"	Hiển thị thông báo thành công			
UC6.1-TC004	Cập nhật chuyên khoa của bác sĩ	1. Nhấn "Chỉnh sửa" tại bác sĩ vừa tạo	Form cập nhật hiển thị	BS. Kiểm Thử 475837	Hiển thị "Đã cập nhật bác sĩ"	Pass
		2. Sửa Chuyên khoa		Ngoại khoa		
		3. Nhấn "Lưu cập nhật"	Hiển thị thông báo thành công			

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC6.1-TC005	Đổi trạng thái nhận lịch của bác sĩ	1. Nhấn "Đặt bận" tại bác sĩ vừa tạo	Trạng thái nhận lịch thay đổi	BS. Kiểm Thử 475837	Hiển thị "Đã cập nhật trạng thái nhận lịch", nút đổi thành "Đặt sẵn sàng"	**Pass**
UC6.1-TC006	Tìm bác sĩ theo chuyên khoa	1. Nhập chuyên khoa vào ô tìm kiếm	Danh sách được lọc	Ngoại khoa	Danh sách chỉ gồm bác sĩ có chuyên khoa "Ngoại khoa"	**Pass**

3.4.2. UC4.1 Nhập kho thuốc / vật tư

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC4.1-TC001	Bỏ trống các trường bắt buộc	1. Chọn "Thêm vật tư"	Form thêm vật tư hiển thị		Hiển thị lỗi "Mã vật tư là bắt buộc", "Tên vật tư là bắt buộc", "Danh mục là bắt buộc", "Đơn vị là bắt buộc", "Giá là bắt buộc"	**Fail**
		2. Không nhập dữ liệu, nhấn "Lưu vật tư"	Hiển thị thông báo lỗi			
UC4.1-TC002	Kiểm thử giá trị biên: Tồn kho âm	1. Chọn "Thêm vật tư"	Form hiển thị		Hiển thị lỗi "Tồn kho không hợp lệ"	**Pass**
		2. Nhập thông tin hợp lệ		TEST-NEG / Thuốc thử / Thuốc / Hộp / 50000		
		3. Nhập Tồn kho âm		-5		
		4. Nhấn "Lưu vật tư"	Hiển thị thông báo lỗi			
UC4.1-TC003	Thêm vật tư hợp lệ kèm hạn sử dụng	1. Chọn "Thêm vật tư"	Form hiển thị		Hiển thị "Thêm vật tư thành công", lô hàng được tạo kèm hạn dùng	**Pass**
		2. Nhập Mã, Tên		MED-475837 / Amoxicillin KT 475837		
		3. Chọn Danh mục, nhập Đơn vị		Thuốc / Hộp		

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
		4. Nhập Tồn kho, Tồn tối thiểu, Giá		20 / 5 / 45000		
		5. Nhập Hạn sử dụng		2027-04-01		
		6. Nhấn "Lưu vật tư"	Hiển thị thông báo thành công			
UC4.1-TC004	Mã vật tư trùng với vật tư đã có	1. Chọn "Thêm vật tư"	Form hiển thị		Hệ thống báo mã vật tư đã tồn tại, không tạo vật tư mới	Fail
		2. Nhập Mã vật tư đã dùng ở TC003		MED-475837		
		3. Nhập các trường còn lại hợp lệ, nhấn "Lưu vật tư"	Hiển thị thông báo lỗi	Thuốc trùng mã / Thuốc / Hộp / 10 / 1000		

3.4.3. UC4.3 Cảnh báo tồn kho thấp

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC4.3-TC001	Vật tư có tồn kho dưới mức tối thiểu	1. Thêm vật tư có Tồn kho 3, Tồn tối thiểu 10	Vật tư được lưu	LOW-475837	Số mặt hàng trong khung "Cảnh báo tồn kho thấp" tăng thêm 1 so với trước khi thêm	Pass
		2. Quan sát trang Kho thuốc	Hiển thị cảnh báo			
UC4.3-TC002	Xem danh sách mặt hàng sắp hết	1. Truy cập Kho thuốc	Khung cảnh báo tồn kho thấp hiển thị		Danh sách gồm 4 mặt hàng mẫu sắp hết (Doxycycline, Thuốc nhỏ mắt, Vaccine mèo 4 bệnh, Ống truyền dịch) và vật tư thêm ở TC001; không có mặt hàng đủ tồn (Rabies Vaccine)	Pass
		2. Nhấn "Xem mặt hàng sắp hết"	Danh sách chỉ còn mặt hàng sắp hết			

3.4.4. UC5.1 Tạo hóa đơn bán thuốc

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC5.1-TC001	Không chọn khách hàng và thuốc	1. Chọn "Tạo hóa đơn"	Form tạo hóa đơn hiển thị		Hiển thị "Vui lòng chọn khách hàng, thuốc và số lượng hợp lệ"	**Pass**
		2. Không chọn khách hàng, thuốc; nhấn "Tạo hóa đơn"	Hiển thị thông báo lỗi			
UC5.1-TC002	Tạo hóa đơn hợp lệ – tồn kho tự động trừ	1. Chọn "Tạo hóa đơn"	Form hiển thị		Hiển thị "Tạo hóa đơn thành công", tồn kho giảm từ 20 xuống 16, tổng tiền 180.000 VND	**Pass**
		2. Chọn Khách hàng		Nguyễn Huy		
		3. Chọn Thuốc		Amoxicillin KT 475837		
		4. Nhập Số lượng		4		
		5. Nhấn "Tạo hóa đơn"	Hiển thị thông báo thành công			
UC5.1-TC003	Số lượng bán lớn hơn tồn kho	1. Chọn "Tạo hóa đơn"	Form hiển thị		Hiển thị "Không thể tạo hóa đơn", không phát sinh hóa đơn nào	**Fail**
		2. Chọn Khách hàng, Thuốc		Trần Lan / Amoxicillin KT 475837		
		3. Nhập Số lượng lớn hơn tồn (16)		100		
		4. Nhấn "Tạo hóa đơn"	Hiển thị thông báo lỗi			

3.4.5. UC5.2 Thanh toán hóa đơn

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC5.2-TC001	Xác nhận đã thu tiền cho hóa đơn chờ thanh toán	1. Truy cập Hóa đơn	Danh sách hóa đơn hiển thị		Hiển thị "Thanh toán thành công", hóa đơn chuyển trạng thái đã thanh toán	**Pass**
		2. Nhấn "Xác nhận đã thu tiền" tại hóa đơn ở TC002	Trạng thái hóa đơn "Đã thanh toán"	180.000 VND		

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC5.2-TC002	Tìm hóa đơn theo mã hóa đơn	1. Nhập mã hóa đơn vào ô tìm kiếm	Danh sách được lọc	INV-2026-0020	Chỉ hiển thị 1 hóa đơn INV-2026-0020	**Pass**
UC5.2-TC003	Tìm hóa đơn theo trạng thái "Quá hạn"	1. Nhập trạng thái hiển thị trên giao diện vào ô tìm kiếm	Danh sách được lọc	Quá hạn	Hiển thị 5 hóa đơn mẫu có trạng thái "Quá hạn" (ô tìm kiếm ghi "Tìm theo mã, khách hàng, trạng thái")	**Fail**

3.4.6. UC7.2 Báo cáo doanh thu

Test Case ID	Test Case Description	Test Procedures		Test Case Input Value	Test Case Expected Result	Status
		Step to Perform	Step Expected Result			
UC7.2-TC001	Doanh thu trên Tổng quan khớp dữ liệu hóa đơn	1. Truy cập trang Tổng quan	Thẻ "Tổng doanh thu" hiển thị		Tổng doanh thu hiển thị bằng tổng tiền các hóa đơn đã thanh toán	**Pass**
		2. So sánh với tổng hóa đơn đã thanh toán	Hai giá trị bằng nhau			
UC7.2-TC002	Thẻ "Số đơn hàng" và "Mặt hàng thuốc" khớp dữ liệu	1. Truy cập trang Tổng quan	Các thẻ thống kê hiển thị		"Số đơn hàng" = tổng số hóa đơn; "Mặt hàng thuốc" = tổng số vật tư trong kho	**Pass**
		2. So sánh với số hóa đơn và số vật tư trong CSDL	Giá trị bằng nhau	≥ 60 hóa đơn, ≥ 30 vật tư		

3.5. Test case phi chức năng

Ngoài kiểm thử chức năng, nhóm xây dựng 28 test case phi chức năng. Cột "Kết quả đo được" là số liệu thực tế của lần chạy ở Chương 5.

3.5.1. Kiểm thử tải và hiệu năng

Test Case ID	Mô tả	Cách thực hiện	Tiêu chí đạt	Kết quả đo được	Status
LOAD-TC001	Trang Tổng quan – GET /api/dashboard/stats	autocannon: 200 kết nối đồng thời trong 30.17 giây	p99 ≤ 3.000 ms; tỷ lệ lỗi 0%	71.419 yêu cầu, 2381 req/s; p50 83 ms, p99 106 ms; lỗi 0%	Pass

Test Case ID	Mô tả	Cách thực hiện	Tiêu chí đạt	Kết quả đo được	Status
LOAD-TC002	Danh sách khách hàng – GET /api/customers	autocannon: 200 kết nối đồng thời trong 30.17 giây	p99 ≤ 3.000 ms; tỷ lệ lỗi 0%	69.616 yêu cầu, 2321 req/s; p50 84 ms, p99 118 ms; lỗi 0%	Pass
LOAD-TC003	Danh sách lịch hẹn – GET /api/appointments	autocannon: 200 kết nối đồng thời trong 30.11 giây	p99 ≤ 3.000 ms; tỷ lệ lỗi 0%	62.641 yêu cầu, 2088 req/s; p50 90 ms, p99 148 ms; lỗi 0%	Pass
LOAD-TC004	Kho thuốc – GET /api/inventory	autocannon: 200 kết nối đồng thời trong 30.06 giây	p99 ≤ 3.000 ms; tỷ lệ lỗi 0%	70.986 yêu cầu, 2367 req/s; p50 83 ms, p99 124 ms; lỗi 0%	Pass
LOAD-TC005	Kịch bản hỗn hợp (đọc nhiều màn hình)	autocannon: 200 kết nối đồng thời trong 30.1 giây	p99 ≤ 3.000 ms; tỷ lệ lỗi 0%	15.558 yêu cầu, 519 req/s; p50 381 ms, p99 637 ms; lỗi 0%	Pass
LOAD-TC006	Tải ghi – POST /api/customers (lễ tân tạo hồ sơ)	autocannon: 200 kết nối đồng thời trong 30.07 giây	p99 ≤ 3.000 ms; tỷ lệ lỗi 0%	21.135 yêu cầu, 705 req/s; p50 276 ms, p99 346 ms; lỗi 0%	Pass
PERF-TC001	Tìm kiếm khách hàng trên 10.030 bản ghi – 1 người dùng	Nạp 10.000 khách hàng; 50 lượt tìm theo tên, SĐT, email	p95 ≤ 2.000 ms (đặc tả mục 1.3)	TB 26 ms, p95 36 ms, max 99 ms	Pass
PERF-TC002	Tìm kiếm trên 10.000 bản ghi – 50 người dùng đồng thời	autocannon: 50 kết nối trong 20 giây	p99 ≤ 2.000 ms; lỗi 0%	2369 req/s; p50 20 ms, p99 33 ms; lỗi 0%	Pass

Bảng 10: Bảng test case kiểm thử tải và hiệu năng

3.5.2. Kiểm thử chịu tải

Test Case ID	Mô tả	Cách thực hiện	Tiêu chí đạt	Kết quả đo được	Status
STRESS-TC001	Tăng dần tải để tìm ngưỡng quá tải	Kịch bản hỗn hợp, 50 → 100 → 200 → 400 → 800 → 1200 → 1600 kết nối, 15 giây/mức	Không quá tải ở mức ≤ 200 kết nối (yêu cầu đặc tả)	Đạt yêu cầu tới 400 kết nối; quá tải từ 800 kết nối (p99 3.717 ms); lỗi cao nhất 27.39%	Pass
STRESS-TC002	Máy chủ không bị sập khi quá tải	Gọi API ngay sau mức tải cao nhất	HTTP 200	HTTP 200, tiến trình vẫn chạy	Pass
STRESS-TC003	Hồi phục sau quá tải	Giảm về 50 kết nối trong 10 giây	p99 ≤ 3.000 ms; lỗi 0%	p99 316 ms; lỗi 0%	Pass

Bảng 11: Bảng test case kiểm thử chịu tải

3.5.3. Kiểm thử toàn vẹn dữ liệu

Test Case ID	Mô tả	Cách thực hiện	Tiêu chí đạt	Kết quả đo được	Status
DI-TC001	20 giao dịch bán đồng thời trên vật tư tồn kho 10	Gửi đồng thời 20 yêu cầu POST /api/invoice-items, mỗi yêu cầu 1 hộp	Đúng 10 giao dịch thành công; tồn kho = 0; không bán vượt	Số đã bán (20) + tồn (8) ≠ 10 – bán vượt tồn kho	Fail
DI-TC002	10 yêu cầu đặt cùng bác sĩ, cùng khung giờ đồng thời	Gửi đồng thời 10 yêu cầu POST /api/appointments	Chỉ 1 lịch hẹn được tạo	Có 10 lịch hẹn cùng bác sĩ, cùng khung giờ	Fail
DI-TC003	Tổng tiền hóa đơn khớp các dòng hóa đơn	Truy vấn SQL đối chiếu toàn bộ hóa đơn	0 hóa đơn lệch	Đạt	Pass
DI-TC004	Không có tồn kho âm	Truy vấn SQL bảng inventory	0 bản ghi stock < 0	Đạt	Pass
DI-TC005	Ràng buộc khóa ngoại thú cưng – khách hàng	Tạo thú cưng với customerId không tồn tại	HTTP 400; 0 thú cưng mồ côi	Đạt	Pass
DI-TC006	Hóa đơn chuyển "Đã thanh toán" khi thu đủ qua nhiều lần	Hóa đơn 300.000đ, thu 100.000đ rồi 200.000đ	Sau lần 1: pending; sau lần 2: paid	Đã thu đủ 300.000đ qua 2 lần nhưng hóa đơn vẫn "pending"	Fail
DI-TC007	Từ chối thanh toán vượt số tiền hóa đơn	Thu 5.000.000đ cho hóa đơn 200.000đ	HTTP 400	Thanh toán 5.000.000đ cho hóa đơn 200.000đ được chấp nhận (HTTP 201)	Fail
DI-TC008	Tiếng Việt, emoji, ký tự HTML được lưu nguyên vẹn	Tạo khách hàng tên có dấu, emoji 🐶, ký tự <b>&'" rồi đọc lại	Dữ liệu đọc lại giống hệt	Đạt	Pass
DI-TC009	Chuỗi SQL Injection trong ô tìm kiếm	4 chuỗi tấn công (' OR '1'='1, DROP TABLE…)	HTTP 200, 0 kết quả, số bản ghi không đổi	Đạt	Pass
DI-TC010	Họ tên 5.000 ký tự	POST /api/customers với tên 5.000 ký tự	Từ chối hoặc lưu nguyên vẹn, không lỗi 5xx	Đạt	Pass
DI-TC011	Cập nhật một trường không làm mất trường khác	PATCH địa chỉ, đọc lại hồ sơ	Các trường khác giữ nguyên	Đạt	Pass

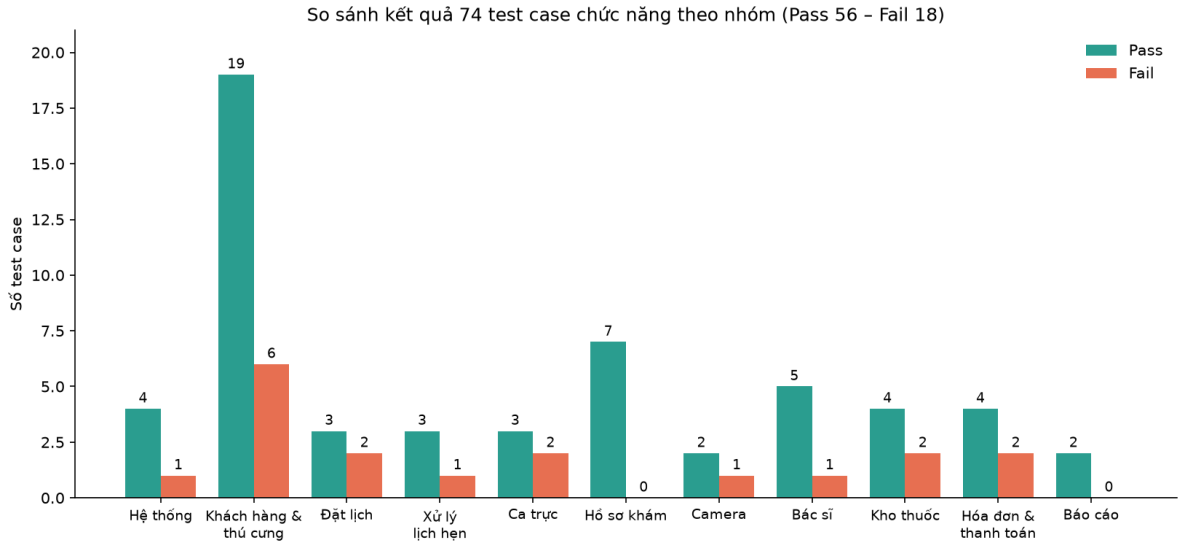
Bảng 12: Bảng test case kiểm thử toàn vẹn dữ liệu

3.5.4. Kiểm thử độ tin cậy và phục hồi

Test Case ID	Mô tả	Cách thực hiện	Tiêu chí đạt	Kết quả đo được	Status
REL-TC001	Yêu cầu sai định dạng JSON	POST /api/customers với body hỏng	HTTP 4xx, ứng dụng vẫn chạy	HTTP 400, ứng dụng vẫn chạy	Pass
REL-TC002	Mất kết nối CSDL khi đang chạy	Dừng MySQL, gọi API	HTTP 5xx có kiểm soát, ứng dụng không sập	HTTP 500; ứng dụng vẫn chạy	Pass
REL-TC003	Tự phục hồi khi CSDL hoạt động lại	Khởi động lại MySQL, không khởi động lại ứng dụng	API trở lại bình thường ≤ 30 giây	Phục hồi sau 1.056 ms	Pass
REL-TC004	Dữ liệu bền vững khi ứng dụng bị tắt đột ngột	Tạo hồ sơ, kill tiến trình, khởi động lại, đọc hồ sơ	Hồ sơ còn nguyên	Hồ sơ còn nguyên	Pass
REL-TC005	Thời gian khởi động lại ứng dụng	Đo từ lúc khởi chạy tới khi API trả 200	≤ 10 giây	834 ms	Pass
REL-TC006	Chạy tải liên tục 180 giây	autocannon: 50 kết nối, 4 API đọc	Lỗi 0%; p99 ≤ 3.000 ms; RAM không tăng liên tục	61.913 yêu cầu; p99 395 ms; lỗi 0%; RAM 219 → 335 → 324 MB	Pass

Bảng 13: Bảng test case kiểm thử độ tin cậy và phục hồi

3.6. Biểu đồ so sánh



Hình 2: Biểu đồ so sánh các test case chức năng theo nhóm

3.7. Danh sách lỗi phát hiện

Bộ kiểm thử phát hiện 22 lỗi, xếp theo mức độ nghiêm trọng. Cột "Vị trí nghi vấn" chỉ ra nơi trong mã nguồn gây lỗi để nhóm phát triển khắc phục.

Test Case	Mức độ	Mô tả lỗi	Kết quả thực tế	Vị trí nghi vấn
SYS-TC004	Cao	Chưa có đăng nhập và phân quyền RBAC; mọi người truy cập URL đều thao tác được dữ liệu	Không có trang đăng nhập – truy cập thẳng được danh sách khách hàng	client/src/App.tsx, server/routes.ts (không có middleware xác thực)
UC1.1-TC010	Cao	Bỏ trống ô Địa chỉ (không bắt buộc) thì không thêm được khách hàng; thông báo lỗi là chuỗi JSON	Hiển thị "Không thể thêm khách hàng" kèm lỗi JSON: "String must contain at least 1 character(s)", path ["address"]	AddCustomerDialog.tsx gửi address = ""; shared/schema.ts: nullableString = z.string().min(1)
UC1.5-TC001	Cao	Ô tìm kiếm khách hàng mất focus sau ký tự đầu, không gõ tiếp được	Ô tìm kiếm mất focus sau ký tự đầu – chỉ nhận "T", danh sách lọc sai	CustomerList.tsx:70 – return sớm khi isLoading làm ô tìm kiếm bị gỡ khỏi DOM
UC1.5-TC002	Cao	Như UC1.5-TC001 khi tìm theo số điện thoại	Ô tìm kiếm mất focus sau ký tự đầu – chỉ nhận "0", danh sách lọc sai	CustomerList.tsx:70
DI-TC001	Cao	Bán đồng thời: tồn kho bị ghi đè (lost update), bán vượt số lượng thực có	Số đã bán (20) + tồn (8) ≠ 10 – bán vượt tồn kho	server/storage.ts – createInvoiceItem(): đọc tồn kho không khóa (thiếu SELECT … FOR UPDATE hoặc UPDATE có điều kiện)
DI-TC002	Cao	Đặt lịch đồng thời: nhiều lịch trùng bác sĩ, trùng giờ cùng được tạo	Có 10 lịch hẹn cùng bác sĩ, cùng khung giờ	server/storage.ts – createAppointment(): kiểm tra rồi mới chèn, không khóa và không có ràng buộc UNIQUE
UC1.1-TC008	Trung bình	Không kiểm tra trùng số điện thoại	Hệ thống vẫn lưu khách hàng có SĐT trùng 0911000001	server/storage.ts – createCustomer()
UC1.1-TC009	Trung bình	Không kiểm tra trùng email	Hệ thống vẫn lưu khách hàng có email trùng	server/storage.ts – createCustomer()
UC1.2-TC004	Trung bình	Không kiểm tra trùng mã microchip	Hệ thống vẫn lưu thú cưng có microchip trùng	server/storage.ts – createPet()
UC2.1-TC005	Trung bình	Cho phép đặt lịch hẹn trong quá khứ	Hệ thống vẫn tạo lịch hẹn ngày 2026-10-02	server/storage.ts – createAppointment()
UC2.2-TC004	Trung bình	Cho phép hủy lịch khi còn dưới 1 tiếng (trái R2.2)	Lịch hẹn vẫn bị hủy dù chỉ còn 30 phút	server/storage.ts – deleteAppointment()
UC2.4-TC003	Trung bình	Ca trực có giờ kết thúc trước giờ bắt đầu vẫn được lưu	Ca trực 17:00 – 13:00 vẫn được tạo	shared/schema.ts – insertDoctorShiftSchema
UC2.4-TC004	Trung bình	Ca trực trùng giờ của cùng bác sĩ vẫn được lưu	Ca trực trùng giờ vẫn được tạo	server/storage.ts – createDoctorShift()

Test Case	Mức độ	Mô tả lỗi	Kết quả thực tế	Vị trí nghi vấn
UC4.1-TC004	Trung bình	Mã vật tư trùng vẫn được lưu	Vẫn tạo vật tư thứ hai có mã MED-085746	server/storage.ts – createInventoryItem()
UC5.1-TC003	Trung bình	Bán vượt tồn kho: báo lỗi nhưng vẫn để lại hóa đơn rỗng 0 VND	Có báo lỗi nhưng vẫn phát sinh 1 hóa đơn rỗng 0 VND ở trạng thái chờ thanh toán	InvoicesPage.tsx – tạo hóa đơn trước, thêm dòng sau, không cùng transaction
DI-TC006	Trung bình	Thu tiền nhiều lần đủ tổng nhưng hóa đơn vẫn "Chờ thanh toán"	Đã thu đủ 300.000đ qua 2 lần nhưng hóa đơn vẫn "pending"	server/routes.ts – POST /api/payments so sánh từng lần thu thay vì tổng đã thu
DI-TC007	Trung bình	Chấp nhận thanh toán vượt số tiền hóa đơn	Thanh toán 5.000.000đ cho hóa đơn 200.000đ được chấp nhận (HTTP 201)	server/routes.ts – POST /api/payments không kiểm tra số tiền
UC2.1-TC001	Thấp	Form đặt lịch không hiển thị thông báo lỗi khi bỏ trống	Form không gửi nhưng không hiển thị thông báo lỗi nào cho người dùng	AddAppointmentDialog.tsx – không render formState.errors
UC8.2-TC002	Thấp	Bộ lọc "Ngoại tuyến" luôn rỗng	Danh sách trống – API /api/camera-streams chỉ trả về camera đang hoạt động nên bộ lọc "Ngoại tuyến" không bao giờ có kết quả	server/storage.ts – getCameraStreams() chỉ lấy isActive = true
UC6.1-TC001	Thấp	Thông báo lỗi hiển thị chuỗi JSON kỹ thuật của Zod	Hiển thị "Không thể thêm bác sĩ" kèm chuỗi lỗi kỹ thuật (JSON của Zod), khó hiểu với người dùng	DoctorsPage.tsx, server/routes.ts trả nguyên error.message
UC4.1-TC001	Thấp	Bỏ trống Giá không có thông báo lỗi	Form không lưu, nhưng thiếu thông báo: "Giá là bắt buộc"	InventoryTable.tsx – thiếu hiển thị errors.price
UC5.2-TC003	Thấp	Tìm hóa đơn theo trạng thái hiển thị ("Quá hạn", "Đã hủy"…) không ra kết quả	Hiển thị 0 hóa đơn – bộ lọc chỉ so khớp mã trạng thái tiếng Anh (overdue), không khớp nhãn tiếng Việt	InvoicesPage.tsx – bộ lọc so khớp inv.status (overdue, paid…) thay vì nhãn tiếng Việt

Bảng 14: Bảng danh sách lỗi phát hiện qua kiểm thử chức năng và phi chức năng

4. Chương 4: Cài đặt phần mềm

4.1. Giới thiệu

Chương này hướng dẫn cài đặt phần mềm Pet Clinic Pro và bộ kiểm thử, gồm yêu cầu hệ thống, quy trình cài đặt và cách sử dụng cơ bản.

4.2. Yêu cầu hệ thống

4.2.1. Yêu cầu phần cứng

- Bộ vi xử lý: Intel Core i3 / AMD Ryzen 3 hoặc tương đương trở lên.
- RAM: tối thiểu 8 GB.
- Dung lượng ổ cứng trống: tối thiểu 10 GB.
- Độ phân giải màn hình: 1366×768 hoặc cao hơn.

4.2.2. Yêu cầu phần mềm

- Node.js 20 trở lên, npm.
- MySQL 8.x (cài trực tiếp hoặc qua Docker).
- Trình duyệt Google Chrome, Microsoft Edge hoặc Mozilla Firefox bản mới nhất.

4.3. Quy trình cài đặt

1. Cài thư viện: chạy lệnh npm install trong thư mục pet-pttkpm.
2. Khởi động MySQL: docker compose up -d db (hoặc dùng MySQL cài sẵn).
3. Tạo file .env từ .env.example và cập nhật DATABASE_URL.
4. Tạo bảng và dữ liệu mẫu: npm run db:setup.
5. Chạy ứng dụng: npm run dev, truy cập http://localhost:5000.

```
cd pet-pttkpm/pet-pttkpm
npm install
cp .env.example .env          # sửa DATABASE_URL nếu cần
npm run db:setup              # drizzle-kit push + seed dữ liệu mẫu
npm run dev                   # http://localhost:5000
```

4.4. Hướng dẫn sử dụng cơ bản

B1: mở trình duyệt, truy cập địa chỉ hệ thống; trang Tổng quan hiển thị doanh thu, số đơn hàng, tồn kho, lịch hẹn hôm nay.

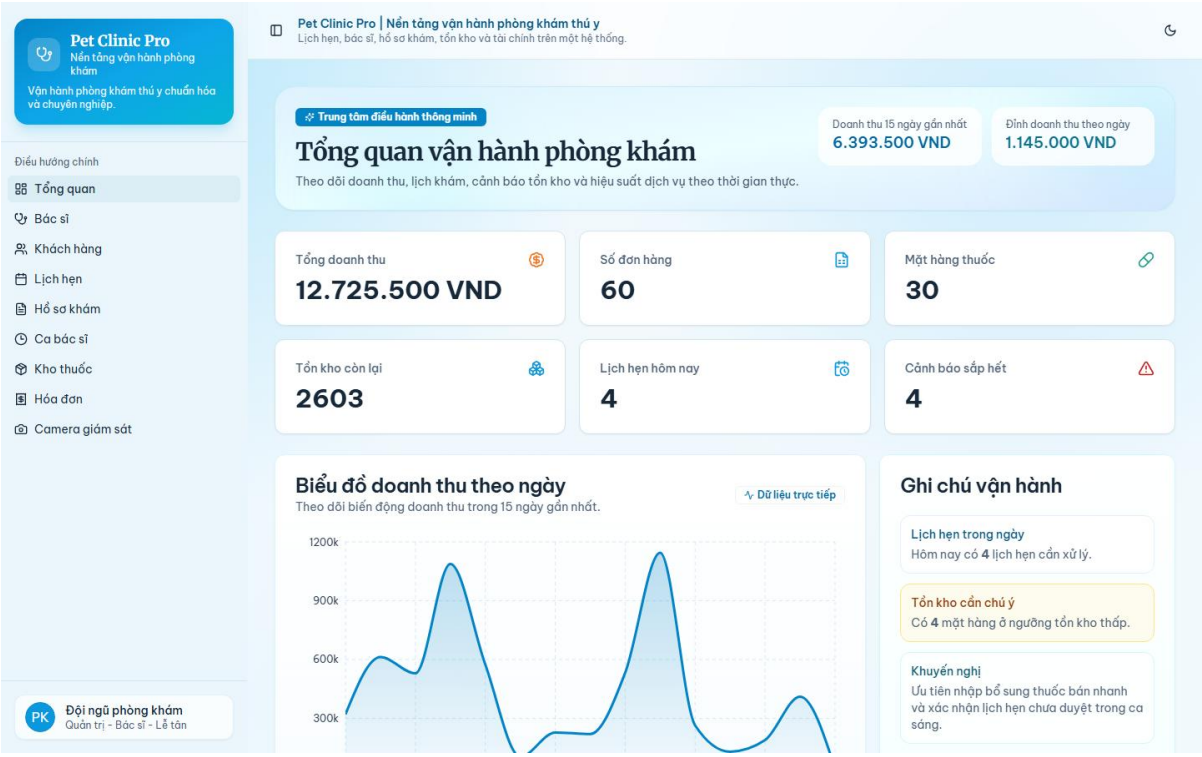
B2: dùng menu bên trái để vào các chức năng: Bác sĩ, Khách hàng, Lịch hẹn, Hồ sơ khám, Ca bác sĩ, Kho thuốc, Hóa đơn, Camera giám sát.

B3: lễ tân thêm khách hàng, thêm thú cưng và tạo lịch hẹn; bác sĩ lập hồ sơ khám; quản trị viên/kế toán quản lý kho, lập hóa đơn và xác nhận thanh toán.

5. Chương 5: Kết quả kiểm thử và kết luận

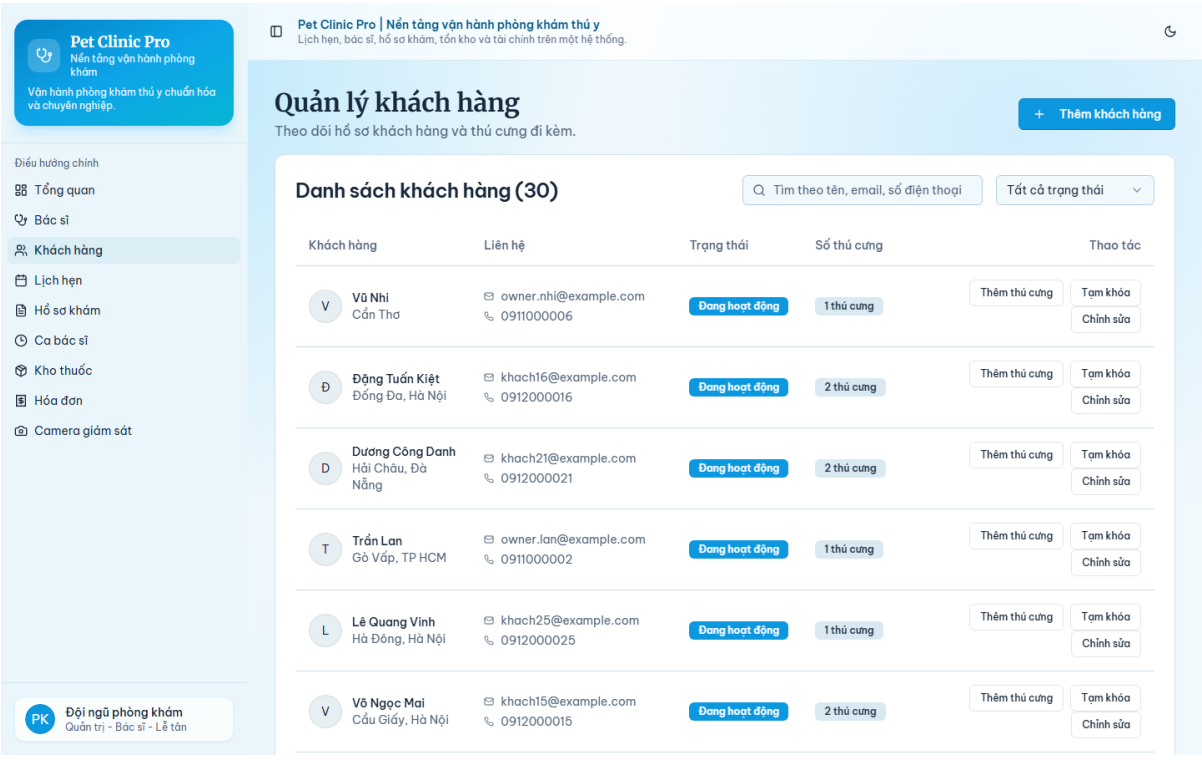
5.1. Giới thiệu trang web quản lý phòng khám thú cưng

5.1.1. Giao diện Tổng quan vận hành phòng khám



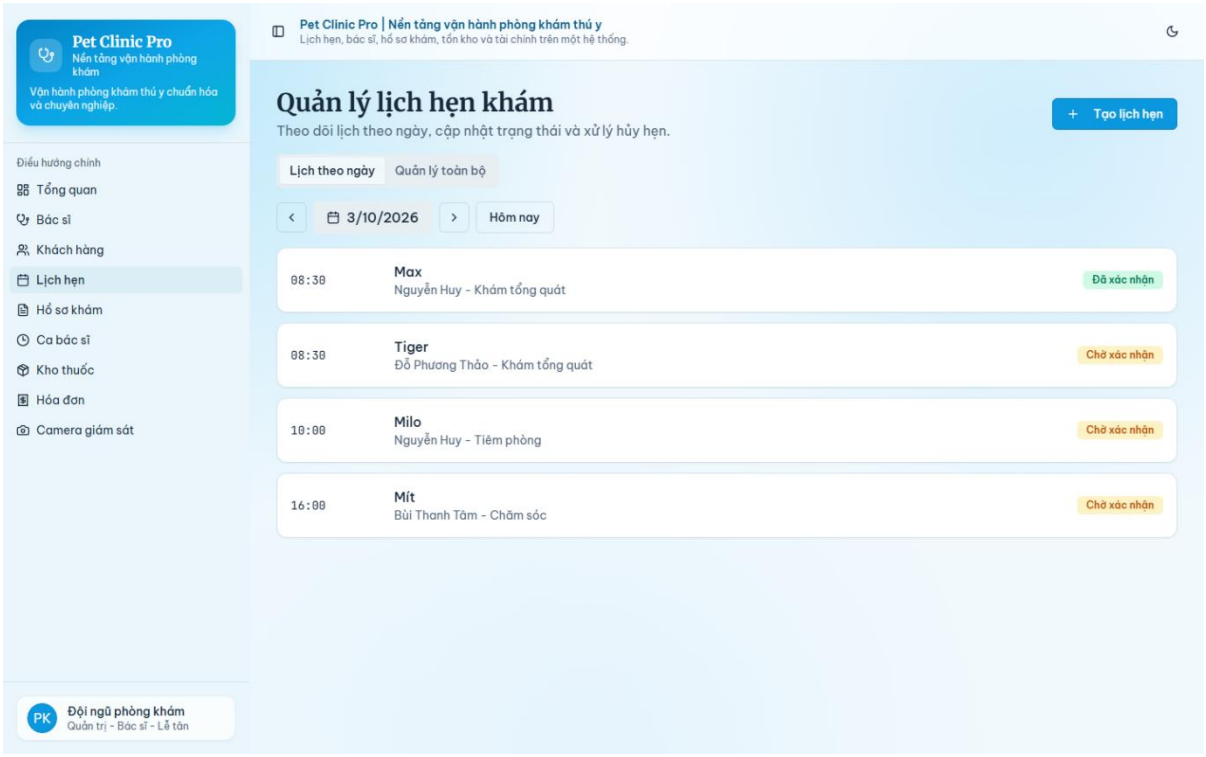
Hình 3: Giao diện Tổng quan vận hành phòng khám

5.1.2. Giao diện Quản lý khách hàng



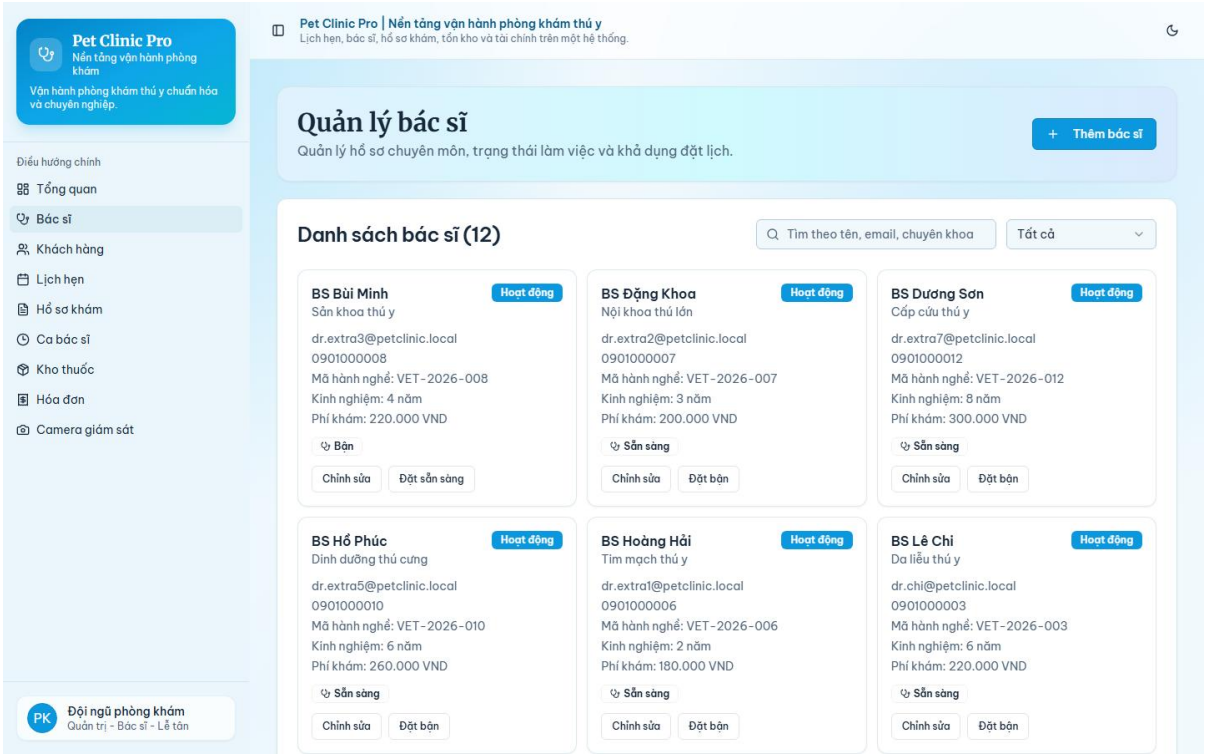
Hình 4: Giao diện Quản lý khách hàng

5.1.3. Giao diện Quản lý lịch hẹn



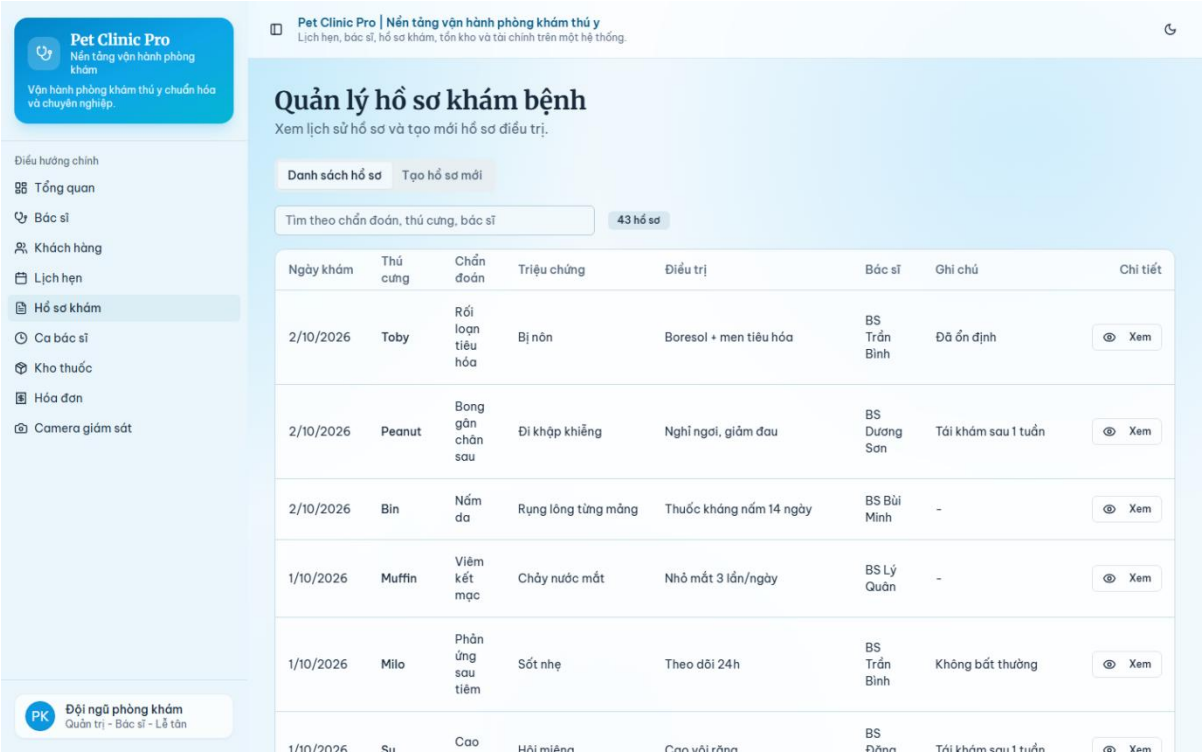
Hình 5: Giao diện Quản lý lịch hẹn

5.1.4. Giao diện Quản lý bác sĩ



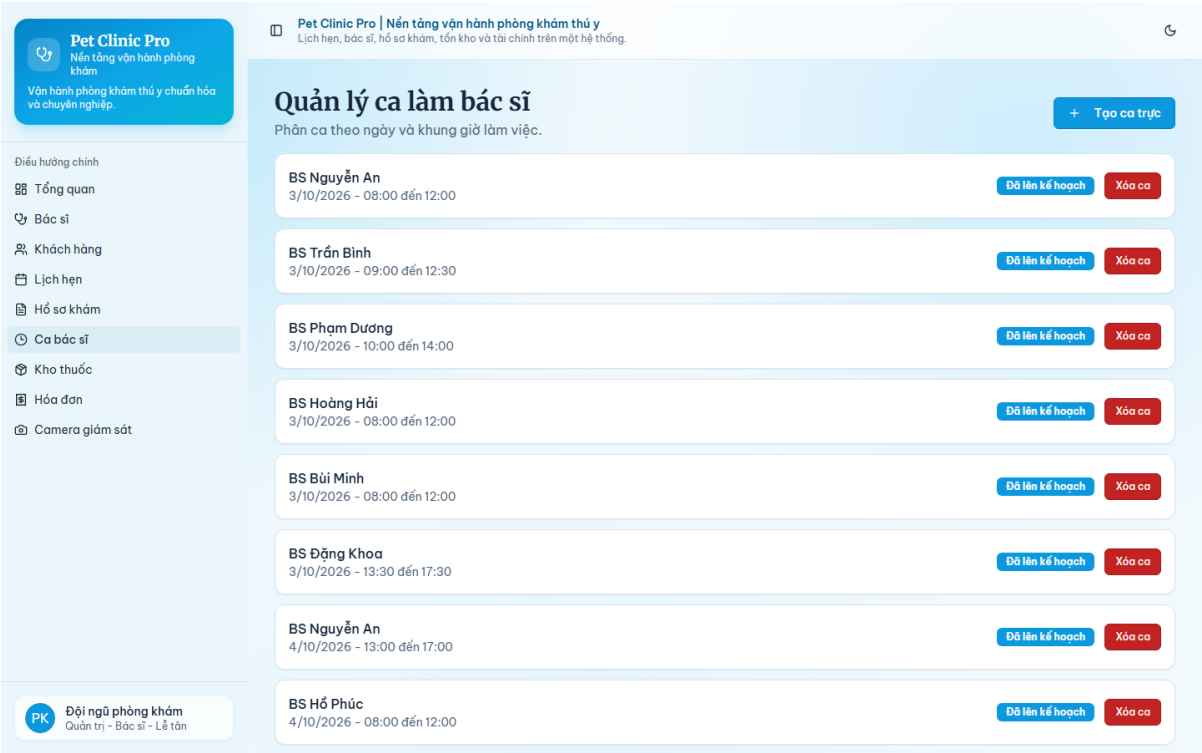
Hình 6: Giao diện Quản lý bác sĩ

5.1.5. Giao diện Quản lý hồ sơ khám bệnh



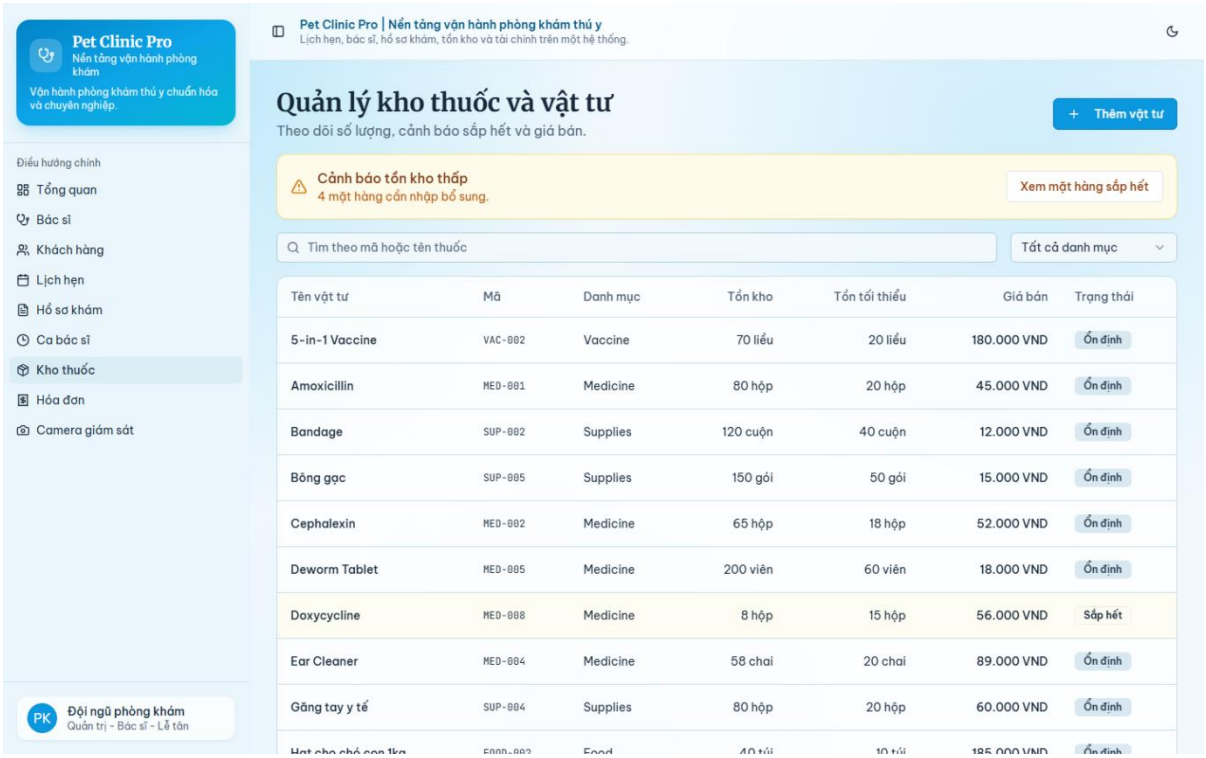
Hình 7: Giao diện Quản lý hồ sơ khám bệnh

5.1.6. Giao diện Ca làm việc bác sĩ



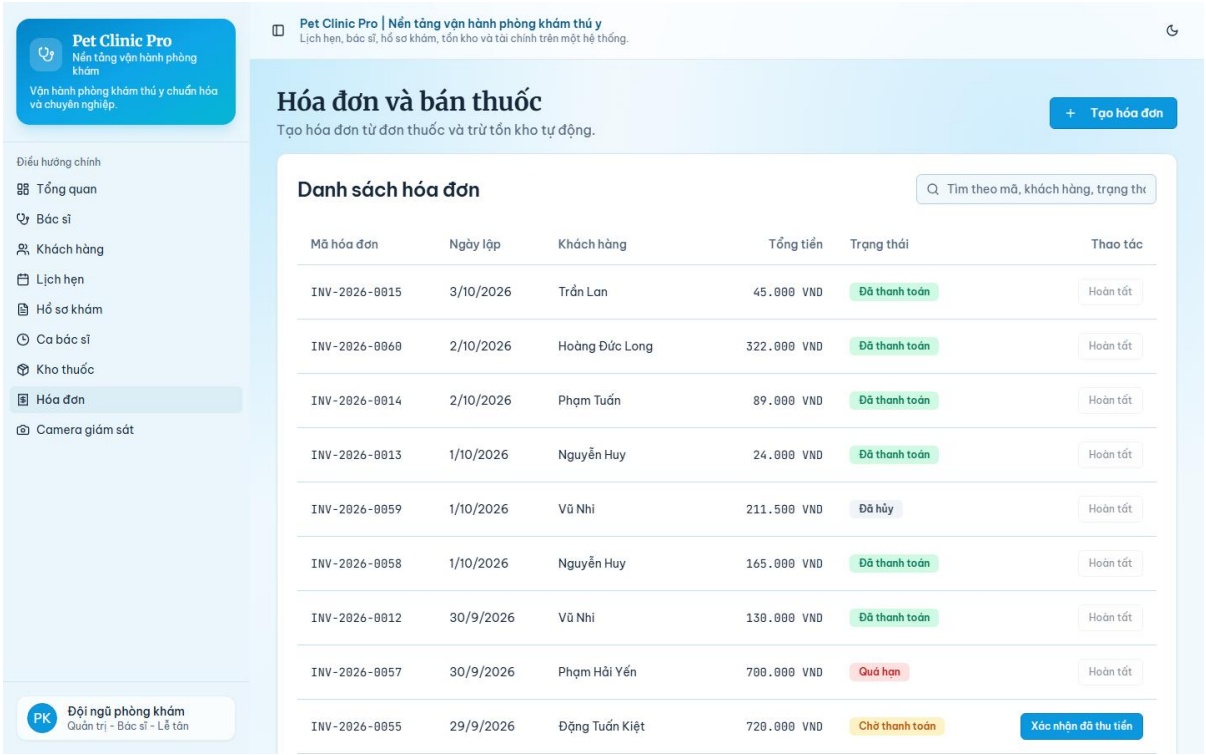
Hình 8: Giao diện Ca làm việc bác sĩ

5.1.7. Giao diện Quản lý kho thuốc và vật tư



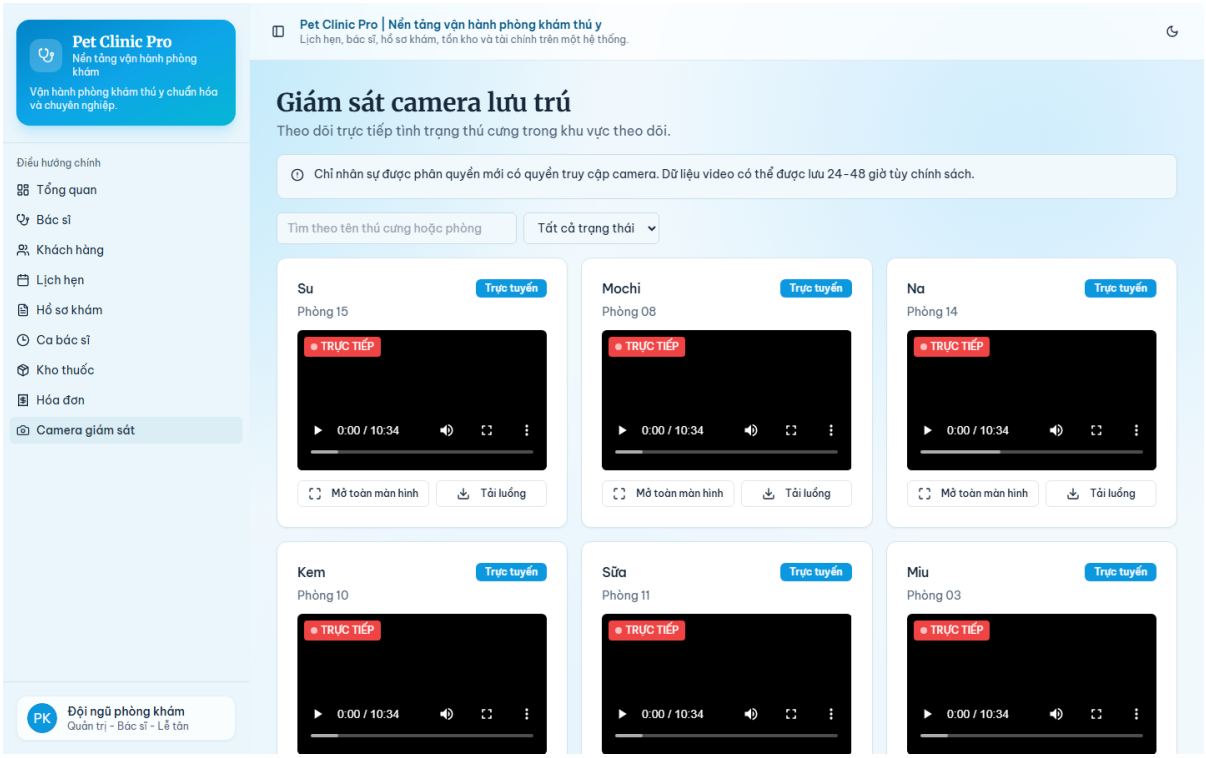
Hình 9: Giao diện Quản lý kho thuốc và vật tư

5.1.8. Giao diện Hóa đơn



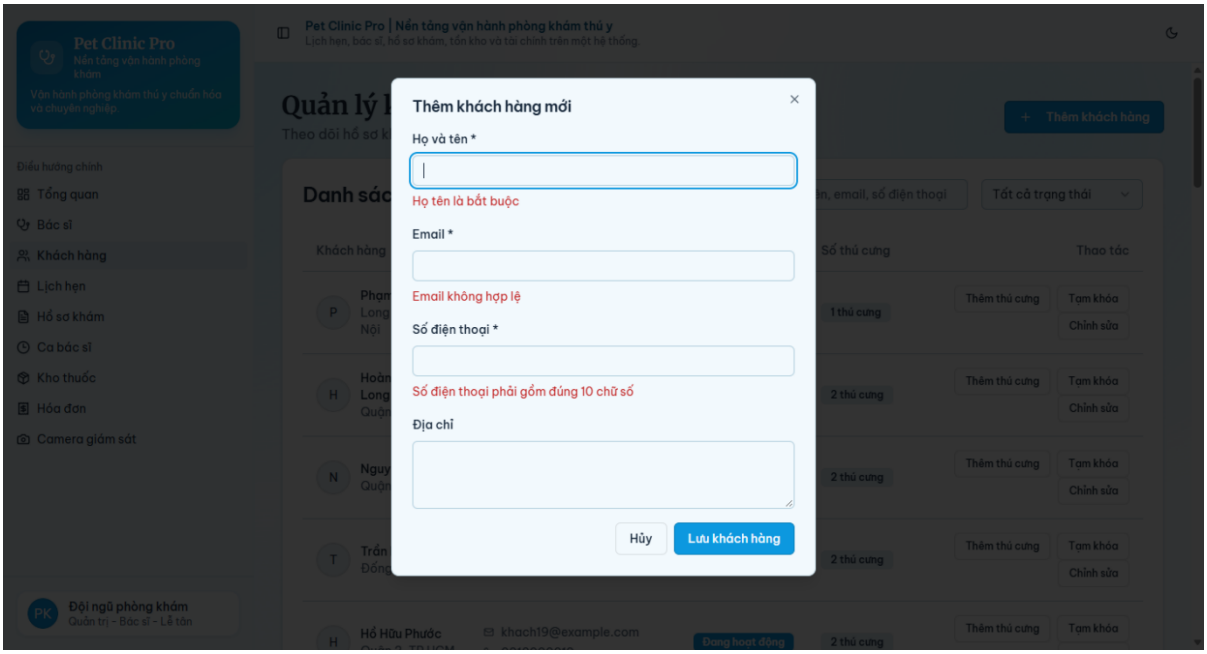
Hình 10: Giao diện Hóa đơn

5.1.9. Giao diện Camera giám sát

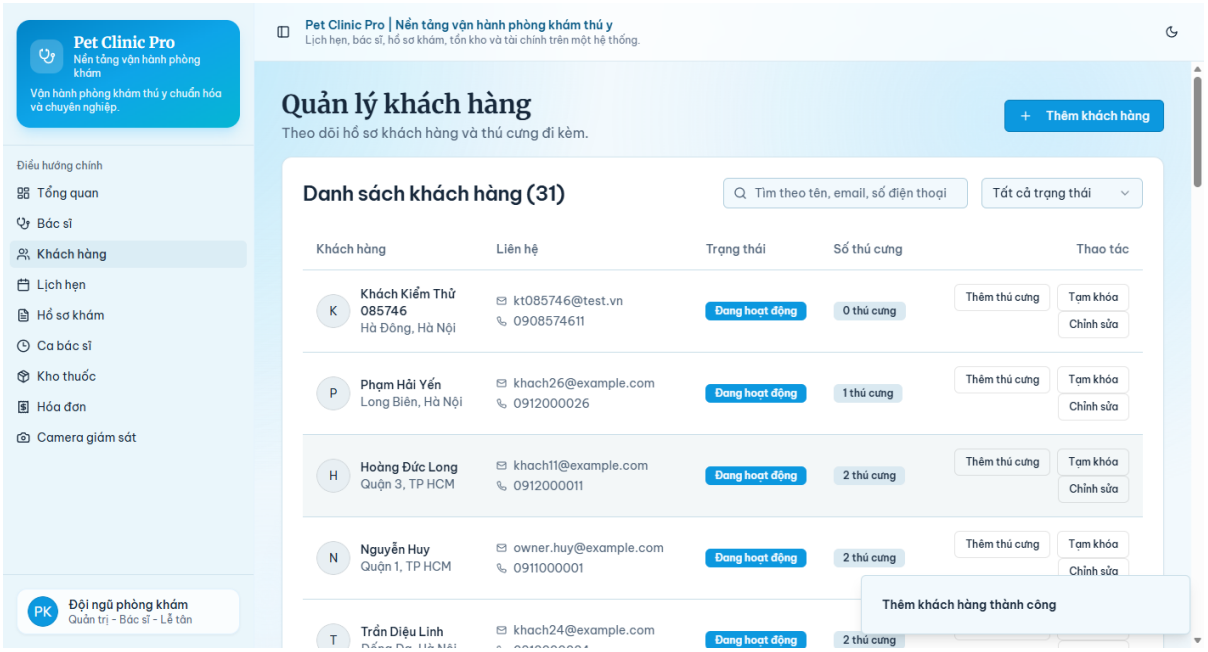


Hình 11: Giao diện Camera giám sát

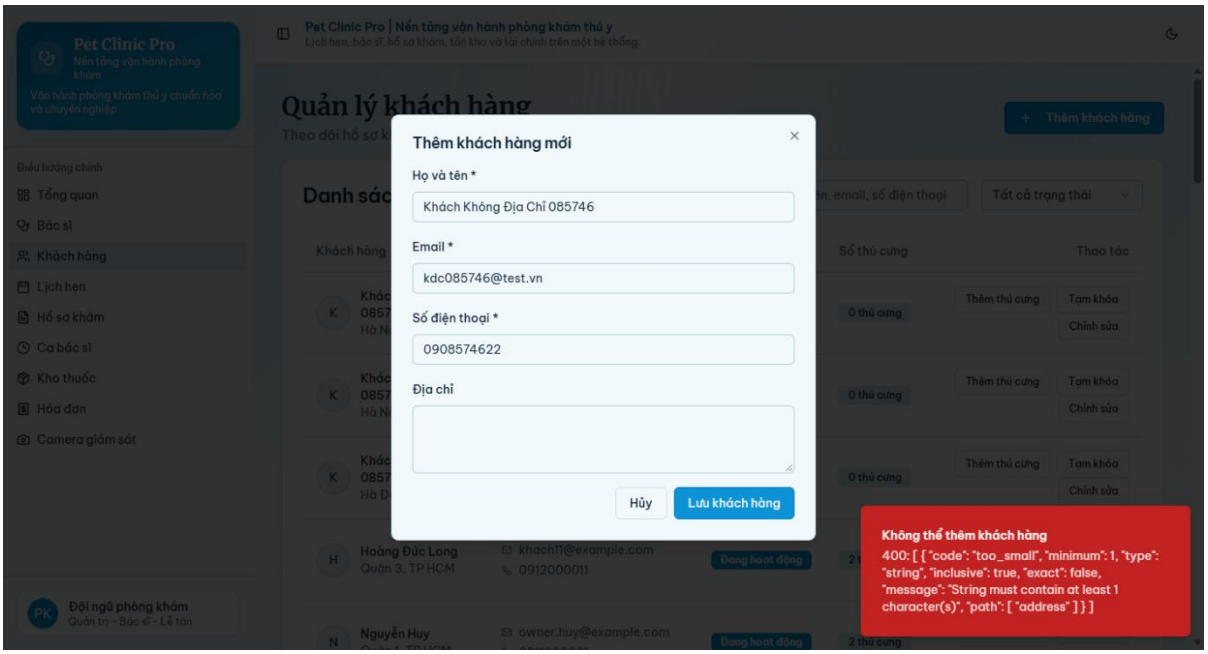
5.1.10. Một số màn hình ghi nhận trong quá trình kiểm thử



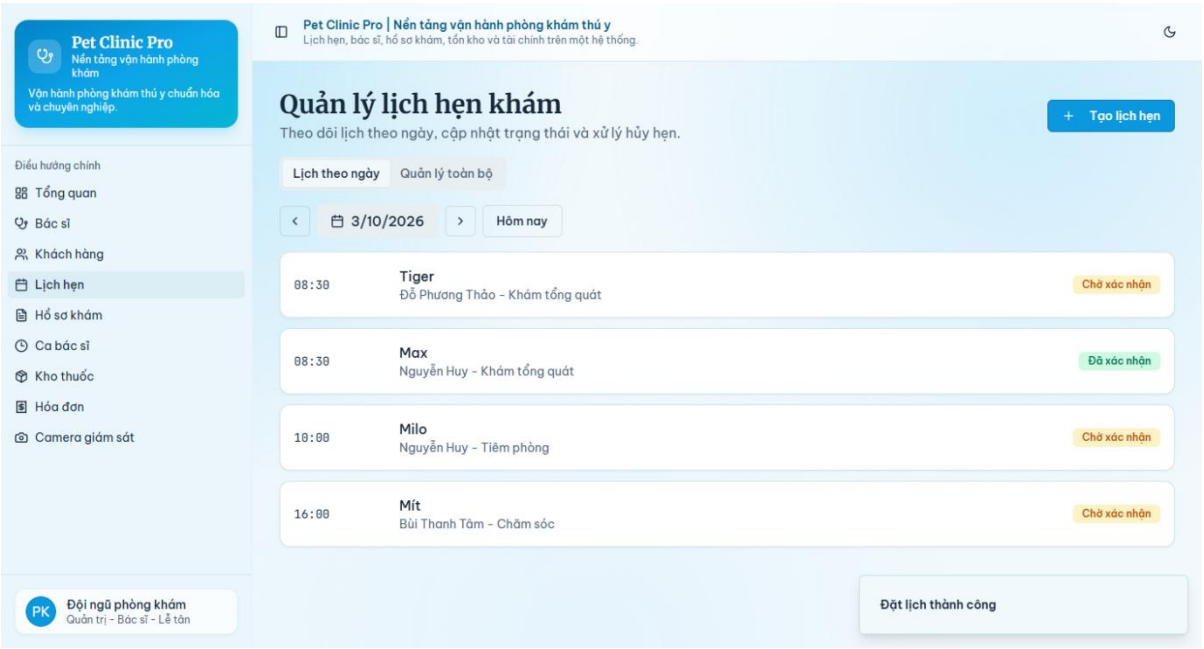
Hình 12: UC1.1-TC001 – Thông báo lỗi khi bỏ trống form thêm khách hàng



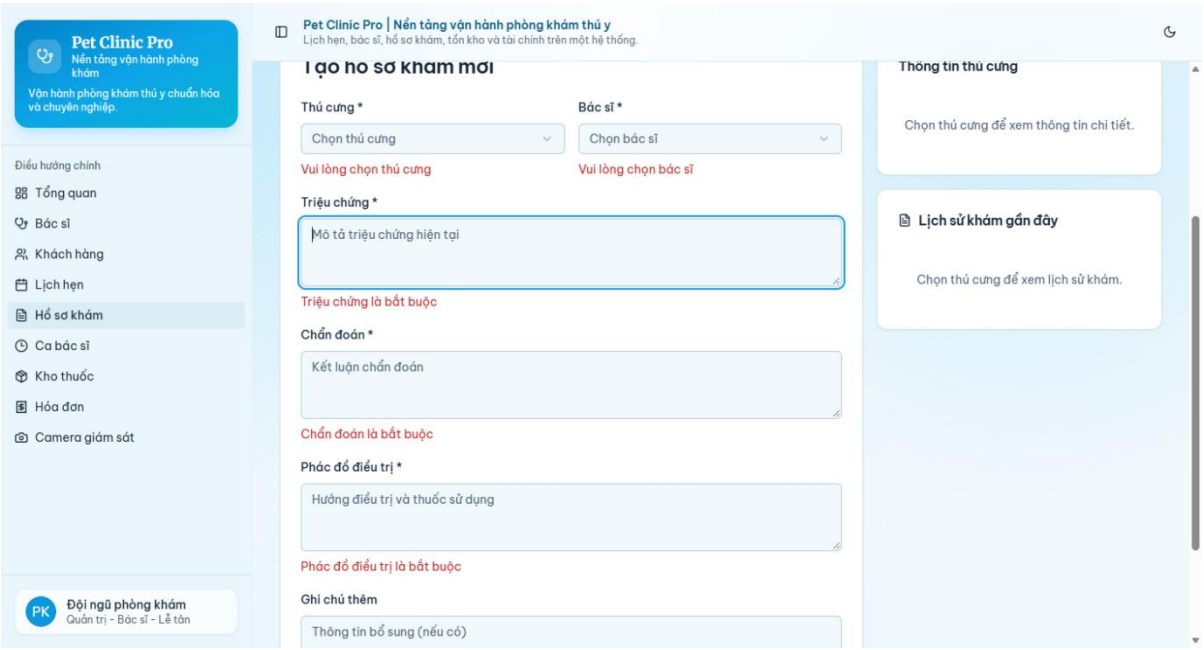
Hình 13: UC1.1-TC007 – Thêm khách hàng thành công



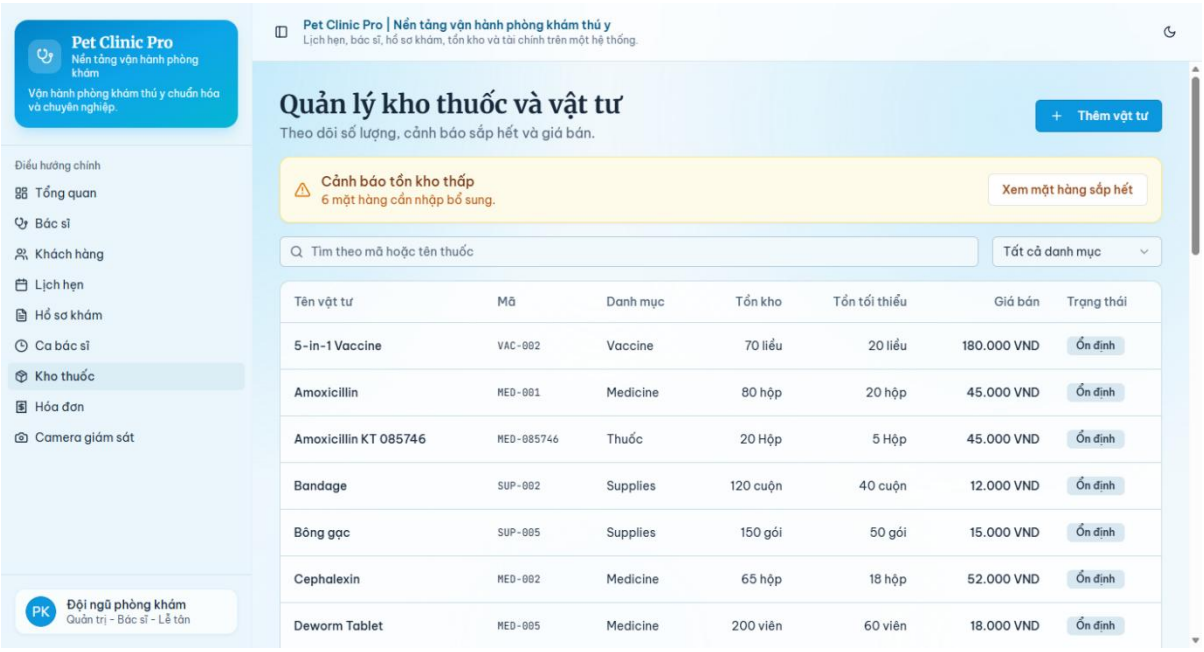
Hình 14: UC1.1-TC010 (Fail) – Bỏ trống Địa chỉ bị từ chối kèm lỗi JSON



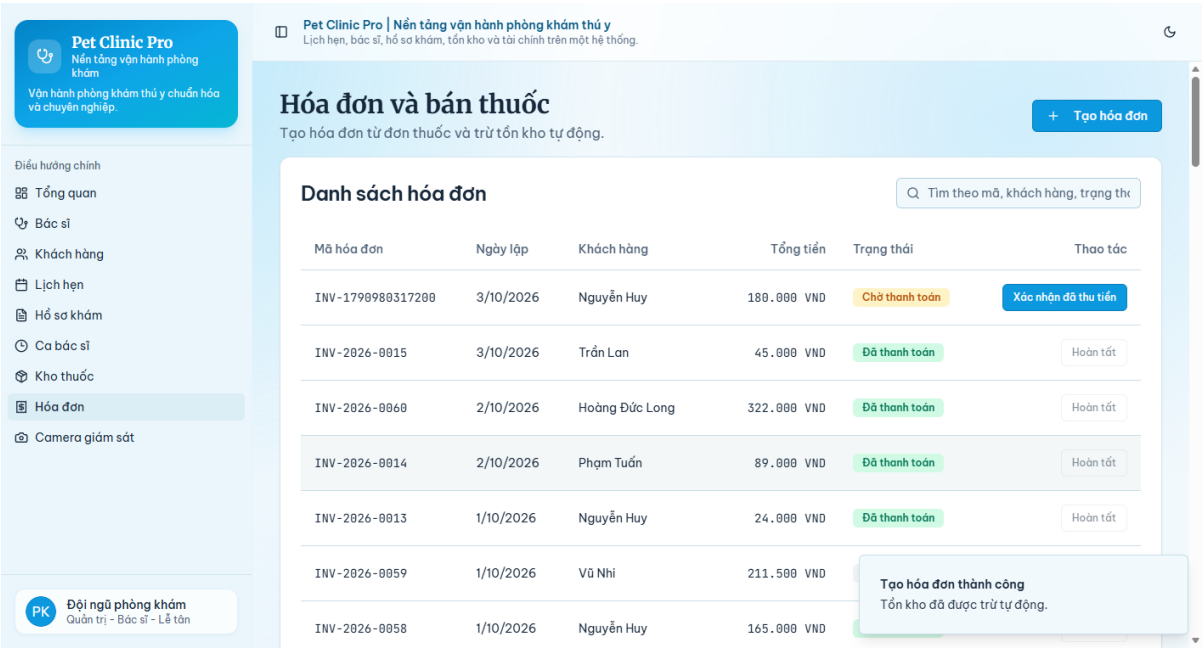
Hình 15: UC2.1-TC002 – Đặt lịch hẹn thành công



Hình 16: UC3.1-TC001 – Thông báo lỗi khi bỏ trống hồ sơ khám



Hình 17: UC4.3-TC001 – Cảnh báo tồn kho thấp



Hình 18: UC5.1-TC002 – Tạo hóa đơn thành công, tồn kho tự trừ

5.2. Kiểm thử đơn vị (Unit Test) bằng Mocha

5.2.1. Mục tiêu và phạm vi

Kiểm thử hộp trắng cho hai thành phần lõi: (1) các schema kiểm tra dữ liệu dùng chung giữa client và server trong shared/schema.ts; (2) lớp nghiệp vụ DatabaseStorage trong server/storage.ts, chạy trên một CSDL MySQL thử nghiệm riêng. Tổng cộng 30 unit test.

5.2.2. Cài đặt và chạy

Mã nguồn unit test nằm trong thư mục pet-pttkpm/tests/unit. Mocha được thêm vào devDependencies, tsx dùng để chạy trực tiếp file TypeScript và hiểu alias @shared:

```
npm install --save-dev mocha@10
# package.json → "test:unit": "mocha --import=tsx \"tests/unit/**/*.test.ts\""

# Chạy trên CSDL thử nghiệm (KHÔNG dùng CSDL thật)
$env:DATABASE_URL = "mysql://root@127.0.0.1:3307/petclinic"   # PowerShell
npm run test:unit
```

5.2.3. Mã nguồn unit test

tests/unit/schema.test.ts – kiểm tra schema khách hàng, thú cưng, lịch hẹn, kho, hóa đơn, ca trực (trích):

```
/**
 * schema.test.ts – UNIT TEST CHO CÁC SCHEMA KIỂM TRA DỮ LIỆU (shared/schema.ts)
 * ------------------------------------------------------------------------
 * Schema (thư viện Zod) là "bộ lọc" dữ liệu: mọi yêu cầu tạo/sửa đều đi qua đây trước khi lưu
vào CSDL.
 * Unit test gọi trực tiếp schema, KHÔNG cần mở web hay CSDL → chạy rất nhanh (vài mili giây).
 *
 * Cách đọc một test:
 *   - schema.parse(dữ liệu)      : dữ liệu hợp lệ thì trả về dữ liệu đã chuẩn hóa, sai thì
ném lỗi.
 *   - schema.safeParse(dữ liệu)  : không ném lỗi, trả về { success: true/false }.
 *   - assert.equal(a, b, "...")   : nếu a khác b → test FAIL, in câu thông báo phía sau.
 *
 * Kỹ thuật thiết kế: phân vùng tương đương (hợp lệ / không hợp lệ) và giá trị biên (0, âm,
rỗng).
 * Chạy: npm run test:unit
 */
import assert from "node:assert/strict";
import {
  insertAppointmentSchema,
  insertCustomerSchema,
  insertDoctorShiftSchema,
  insertInventorySchema,
  insertInvoiceItemSchema,
  insertPaymentSchema,
  insertPetSchema,
  insertStockBatchSchema,
} from "@shared/schema";

// Một khách hàng hợp lệ mẫu – các test sửa từng trường để tạo dữ liệu sai.
const validCustomer = {
  name: "Nguyễn Văn A",
  email: "a@example.com",
  phone: "0912345678",
  address: "Hà Nội",
};

describe("UT-SCH: insertCustomerSchema", () => {
  // Dữ liệu đúng chuẩn phải được chấp nhận; trường isActive không truyền vào thì tự nhận giá
trị mặc định true.
  it("UT-SCH-01: chấp nhận dữ liệu khách hàng hợp lệ và gán isActive mặc định = true", () => {
    const parsed = insertCustomerSchema.parse(validCustomer);
    assert.equal(parsed.isActive, true);
  });

  // Email thiếu @ phải bị từ chối (safeParse trả success = false).
```

```
  it("UT-SCH-02: từ chối email sai định dạng", () => {
    const r = insertCustomerSchema.safeParse({ ...validCustomer, email: "a.example.com" });
    assert.equal(r.success, false);
  });

  // Họ tên là trường bắt buộc – chuỗi rỗng phải bị từ chối.
  it("UT-SCH-03: từ chối họ tên rỗng", () => {
    const r = insertCustomerSchema.safeParse({ ...validCustomer, name: "" });
    assert.equal(r.success, false);
  });

  // Địa chỉ không bắt buộc. Form gửi address = "" khi bỏ trống, nên schema phải chấp nhận.
  // Hiện FAIL: nullableString = z.string().min(1) từ chối chuỗi rỗng → nguyên nhân lỗi UC1.1-
TC010.
  it("UT-SCH-04: chấp nhận địa chỉ rỗng vì đây là trường không bắt buộc", () => {
    // Form thêm khách hàng gửi address = "" khi người dùng bỏ trống ô Địa chỉ.
    const r = insertCustomerSchema.safeParse({ ...validCustomer, address: "" });
    assert.equal(r.success, true, "address = \"\" bị từ chối: String must contain at least 1
character(s)");
  });

  // Quy tắc nghiệp vụ: SĐT phải đủ 10 chữ số. Kiểm tra phía server (gọi API trực tiếp vẫn
phải bị chặn).
  // Hiện FAIL: chỉ form giao diện kiểm tra, schema server chấp nhận "123".
  it("UT-SCH-05: từ chối số điện thoại không đủ 10 chữ số (quy tắc nghiệp vụ UC1.1)", () => {
    const r = insertCustomerSchema.safeParse({ ...validCustomer, phone: "123" });
    assert.equal(r.success, false, "Schema phía server chấp nhận SĐT \"123\"");
  });
});
```

tests/unit/storage.test.ts – kiểm tra quy tắc trùng lịch, xóa mềm, trừ tồn kho trong transaction (trích):

```
describe("UT-STO: DatabaseStorage", function () {
  this.timeout(20000);

  let customerId: string;
  let petId: string;
  let vetId: string;

  before(async () => {
    const customer = await storage.createCustomer({ name: `UT Khách ${RUN}`, email:
`ut${RUN}@test.vn`, phone: `08${RUN}00`, isActive: true });
    customerId = customer.id;
    const pet = await storage.createPet({ customerId, name: `UT Pet ${RUN}`, species: "Chó",
isActive: true });
    petId = pet.id;
    const vet = await storage.createDoctor({
      name: `UT BS ${RUN}`, email: `utbs${RUN}@test.vn`, specialization: "Nội khoa",
licenseNumber: `UT-${RUN}`,
      yearsExperience: 1, consultationFee: "100000", isActive: true, isAvailable: true,
    });
    vetId = vet.id;
  });

  after(async () => {
    await pool.end();
  });

  describe("Lịch hẹn (UC2.1, UC2.2)", () => {
    const slot = new Date("2031-03-10T09:00:00");
```

```
    // Luồng chuẩn: tạo lịch hẹn hợp lệ, trạng thái mặc định pending.
    it("UT-STO-01: tạo lịch hẹn hợp lệ với trạng thái mặc định pending", async () => {
      const ap = await storage.createAppointment({ petId, customerId, veterinarianId: vetId,
appointmentDate: slot, appointmentType: "Khám tổng quát", status: "pending" });
      assert.equal(ap.status, "pending");
    });

    // Nhánh kiểm tra trùng bác sĩ trong createAppointment(): cùng bác sĩ, cùng giờ → phải ném
lỗi.
    it("UT-STO-02: từ chối lịch hẹn trùng khung giờ của bác sĩ", async () => {
      const otherPet = await storage.createPet({ customerId, name: `UT Pet2 ${RUN}`, species:
"Mèo", isActive: true });
      await rejects(
        storage.createAppointment({ petId: otherPet.id, customerId, veterinarianId: vetId,
appointmentDate: slot, appointmentType: "Tiêm phòng", status: "pending" }),
        /Veterinarian already has an appointment/,
      );
    });

    // Nhánh kiểm tra trùng thú cưng: cùng thú cưng, cùng giờ → phải ném lỗi.
    it("UT-STO-03: từ chối lịch hẹn trùng khung giờ của thú cưng", async () => {
      await rejects(
        storage.createAppointment({ petId, customerId, appointmentDate: slot, appointmentType:
"Tái khám", status: "pending" }),
        /Pet already has an appointment/,
      );
    });

    // deleteAppointment() không xóa hẳn mà đổi status = cancelled (xóa mềm).
    // Sau khi hủy, khung giờ đó phải đặt lại được (vì truy vấn trùng lịch bỏ qua lịch đã
hủy).
    it("UT-STO-04: hủy lịch hẹn là xóa mềm (status = cancelled), khung giờ được giải phóng",
async () => {
      const t = new Date("2031-03-11T10:00:00");
      const ap = await storage.createAppointment({ petId, customerId, appointmentDate: t,
appointmentType: "Chăm sóc", status: "pending" });
      await storage.deleteAppointment(ap.id);
      const after = await storage.getAppointmentById(ap.id);
      assert.equal(after?.status, "cancelled");
      const again = await storage.createAppointment({ petId, customerId, appointmentDate: t,
appointmentType: "Chăm sóc", status: "pending" });
      assert.ok(again.id);
    });

    // Lịch hẹn trong quá khứ phải bị từ chối. Hiện FAIL: storage không kiểm tra ngày.
    it("UT-STO-05: từ chối lịch hẹn có ngày trong quá khứ", async () => {
      const past = new Date(Date.now() - 24 * 3600 * 1000);
      await assert.rejects(
        storage.createAppointment({ petId, customerId, appointmentDate: past, appointmentType:
"Khám tổng quát", status: "pending" }),
        "Lịch hẹn trong quá khứ vẫn được tạo",
      );
    });
  });

  describe("Kho thuốc và hóa đơn (UC4, UC5)", () => {
    let itemId: string;
    let invoiceId: string;

    before(async () => {
      const item = await storage.createInventoryItem({ code: `UT-${RUN}`, name: `UT Thuốc
${RUN}`, category: "Thuốc", unit: "Hộp", stock: 10, minStock: 3, price: "20000" });
      itemId = item.id;
      const inv = await storage.createInvoice({ customerId, invoiceNumber: `UT-INV-${RUN}`,
totalAmount: "0", status: "pending" });
      invoiceId = inv.id;
```

```
    });

    // updateInventoryStock(): +5 rồi −3 trên tồn kho 10 → phải còn 12.
    it("UT-STO-06: điều chỉnh tồn kho cộng/trừ đúng số lượng", async () => {
      await storage.updateInventoryStock(itemId, 5);
      await storage.updateInventoryStock(itemId, -3);
      assert.equal((await storage.getInventoryById(itemId))?.stock, 12);
    });

    // Trừ quá số đang có phải ném lỗi "Insufficient stock" và tồn kho giữ nguyên.
    it("UT-STO-07: từ chối điều chỉnh làm tồn kho âm", async () => {
      await rejects(storage.updateInventoryStock(itemId, -100), /Insufficient stock/);
      assert.equal((await storage.getInventoryById(itemId))?.stock, 12);
    });

    // createInvoiceItem(): bán 2 hộp → tồn kho giảm 2 và tổng tiền hóa đơn = tổng các dòng
(40.000đ).
    it("UT-STO-08: thêm dòng hóa đơn tự trừ tồn kho và cập nhật tổng tiền hóa đơn", async ()
=> {
      await storage.createInvoiceItem({ invoiceId, inventoryId: itemId, description: "UT
Thuốc", quantity: 2, unitPrice: "20000", totalPrice: "40000" });
      assert.equal((await storage.getInventoryById(itemId))?.stock, 10);
      assert.equal(parseFloat((await storage.getInvoiceById(invoiceId))!.totalAmount), 40000);
    });

    // Bán vượt tồn kho → ném lỗi; nhờ transaction, tồn kho không bị trừ dở dang (rollback).
    it("UT-STO-09: từ chối dòng hóa đơn vượt tồn kho, không thay đổi tồn kho (transaction
rollback)", async () => {
      await rejects(
        storage.createInvoiceItem({ invoiceId, inventoryId: itemId, description: "UT Thuốc",
quantity: 999, unitPrice: "20000", totalPrice: "19980000" }),
        /Insufficient inventory stock/,
      );
      assert.equal((await storage.getInventoryById(itemId))?.stock, 10);
    });

    // Vật tư có tồn kho ≤ tồn tối thiểu phải nằm trong getLowStockItems() (cảnh báo sắp hết).
    it("UT-STO-10: vật tư có tồn kho <= tồn tối thiểu nằm trong danh sách sắp hết", async ()
=> {
      await storage.updateInventoryStock(itemId, -8);
      const low = await storage.getLowStockItems();
      assert.ok(low.some((i) => i.id === itemId));
    });

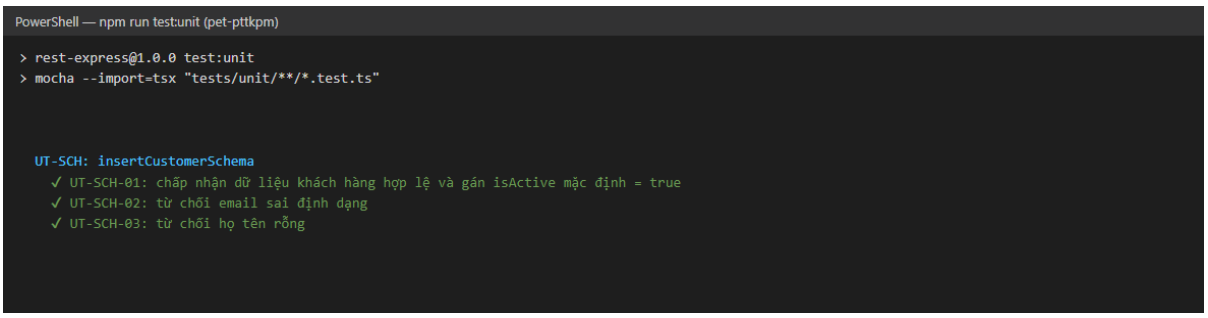
    // Lô hàng hết hạn trong 10 ngày tới phải nằm trong getExpiringBatches(30).
    it("UT-STO-11: lô hàng hết hạn trong 30 ngày nằm trong danh sách sắp hết hạn", async () =>
{
      const soon = new Date(Date.now() + 10 * 24 * 3600 * 1000);
      const batch = await storage.createStockBatch({ inventoryId: itemId, batchNumber: `UT-B-
${RUN}`, quantity: 5, expiryDate: soon });
      const expiring = await storage.getExpiringBatches(30);
      assert.ok(expiring.some((b) => b.id === batch.id));
    });

    // Đặc tả R4.1 – nhập thêm lô hàng phải cộng vào tồn kho. Hiện FAIL: createStockBatch()
chỉ lưu lô, không cập nhật tồn kho.
    it("UT-STO-12: nhập lô hàng mới làm tăng tồn kho của vật tư (R4.1)", async () => {
      const before = (await storage.getInventoryById(itemId))!.stock;
      await storage.createStockBatch({ inventoryId: itemId, batchNumber: `UT-B2-${RUN}`,
quantity: 20, expiryDate: new Date("2032-01-01") });
      const after = (await storage.getInventoryById(itemId))!.stock;
      assert.equal(after, before + 20, `Tồn kho không đổi (${before} → ${after}) sau khi nhập
lô 20`);
    });
  });
```

5.2.4. Kết quả unit test

Mã UT	Nội dung kiểm thử	Status
UT-SCH-01	chấp nhận dữ liệu khách hàng hợp lệ và gán isActive mặc định = true	Pass
UT-SCH-02	từ chối email sai định dạng	Pass
UT-SCH-03	từ chối họ tên rỗng	Pass
UT-SCH-04	chấp nhận địa chỉ rỗng vì đây là trường không bắt buộc	Fail
UT-SCH-05	từ chối số điện thoại không đủ 10 chữ số (quy tắc nghiệp vụ UC1.1)	Fail
UT-SCH-06	chuyển cân nặng kiểu số sang chuỗi (decimal)	Pass
UT-SCH-07	từ chối tuổi âm	Pass
UT-SCH-08	từ chối thú cưng thiếu loài	Pass
UT-SCH-09	chuyển chuỗi ngày giờ thành đối tượng Date, trạng thái mặc định pending	Pass
UT-SCH-10	từ chối trạng thái không thuộc danh sách cho phép	Pass
UT-SCH-11	từ chối ngày giờ không hợp lệ	Pass
UT-SCH-12	từ chối vật tư có tồn kho âm	Pass
UT-SCH-13	từ chối lô hàng có số lượng 0	Pass
UT-SCH-14	từ chối dòng hóa đơn có số lượng 0	Pass
UT-SCH-15	chỉ chấp nhận phương thức thanh toán cash, card, bank_transfer, e_wallet	Pass
UT-SCH-16	từ chối ca trực có giờ kết thúc trước giờ bắt đầu	Fail
UT-STO-01	tạo lịch hẹn hợp lệ với trạng thái mặc định pending	Pass
UT-STO-02	từ chối lịch hẹn trùng khung giờ của bác sĩ	Pass
UT-STO-03	từ chối lịch hẹn trùng khung giờ của thú cưng	Pass
UT-STO-04	hủy lịch hẹn là xóa mềm (status = cancelled), khung giờ được giải phóng	Pass
UT-STO-05	từ chối lịch hẹn có ngày trong quá khứ	Fail
UT-STO-06	điều chỉnh tồn kho cộng/trừ đúng số lượng	Pass
UT-STO-07	từ chối điều chỉnh làm tồn kho âm	Pass
UT-STO-08	thêm dòng hóa đơn tự trừ tồn kho và cập nhật tổng tiền hóa đơn	Pass
UT-STO-09	từ chối dòng hóa đơn vượt tồn kho, không thay đổi tồn kho (transaction rollback)	Pass
UT-STO-10	vật tư có tồn kho <= tồn tối thiểu nằm trong danh sách sắp hết	Pass
UT-STO-11	lô hàng hết hạn trong 30 ngày nằm trong danh sách sắp hết hạn	Pass
UT-STO-12	nhập lô hàng mới làm tăng tồn kho của vật tư (R4.1)	Fail
UT-STO-13	từ chối tạo khách hàng trùng số điện thoại	Fail
UT-STO-14	tìm kiếm khách hàng theo số điện thoại	Pass

Bảng 15: Bảng kết quả kiểm thử đơn vị



Hình 19: Kết quả chạy unit test bằng Mocha

Kết quả: 24/30 unit test Pass, 6 Fail. Các unit test Fail trùng khớp với lỗi tìm thấy ở kiểm thử giao diện và chỉ rõ nguyên nhân gốc:

- **UT-SCH-04:** nullableString = z.string().min(1) từ chối chuỗi rỗng → nguyên nhân của UC1.1-TC010.
- **UT-SCH-05:** schema phía server không kiểm tra SĐT 10 chữ số – chỉ form phía client kiểm tra, gọi API trực tiếp vẫn lưu được SĐT "123".
- **UT-SCH-16, UT-STO-05, UT-STO-13:** thiếu kiểm tra giờ ca trực, lịch hẹn quá khứ, SĐT trùng.
- **UT-STO-12:** nhập thêm lô hàng không làm tăng tồn kho của vật tư (R4.1).

5.3. Kiểm thử tự động bằng Mocha và Selenium WebDriver

5.3.1. Giới thiệu về Mocha

Mocha là framework kiểm thử đơn giản, linh hoạt và phổ biến cho Node.js:

- Hỗ trợ nhiều dạng kiểm thử như unit test, integration test, end-to-end test.
- Chạy đồng bộ và bất đồng bộ (async/await), phù hợp điều khiển trình duyệt.
- Cung cấp nhiều dạng báo cáo (spec, json…), dễ theo dõi kết quả.

5.3.2. Giới thiệu về Selenium WebDriver

WebDriver là khuôn khổ tự động hóa web trong bộ Selenium, cho phép điều khiển các trình duyệt khác nhau:

- Miễn phí, mã nguồn mở.
- Kiến trúc đơn giản: điều khiển trình duyệt trực tiếp qua driver (chromedriver, msedgedriver, geckodriver được Selenium Manager tự tải theo phiên bản trình duyệt).
- Hỗ trợ nhiều ngôn ngữ: Java, C#, Python, JavaScript…
- Tốc độ cao do tương tác trực tiếp với trình duyệt.

Cách cài đặt:

```
cd PetClinic_TestCode/02_AutomationTest
npm install                   # selenium-webdriver, mocha, cross-env

# Ứng dụng phải đang chạy tại http://localhost:5000
npm run test:edge             # Microsoft Edge (mặc định)
npm run test:chrome           # Google Chrome
npm run test:firefox          # Mozilla Firefox
```

5.3.3. Cấu trúc bộ kiểm thử tự động

File	Vai trò
automation/helpers.js	Khởi tạo trình duyệt headless (Chrome/Edge/Firefox), các hàm thao tác: mở trang, nhập liệu, chọn dropdown (Radix Select), chờ thông báo, chụp màn hình, gọi API
automation/cases.js	Định nghĩa 74 test case: mã, mô tả, các bước, dữ liệu, kết quả mong đợi và hàm run() thực thi
automation/petclinic.spec.js	Bộ Mocha: duyệt các test case theo nhóm tác nhân, ghi kết quả ra results.json
automation/results.json	Kết quả Pass/Fail và kết quả thực tế của từng test case – nguồn của cột Status ở Chương 3

Bảng 16: Bảng cấu trúc mã nguồn kiểm thử tự động

5.3.4. Mã nguồn kiểm thử tự động

helpers.js – khởi tạo trình duyệt và chọn giá trị trong dropdown (trích):

```
let driver;

/**
 * Mở trình duyệt.
 * Chọn loại trình duyệt bằng biến môi trường BROWSER = chrome | edge | firefox (mặc định:
edge).
 * Cửa sổ 1440×900, ngôn ngữ tiếng Việt, chạy ẩn (headless).
 * Selenium Manager tự tải driver (chromedriver/msedgedriver/geckodriver) phù hợp phiên bản
trình duyệt.
 */
async function start() {
  const browser = (process.env.BROWSER || "edge").toLowerCase();
  const args = ["--headless=new", "--window-size=1440,900", "--lang=vi-VN"];
  const builder = new Builder();
  if (browser === "chrome") {
    builder.forBrowser("chrome").setChromeOptions(new chrome.Options().addArguments(...args));
  } else if (browser === "firefox") {
    builder.forBrowser("firefox").setFirefoxOptions(new firefox.Options().addArguments("-
headless", "--width=1440", "--height=900"));
  } else {
    builder.forBrowser("MicrosoftEdge").setEdgeOptions(new
edge.Options().addArguments(...args));
  }
  driver = await builder.build();
  return driver;
}

/** Đóng trình duyệt khi chạy xong toàn bộ test. */
```

```
async function selectByLabel(label, optionText, scope = "//div[@role='dialog']") {
  const trigger = await driver.wait(
    until.elementLocated(xp(`${scope}//label[normalize-
space()=${lit(label)}]/following::button[@role='combobox'][1]`)),
    8000,
  );
```

```
  await selectOption(trigger, optionText);
}

/**
 * Chọn một lựa chọn trong danh sách thả xuống (component Select của thư viện Radix UI).
 * 1. Bấm vào ô chọn (trigger) để mở danh sách.
 * 2. Tìm dòng lựa chọn (role="option") có chứa chữ cần chọn.
 * 3. Danh sách có thể dài (30 khách hàng, 50 thú cưng) nên phải cuộn tới lựa chọn rồi mới
bấm;
 *    thử tối đa 4 lần nếu bị phần tử khác che.
 */
async function selectOption(trigger, optionText) {
  await driver.executeScript("arguments[0].scrollIntoView({block:'center'})", trigger);
  await trigger.click();
  const opt = await
driver.wait(until.elementLocated(xp(`//*[@role='option'][contains(normalize-space(),
${lit(optionText)})]`)), 8000);
  for (let i = 0; i < 4; i++) {
    await driver.executeScript("arguments[0].scrollIntoView({block:'center'})", opt);
    await sleep(250);
    try {
      await opt.click();
      break;
    } catch (e) {
      if (i === 3) throw e;
    }
  }
  await sleep(400);
}

/** Lấy toàn bộ chữ đang hiển thị trên trang – dùng để kiểm tra thông báo, kết quả tìm kiếm.
*/
```

petclinic.spec.js – bộ kiểm thử Mocha:

```
/**
 * petclinic.spec.js – FILE CHẠY BỘ KIỂM THỬ TỰ ĐỘNG GIAO DIỆN (Mocha)
 * ---------------------------------------------------------------------
 * Cách chạy:  npx mocha petclinic.spec.js --reporter spec
 *             (ứng dụng phải đang chạy ở http://localhost:5000)
 *
 * Mocha là "người điều phối": gom các test case thành nhóm (describe), chạy từng case (it),
 * đếm số Pass/Fail và in kết quả ra màn hình.
 * File này không chứa kịch bản cụ thể – kịch bản nằm trong cases.js.
 *
 * Trình tự:
 *   1. before : mở trình duyệt, đọc dữ liệu mẫu (khách hàng, thú cưng, bác sĩ…)
 *   2. Với mỗi nhóm tác nhân, với mỗi test case: chạy run(), ghi Pass/Fail + kết quả thực tế
 *   3. after  : đóng trình duyệt, ghi toàn bộ kết quả ra results.json (dùng để làm báo cáo,
file Excel)
 */
const fs = require("fs");
const path = require("path");
const h = require("./helpers");
const { cases, loadContext } = require("./cases");

// Kết quả từng test case: { "UC1.1-TC001": { status: "Pass", actual: "...", ms: 3600 }, ... }
const results = {};

describe("Pet Clinic – Kiểm thử tự động (Mocha + Selenium WebDriver / Microsoft Edge)",
function () {
  // Mỗi test case được phép chạy tối đa 90 giây (thao tác giao diện chậm hơn gọi API).
  this.timeout(90000);
```

```
  before(async () => {
    await h.start(); // mở trình duyệt
    await loadContext(); // đọc dữ liệu mẫu để các test case dùng
  });

  after(async () => {
    await h.stop(); // đóng trình duyệt
    fs.writeFileSync(path.join(__dirname, "results.json"), JSON.stringify(results, null, 2));
  });

  // Tạo một nhóm describe cho mỗi tác nhân, giữ nguyên thứ tự trong cases.js.
  const groups = [...new Set(cases.map((c) => c.group))];
  for (const group of groups) {
    describe(group, () => {
      for (const c of cases.filter((x) => x.group === group)) {
        it(`${c.id}: ${c.title}`, async () => {
          const t0 = Date.now();
          try {
            // run() trả về "kết quả thực tế" nếu đạt → ghi Pass
            const actual = await c.run();
            results[c.id] = { status: "Pass", actual, ms: Date.now() - t0 };
          } catch (e) {
            // run() ném lỗi nếu không đạt → ghi Fail kèm mô tả, rồi ném tiếp để Mocha đếm là
failing
            results[c.id] = { status: "Fail", actual: e.actual || `Lỗi khi thực thi:
${e.message.split("\n")[0]}`, ms: Date.now() - t0 };
            throw e;
          } finally {
            // Dù Pass hay Fail: chụp ảnh (nếu test case yêu cầu) và đóng hộp thoại còn mở
            if (c.shot) await h.shot(c.shot).catch(() => {});
            await h.closeDialogs().catch(() => {});
          }
        });
      }
    });
  }
});
```

cases.js – ví dụ hai test case UC1.1-TC007 (hợp lệ) và UC2.1-TC003 (trùng giờ bác sĩ):

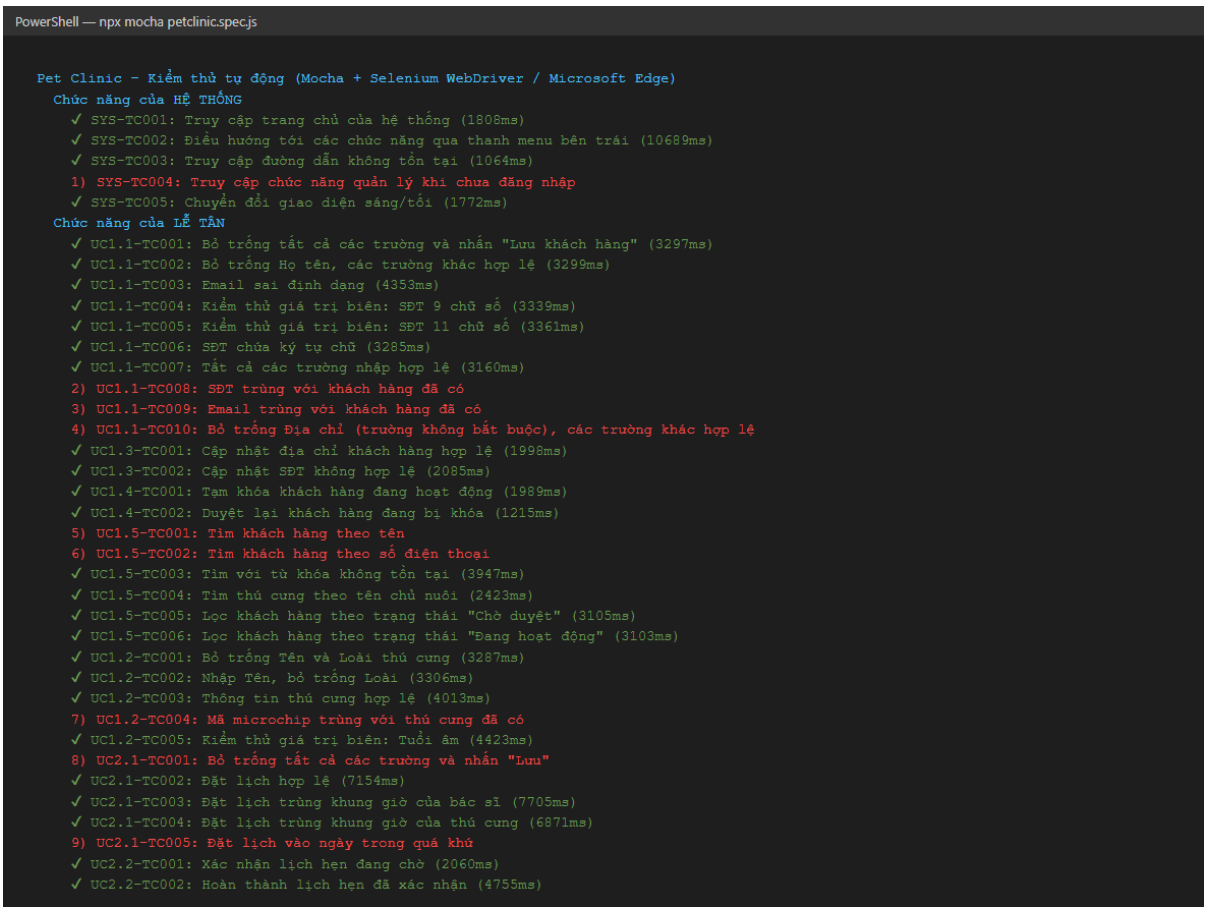
```
  id: "UC1.1-TC007", group: G2, uc: UC11, shot: "uc11_success",
  title: "Tất cả các trường nhập hợp lệ",
  steps: [
    ["1. Chọn \"Thêm khách hàng\"", "Form hiển thị", ""],
    ["2. Nhập Họ tên", "", ctx.newCustomer.name],
    ["3. Nhập Email", "", ctx.newCustomer.email],
    ["4. Nhập SĐT", "", ctx.newCustomer.phone],
    ["5. Nhập Địa chỉ", "", ctx.newCustomer.address],
    ["6. Nhấn \"Lưu khách hàng\"", "Hiển thị thông báo thành công", ""],
  ],
  expected: "Hiển thị \"Thêm khách hàng thành công\", khách hàng xuất hiện trong danh sách",
  async run() {
    await openAddCustomer();
    await fillCustomer(ctx.newCustomer);
    const toast = await h.waitText("Thêm khách hàng thành công", 3000);
    const listed = await h.waitText(ctx.newCustomer.name, 3000);
    return expect(toast && listed, "Thông báo thành công, khách hàng có trong danh sách",
"Không lưu được khách hàng");
  },
});

// ──────────────────────────────────────────────────────────────────────
// UC1.1-TC008: SĐT trùng với khách hàng đã có
```

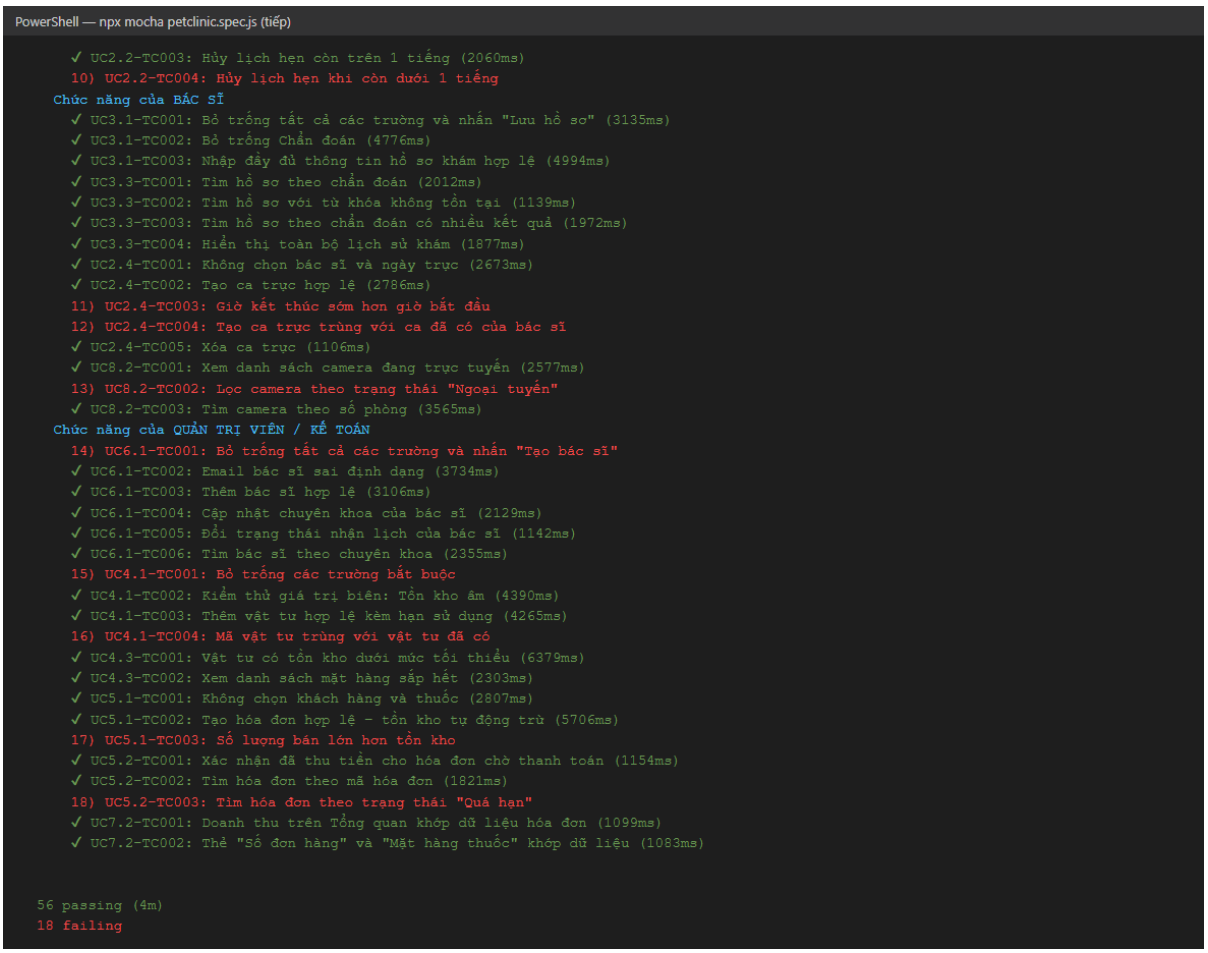
```
// Pass khi: Hiển thị lỗi số điện thoại đã tồn tại, không tạo khách hàng (quy tắc nghiệp vụ
UC1.1)
// ──────────────────────────────────────────────────────────────────────────
add({
  id: "UC2.1-TC003", group: G2, uc: UC21,
  title: "Đặt lịch trùng khung giờ của bác sĩ",
  steps: [
    ["1. Chọn \"Tạo lịch hẹn\"", "Form hiển thị", ""],
    ["2. Chọn khách hàng, thú cưng khác", "", "Trần Lan"],
    ["3. Chọn cùng Bác sĩ, Ngày, Giờ với TC002", "", `${tomorrow} 15:00`],
    ["4. Chọn dịch vụ, nhấn lưu", "Hiển thị thông báo lỗi", "Tiêm phòng"],
  ],
  expected: "Hiển thị \"Không thể đặt lịch\" – bác sĩ đã có lịch trong khung giờ",
  async run() {
    const lan = ctx.customers.find((c) => c.name === "Trần Lan");
    const lanPet = ctx.pets.find((p) => p.customerId === lan.id);
    await openAppointmentDialog();
    await fillAppointment({ customer: "Trần Lan", pet: lanPet.name, vet: ctx.vet.name, date:
tomorrow, time: "15:00", service: "Tiêm phòng" });
    const t = await h.bodyText();
    return expect(t.includes("Không thể đặt lịch"), "Hiển thị \"Không thể đặt lịch\"
(Veterinarian already has an appointment in this timeslot)", "Lịch trùng vẫn được tạo");
  },
});
// ──────────────────────────────────────────────────────────────────────────
// UC2.1-TC004: Đặt lịch trùng khung giờ của thú cưng
// Pass khi: Hiển thị "Không thể đặt lịch" – thú cưng đã có lịch trong khung giờ
// ──────────────────────────────────────────────────────────────────────────
add({
```

5.3.5. Kết quả kiểm thử tự động

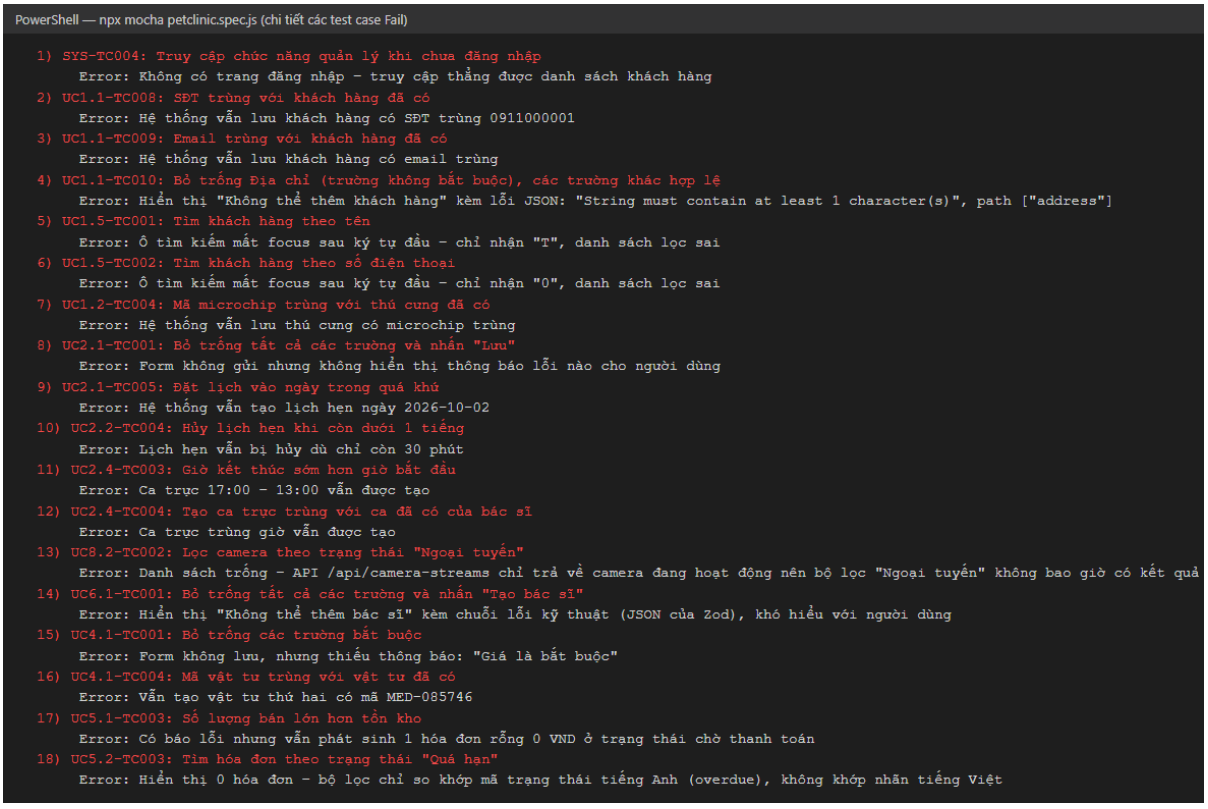
Kết quả dưới đây là lượt chạy đầy đủ trên Microsoft Edge (chế độ headless), dữ liệu mẫu được nạp lại trước khi chạy. Cùng bộ kịch bản có thể chạy lại trên Chrome và Firefox bằng biến môi trường BROWSER.



Hình 20: Kết quả chạy kiểm thử tự động (phần 1)



Hình 21: Kết quả chạy kiểm thử tự động (phần 2)



Hình 22: Chi tiết các test case Fail trong lần chạy tự động

5.3.6. Mục tiêu và phạm vi kiểm thử

- **Mục tiêu:** tự động hóa các tác vụ kiểm thử để đảm bảo chức năng hoạt động đúng yêu cầu; tiết kiệm thời gian so với kiểm thử thủ công (toàn bộ 74 test case chạy trong khoảng 4 phút); dùng làm bộ kiểm thử hồi quy sau mỗi lần sửa lỗi.
- **Phạm vi:** điều hướng, quản lý khách hàng và thú cưng, lịch hẹn, ca bác sĩ, hồ sơ khám, camera, bác sĩ, kho thuốc, hóa đơn, thanh toán và báo cáo doanh thu.

5.4. Kiểm thử hiệu năng: tải và chịu tải

5.4.1. Giới thiệu công cụ autocannon

autocannon là công cụ đo hiệu năng HTTP mã nguồn mở viết bằng Node.js, có thể chạy từ dòng lệnh hoặc gọi trong mã kiểm thử:

- Giả lập hàng trăm đến hàng nghìn kết nối đồng thời, gửi yêu cầu liên tục trong thời gian định trước.
- Hỗ trợ nhiều kịch bản: một API, chuỗi API luân phiên, yêu cầu ghi (POST) với dữ liệu sinh động.
- Báo cáo đầy đủ phân phối thời gian phản hồi (p50, p90, p99, max), thông lượng req/s, số lỗi và timeout.
- Tích hợp được với Mocha để mỗi kịch bản là một test case có tiêu chí Pass/Fail tự động.

5.4.2. Cài đặt và chạy

```
cd PetClinic_TestCode/03_PerformanceTest
npm install                       # autocannon, mocha, mysql2


# Ứng dụng chạy bản production: npm run build && npm start
npm run test:load                 # kiểm thử tải – 200 người dùng
npm run test:search               # tìm kiếm trên 10.000 bản ghi (cần DATABASE_URL)
npm run test:stress               # kiểm thử chịu tải – tăng dần kết nối


# Tùy chỉnh: $env:LOAD_USERS=300; $env:LOAD_DURATION=60; $env:STRESS_LEVELS="100,500,1000"
```

5.4.3. Mã nguồn kiểm thử hiệu năng

perf/load.test.js – mỗi kịch bản tải là một test case Mocha (trích):

```
describe(`Kiểm thử tải – ${USERS} người dùng đồng thời, ${DURATION}s/kịch bản`, function () {
  this.timeout((DURATION + 30) * 1000);
  after(() => save("load.json", results));


  for (const sc of scenarios) {
    it(`${sc.id}: ${sc.name}`, async () => {
      // Chạy autocannon cho kịch bản, rút gọn kết quả, in ra màn hình
      const res = await run({ connections: USERS, duration: DURATION, path: sc.path, requests:
sc.requests });
      const s = { id: sc.id, ...summarize(sc.name, res) };
      results.push(s);
```

```
      printRow(s);
      // Tiêu chí PASS: không có yêu cầu lỗi VÀ p99 ≤ 3 giây
      assert.equal(s.errorRatePct, 0, `Có ${s.errors + s.timeouts + s.non2xx} yêu cầu lỗi`);
      assert.ok(s.latencyP99Ms <= SLA_MS, `p99 = ${s.latencyP99Ms} ms vượt ${SLA_MS} ms`);
    });
  }
});
```

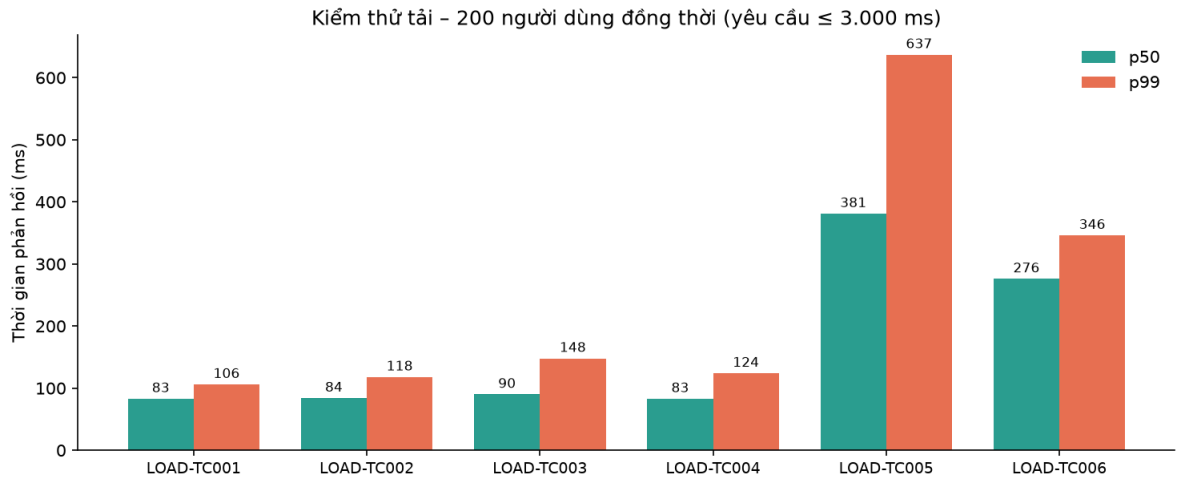
perf/stress.test.js – tăng dần kết nối và xác định ngưỡng quá tải (trích):

```
  it("STRESS-TC001: Tăng dần tải và xác định ngưỡng quá tải", async () => {
    // Lần lượt tăng tải; mức nào có p99 > 3 giây hoặc lỗi > 1% thì đánh dấu "quá tải"
    for (const c of LEVELS) {
      const s = summarize(`Mức ${c} kết nối`, await run({ connections: c, duration: STEP_S,
requests: MIXED }));
      s.overloaded = s.latencyP99Ms > 3000 || s.errorRatePct > 1;
      steps.push(s);
      printRow(s);
    }
    const bp = steps.find((s) => s.overloaded);
    console.log(bp ? `  → Ngưỡng quá tải: ${bp.connections} kết nối` : "  → Chưa chạm ngưỡng
quá tải trong dải đã thử");
    // PASS nếu ở mọi mức ≤ 200 kết nối (mức đặc tả yêu cầu) hệ thống đều chưa quá tải
    assert.ok(steps.filter((s) => s.connections <= 200).every((s) => !s.overloaded), "Hệ thống
quá tải ở mức ≤ 200 kết nối (yêu cầu đặc tả)");
  });
```

5.4.4. Kết quả kiểm thử tải

Kịch bản	Kết nối	Yêu cầu	req/s	p50 (ms)	p99 (ms)	Lỗi	Status
LOAD-TC001 Trang Tổng quan – GET /api/dashboard/stats	200	71.419	2381	83	106	0%	Pass
LOAD-TC002 Danh sách khách hàng – GET /api/customers	200	69.616	2321	84	118	0%	Pass
LOAD-TC003 Danh sách lịch hẹn – GET /api/appointments	200	62.641	2088	90	148	0%	Pass
LOAD-TC004 Kho thuốc – GET /api/inventory	200	70.986	2367	83	124	0%	Pass
LOAD-TC005 Kịch bản hỗn hợp (đọc nhiều màn hình)	200	15.558	519	381	637	0%	Pass
LOAD-TC006 Tải ghi – POST /api/customers (lễ tân tạo hồ sơ)	200	21.135	705	276	346	0%	Pass

Bảng 17: Bảng kết quả kiểm thử tải với 200 người dùng đồng thời



Hình 23: Thời gian phản hồi p50/p99 của các kịch bản tải

```
PowerShell — npm run test:load

  Kiểm thử tải – 200 người dùng đồng thời, 30s/kịch bản
Trang Tổng quan – GET /api/dashboard/stats conn= 200  req=  71419  rps= 2381  p50=   83ms  p99=  106ms  max=  303ms  lỗi=0%
    ✓ LOAD-TC001: Trang Tổng quan – GET /api/dashboard/stats (30189ms)
Danh sách khách hàng – GET /api/customers  conn= 200  req=  69616  rps= 2321  p50=   84ms  p99=  118ms  max=  265ms  lỗi=0%
    ✓ LOAD-TC002: Danh sách khách hàng – GET /api/customers (30171ms)
Danh sách lịch hẹn – GET /api/appointments conn= 200  req=  62641  rps= 2088  p50=   90ms  p99=  148ms  max=  273ms  lỗi=0%
    ✓ LOAD-TC003: Danh sách lịch hẹn – GET /api/appointments (30111ms)
Kho thuốc – GET /api/inventory             conn= 200  req=  70986  rps= 2367  p50=   83ms  p99=  124ms  max=  281ms  lỗi=0%
    ✓ LOAD-TC004: Kho thuốc – GET /api/inventory (30064ms)
Kịch bản hỗn hợp (đọc nhiều màn hình)      conn= 200  req=  15558  rps=  519  p50=  381ms  p99=  637ms  max=  726ms  lỗi=0%
    ✓ LOAD-TC005: Kịch bản hỗn hợp (đọc nhiều màn hình) (30111ms)
Tải ghi – POST /api/customers (lễ tân tạo hồ sơ) conn= 200  req=  21135  rps=  705  p50=  276ms  p99=  346ms  max=  753ms  lỗi=0%
    ✓ LOAD-TC006: Tải ghi – POST /api/customers (lễ tân tạo hồ sơ) (30073ms)

  6 passing (3m)
```

Hình 24: Kết quả chạy kiểm thử tải

Tìm kiếm trên 10.030 khách hàng: một người dùng có thời gian trung bình 26 ms, p95 36 ms; 50 người dùng đồng thời đạt 2369 req/s với p99 33 ms – đạt yêu cầu ≤ 2 giây.

```
PowerShell — npm run test:search

  Hiệu năng tìm kiếm trên 10.000 khách hàng
    10030 khách hàng – trung bình 26 ms, p95 36 ms, max 99 ms
    ✓ PERF-TC001: Một người dùng – tìm theo tên, SĐT, email (50 lượt) (1299ms)
Tìm kiếm đồng thời trên 10.000 KH          conn=  50  req=  47376  rps= 2369  p50=   20ms  p99=   33ms  max=   86ms  lỗi=0%
    ✓ PERF-TC002: 50 người dùng tìm kiếm đồng thời trong 20 giây (20058ms)

  2 passing (22s)
```

Hình 25: Kết quả kiểm thử tìm kiếm trên 10.000 bản ghi

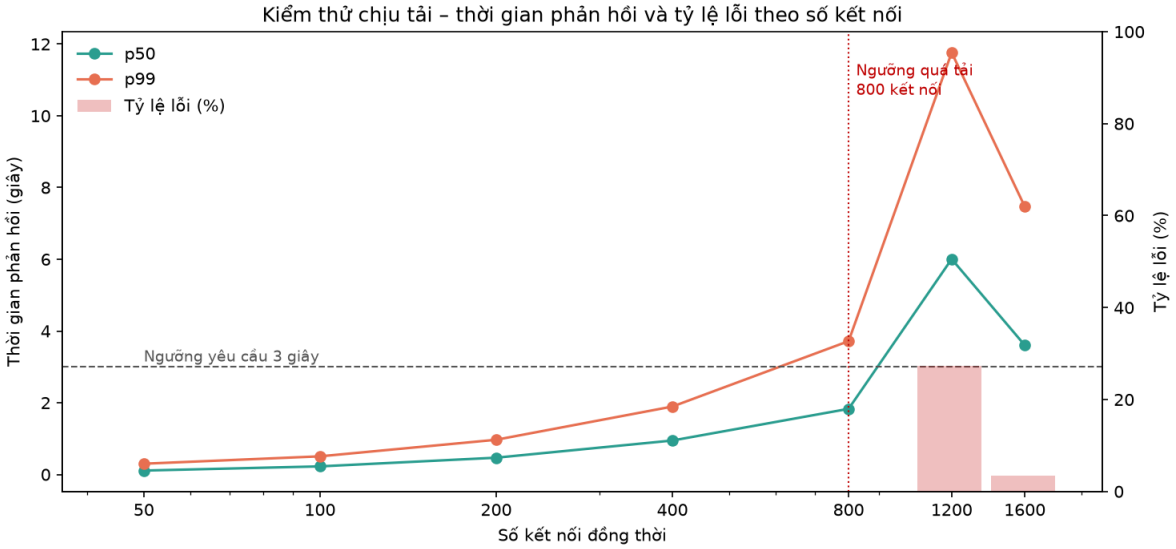
Nhận xét: cả 6 kịch bản tải đạt yêu cầu: p99 cao nhất 637 ms, thấp hơn nhiều so với ngưỡng 3 giây, không có yêu cầu lỗi. Kịch bản hỗn hợp chậm hơn vì trả về nhiều dữ liệu liên kết (thú cưng, hồ sơ khám, hóa đơn).

5.4.5. Kết quả kiểm thử chịu tải

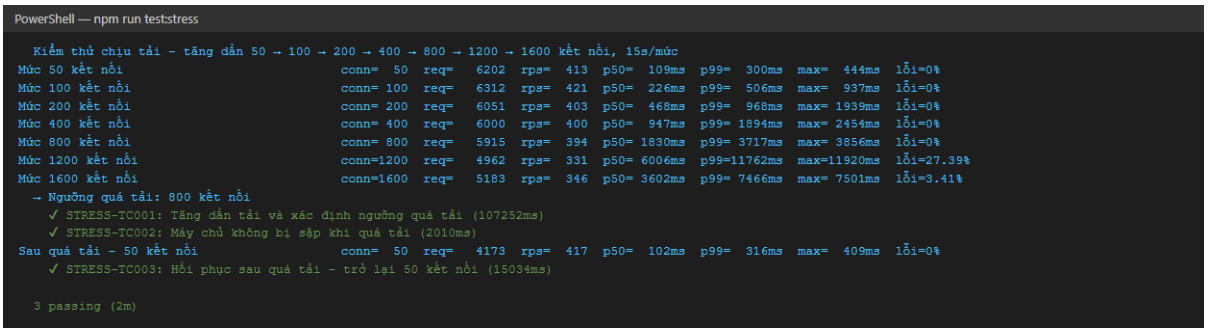
Số kết nối	req/s	p50 (ms)	p99 (ms)	Max (ms)	Tỷ lệ lỗi	Đánh giá
50	413	109	300	444	0%	Đạt
100	421	226	506	937	0%	Đạt
200	403	468	968	1939	0%	Đạt
400	400	947	1894	2454	0%	Đạt

Số kết nối	req/s	p50 (ms)	p99 (ms)	Max (ms)	Tỷ lệ lỗi	Đánh giá
800	394	1830	3717	3856	0%	Quá tải
1200	331	6006	11762	11920	27.39%	Quá tải
1600	346	3602	7466	7501	3.41%	Quá tải
50 (hồi phục)	417	102	316	409	0%	Hồi phục

Bảng 18: Bảng kết quả kiểm thử chịu tải theo từng mức kết nối



Hình 26: Thời gian phản hồi và tỷ lệ lỗi khi tăng dần tải



Hình 27: Kết quả chạy kiểm thử chịu tải

Nhận xét: thông lượng của kịch bản hỗn hợp đạt khoảng 421 req/s và gần như không tăng khi thêm kết nối, nên thời gian phản hồi tăng tuyến tính theo số kết nối. Hệ thống đạt yêu cầu tới 400 kết nối (gấp đôi yêu cầu 200), bắt đầu quá tải từ 800 kết nối và phát sinh lỗi timeout từ 1200 kết nối. Máy chủ không bị sập và hồi phục hoàn toàn khi tải giảm về 50 kết nối. Đề xuất: thêm phân trang và cache cho các API danh sách, tăng connectionLimit của pool MySQL, chạy nhiều tiến trình Node (cluster/PM2) khi triển khai.

5.5. Kiểm thử toàn vẹn dữ liệu

5.5.1. Cài đặt và chạy

```
cd PetClinic_TestCode/04_DataIntegrityTest
npm install
$env:BASE_URL = "http://localhost:5000"
$env:DATABASE_URL = "mysql://<user>:<pass>@localhost:3306/petclinic_test"
npm run test:integrity
```

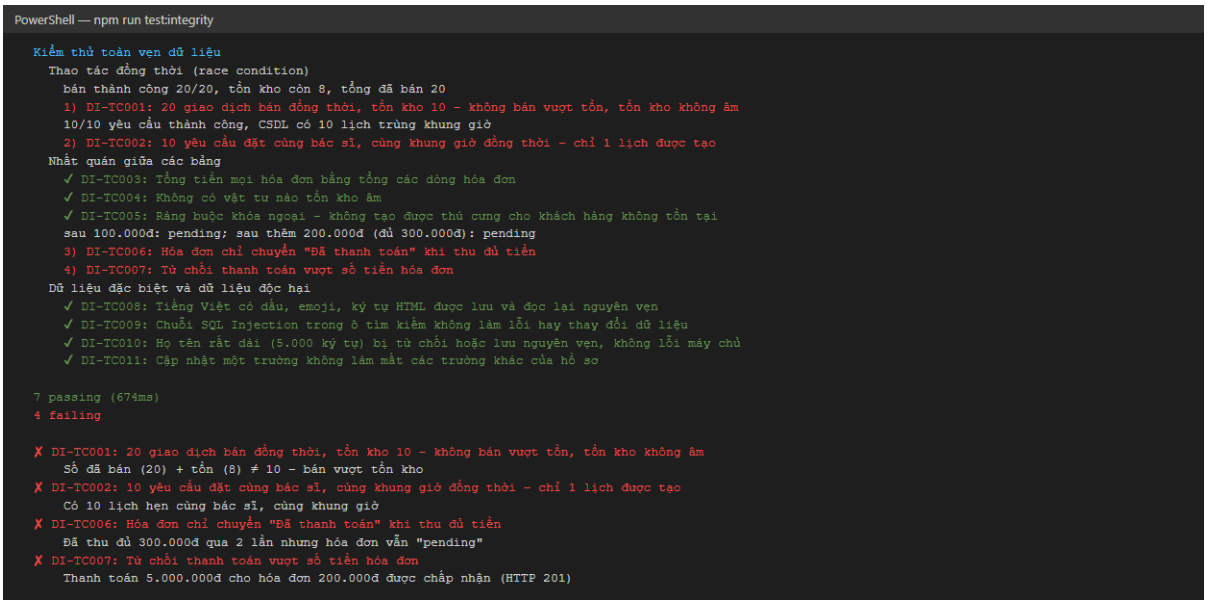
5.5.2. Mã nguồn kiểm thử toàn vẹn dữ liệu

integrity/integrity.test.js – kiểm tra bán thuốc đồng thời (race condition) và đối chiếu trực tiếp trong CSDL (trích):

```
    it("DI-TC001: 20 giao dịch bán đồng thời, tồn kho 10 – không bán vượt tồn, tồn kho không
âm", async () => {
      // Ý tưởng: vật tư chỉ có 10 hộp, 20 người cùng bấm "bán 1 hộp" ĐỒNG THỜI (Promise.all).
      // Hệ thống đúng: 10 giao dịch thành công, 10 bị từ chối, tồn kho về 0.
      // Nếu hệ thống đọc tồn kho rồi mới trừ mà không khóa → nhiều giao dịch đọc cùng giá trị
cũ → bán vượt (race condition).
      const item = (await api("POST", "/api/inventory", { code: `DI-${RUN}`, name: `DI Thuốc
${RUN}`, category: "Medicine", unit: "hộp", stock: 10, minStock: 1, price: "10000" })).data;
      const invoices = await Promise.all(Array.from({ length: 20 }, (_, i) =>
        api("POST", "/api/invoices", { customerId: customer.id, invoiceNumber: `DI-INV-${RUN}-
${i}`, totalAmount: "0" }).then((r) => r.data)));
      const sales = await Promise.all(invoices.map((inv) =>
        api("POST", "/api/invoice-items", { invoiceId: inv.id, inventoryId: item.id,
description: item.name, quantity: 1, unitPrice: "10000", totalPrice: "10000" })));
      const ok = sales.filter((s) => s.status === 201).length;
      const [[row]] = await db.query("SELECT stock FROM inventory WHERE id = ?", [item.id]);
      // Đối chiếu trong CSDL: tổng số đã bán + tồn kho còn lại phải bằng 10
      const [[sold]] = await db.query("SELECT COALESCE(SUM(ii.quantity),0) q FROM
invoice_items ii WHERE ii.inventory_id = ?", [item.id]);
      console.log(`      bán thành công ${ok}/20, tồn kho còn ${row.stock}, tổng đã bán
${sold.q}`);
      assert.ok(row.stock >= 0, `Tồn kho âm: ${row.stock}`);
      assert.equal(Number(sold.q), 10 - row.stock, `Số đã bán (${sold.q}) + tồn (${row.stock})
≠ 10 – bán vượt tồn kho`);
      assert.equal(ok, 10, `${ok} giao dịch thành công trong khi chỉ có 10 hộp`);
    });
```

```
    it("DI-TC003: Tổng tiền mọi hóa đơn bằng tổng các dòng hóa đơn", async () => {
      // Với MỌI hóa đơn: cột total_amount phải bằng tổng total_price các dòng hóa đơn
      const [rows] = await db.query(`
        SELECT i.invoice_number, i.total_amount, COALESCE(SUM(ii.total_price),0) s
        FROM invoices i LEFT JOIN invoice_items ii ON ii.invoice_id = i.id
        GROUP BY i.id HAVING ABS(i.total_amount - s) > 0.001`);
      assert.equal(rows.length, 0, `${rows.length} hóa đơn lệch tổng tiền: ${rows.slice(0,
3).map((r) => r.invoice_number).join(", ")}`);
    });
```

5.5.3. Kết quả kiểm thử toàn vẹn dữ liệu



Hình 28: Kết quả chạy kiểm thử toàn vẹn dữ liệu

Kết quả: 7/11 test case đạt. Dữ liệu tĩnh nhất quán: tổng tiền hóa đơn khớp các dòng, không có tồn kho âm, khóa ngoại được CSDL bảo vệ, dữ liệu tiếng Việt, emoji và ký tự HTML được lưu nguyên vẹn, truy vấn có tham số nên miễn nhiễm SQL Injection. Tuy nhiên phát hiện hai lỗi nghiêm trọng khi thao tác đồng thời:

- **DI-TC001 – bán vượt tồn kho:** 20 giao dịch bán đồng thời trên vật tư chỉ còn 10 hộp đều thành công, tồn kho ghi nhận còn 8 trong khi đã bán 20. Nguyên nhân: các giao dịch cùng đọc giá trị tồn kho cũ rồi ghi đè (lost update). Khắc phục: dùng UPDATE inventory SET stock = stock - ? WHERE id = ? AND stock >= ? và kiểm tra số dòng bị ảnh hưởng, hoặc SELECT … FOR UPDATE trong transaction.
- **DI-TC002 – trùng lịch khi đặt đồng thời:** 10 yêu cầu cùng bác sĩ, cùng giờ đều được tạo. Khắc phục: thêm ràng buộc UNIQUE (veterinarian_id, appointment_date) cho lịch chưa hủy hoặc khóa bản ghi khi kiểm tra.
- **DI-TC006, DI-TC007 – thanh toán:** thu tiền nhiều lần không cộng dồn nên hóa đơn không chuyển "Đã thanh toán", và hệ thống chấp nhận thu vượt số tiền hóa đơn.

5.6. Kiểm thử độ tin cậy và phục hồi

5.6.1. Cài đặt và chạy

Bộ test tự khởi chạy một tiến trình ứng dụng riêng ở cổng 5055 từ bản build production và cần lệnh dừng/khởi động MySQL phù hợp môi trường:

```
cd PetClinic_TestCode/05_ReliabilityTest
npm install
$env:APP_DIR      = "..\..\pet-pttkpm\pet-pttkpm"     # đã chạy npm run build
$env:DATABASE_URL = "mysql://<user>:<pass>@localhost:3306/petclinic_test"
$env:DB_STOP_CMD  = "net stop MySQL80"                 # hoặc: docker stop petclinic-db
$env:DB_START_CMD = "net start MySQL80"                # hoặc: docker start petclinic-db
$env:SOAK_S       = "180"
npm run test:reliability
```

5.6.2. Mã nguồn kiểm thử độ tin cậy và phục hồi

reliability/reliability.test.js – mất CSDL và tự phục hồi (trích):

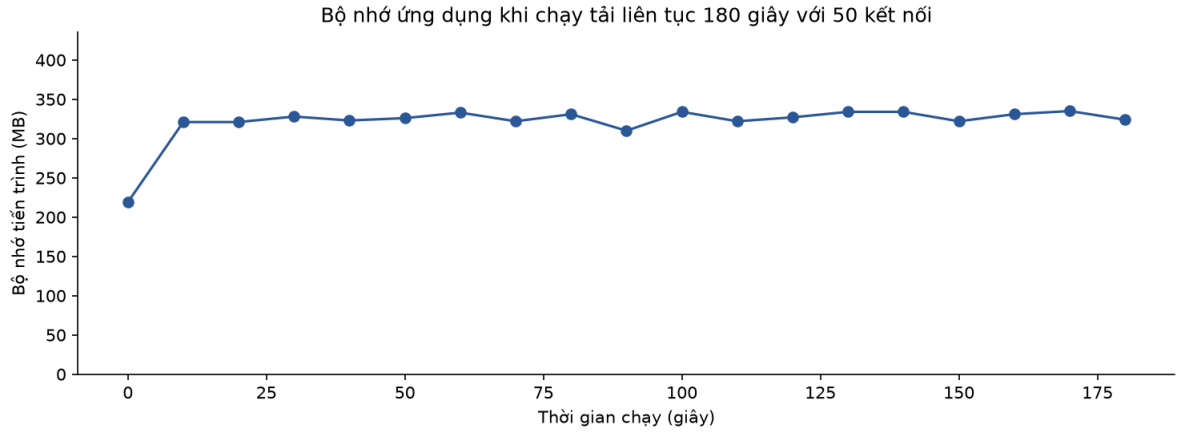
```
  it("REL-TC002: Mất kết nối CSDL – API báo lỗi có kiểm soát, ứng dụng không bị sập", async
function () {
    if (!process.env.DB_STOP_CMD) this.skip();
    // Sự cố giả lập: dừng MySQL khi ứng dụng đang chạy → API phải trả lỗi 5xx, tiến trình
KHÔNG được thoát
    execSync(process.env.DB_STOP_CMD, { stdio: "ignore" });
    await sleep(3000);
    const status = await get("/api/customers", 15000);
    await sleep(1000);
    out.dbDown = { apiStatus: status, appAlive: appExit === null };
    console.log(`      khi CSDL dừng: HTTP ${status}, tiến trình ứng dụng ${appExit === null ?
"vẫn chạy" : `đã thoát (mã ${appExit})`}`);
    assert.equal(appExit, null, `Ứng dụng bị sập (exit code ${appExit}) khi mất CSDL`);
    assert.ok(status >= 500 && status < 600, `HTTP ${status}`);
  });


  it("REL-TC003: Tự phục hồi khi CSDL hoạt động trở lại (không cần khởi động lại ứng dụng)
trong ≤ 30 giây", async function () {
    if (!process.env.DB_START_CMD) this.skip();
    // Bật lại MySQL, KHÔNG khởi động lại ứng dụng; đo thời gian tới khi API trả 200 trở lại
    execSync(process.env.DB_START_CMD, { stdio: "ignore" });
    const t0 = Date.now();
    await waitFor(async () => (await get("/api/dashboard/stats").catch(() => 0)) !== 0 ||
appExit !== null, 5000);
    const ms = await waitFor(async () => appExit === null && (await get("/api/customers")) ===
200, 60000, 500);
    out.dbRecoveryMs = ms === null ? null : Date.now() - t0;
    console.log(`      ${appExit !== null ? "ứng dụng đã thoát, không thể tự phục hồi" : `phục
hồi sau ${out.dbRecoveryMs} ms`}`);
    if (appExit !== null) startApp(); // khởi động lại để các test sau chạy được
    await waitFor(async () => (await get("/api/dashboard/stats")) === 200, 60000);
    assert.ok(ms !== null && out.dbRecoveryMs <= 30000, "Ứng dụng không tự phục hồi");
  });
```

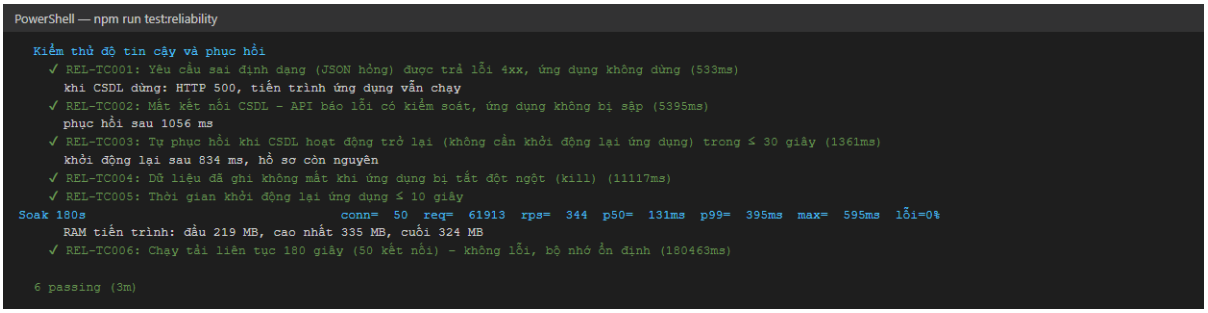
5.6.3. Kết quả kiểm thử độ tin cậy và phục hồi

Chỉ số	Kết quả đo	Yêu cầu
Khởi động ứng dụng (lần đầu)	841 ms	–
Phản hồi khi mất CSDL	HTTP 500, ứng dụng không sập	Báo lỗi có kiểm soát, không sập
Thời gian tự phục hồi khi CSDL hoạt động lại	1.056 ms	≤ 30.000 ms
Khởi động lại sau khi bị tắt đột ngột	834 ms	≤ 10.000 ms
Chạy tải liên tục 180 giây (50 kết nối)	61.913 yêu cầu, lỗi 0%, p99 395 ms	Lỗi 0%, p99 ≤ 3.000 ms
Bộ nhớ tiến trình khi chạy liên tục	219 MB → cao nhất 335 MB → cuối 324 MB	Không tăng liên tục

Bảng 19: Bảng kết quả kiểm thử độ tin cậy và phục hồi



Hình 29: Bộ nhớ ứng dụng trong thời gian chạy tải liên tục



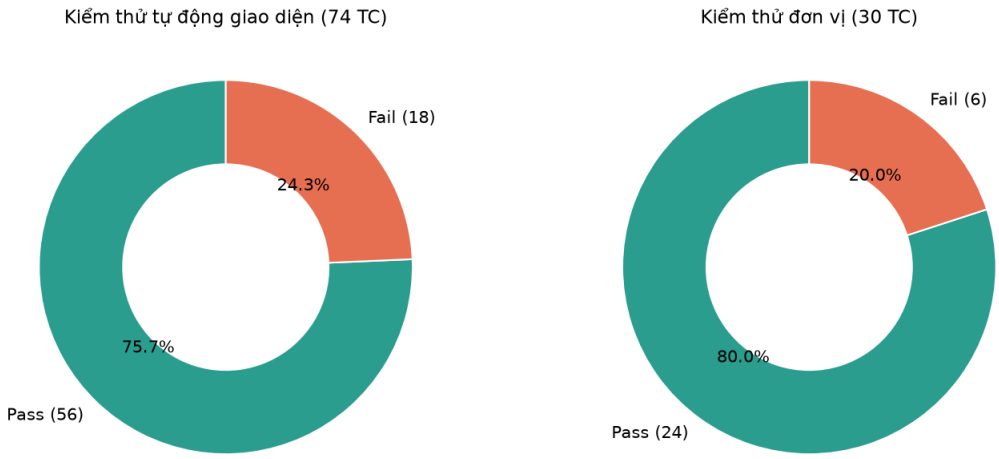
Hình 30: Kết quả chạy kiểm thử độ tin cậy và phục hồi

Nhận xét: 6/6 test case đạt. Ứng dụng không sập khi mất CSDL mà trả HTTP 500, rồi tự kết nối lại sau khoảng 1.1 giây khi CSDL hoạt động trở lại mà không cần khởi động lại. Dữ liệu đã ghi không mất khi tiến trình bị tắt đột ngột, và ứng dụng khởi động lại trong dưới 1 giây. Khi chạy liên tục, bộ nhớ tăng trong khoảng 30 giây đầu (khởi tạo bộ đệm, kết nối) rồi ổn định quanh 324 MB. Chưa thấy dấu hiệu rò rỉ bộ nhớ trong 180 giây; cần chạy dài hơn (vài giờ) trước khi triển khai thật.

5.7. Tổng hợp kết quả

Loại kiểm thử	Công cụ	Số test case	Pass	Fail	Tỷ lệ Pass
Kiểm thử chức năng tự động	Mocha + Selenium WebDriver	74	56	18	75.7%
Kiểm thử đơn vị (Unit Test)	Mocha + tsx	30	24	6	80.0%
Kiểm thử tải và hiệu năng	autocannon + Mocha	8	8	0	100.0%
Kiểm thử chịu tải	autocannon + Mocha	3	3	0	100.0%
Kiểm thử toàn vẹn dữ liệu	Mocha + mysql2	11	7	4	63.6%
Kiểm thử độ tin cậy và phục hồi	Mocha + autocannon	6	6	0	100.0%
Tổng		132	104	28	78.8%

Bảng 20: Bảng tổng hợp kết quả kiểm thử



Hình 31: Tỷ lệ Pass/Fail của kiểm thử chức năng và kiểm thử đơn vị

Với tỷ lệ Pass của kiểm thử chức năng 75.7% và còn 6 lỗi mức Cao, phiên bản hiện tại **chưa đạt** điều kiện chấp thuận ở mục 2.5 (≥ 90% Pass, không còn lỗi mức Cao). Về hiệu năng, hệ thống đạt yêu cầu 200 người dùng đồng thời với dư địa lớn; về độ tin cậy, hệ thống tự phục hồi tốt sau sự cố.

5.8. KẾT LUẬN

Hệ thống Pet Clinic Pro đã triển khai đầy đủ các nhóm chức năng chính: quản lý khách hàng và thú cưng, đặt lịch có kiểm tra trùng giờ bác sĩ và thú cưng, hồ sơ khám, ca trực, kho thuốc với cảnh báo tồn thấp, hóa đơn tự trừ tồn kho trong transaction, thanh toán và báo cáo doanh thu. Các luồng hợp lệ đều chạy đúng; trang Tổng quan phản ánh chính xác doanh thu từ hóa đơn đã thanh toán.

Về phi chức năng, hệ thống đáp ứng tốt yêu cầu hiệu năng (p99 dưới 0,7 giây với 200 người dùng, tìm kiếm trên 10.000 bản ghi dưới 0,1 giây, chịu được tới 400 kết nối), không sập khi quá tải hoặc mất CSDL và tự phục hồi nhanh.

Tuy nhiên hệ thống còn những hạn chế đáng kể:

- Chưa có đăng nhập và phân quyền RBAC, bất kỳ ai truy cập đường dẫn đều thao tác được dữ liệu.
- Không thêm được khách hàng khi bỏ trống ô Địa chỉ, và ô tìm kiếm khách hàng mất focus sau ký tự đầu – hai lỗi ảnh hưởng trực tiếp tới công việc hằng ngày của lễ tân.
- Mất toàn vẹn dữ liệu khi thao tác đồng thời: bán vượt tồn kho, tạo trùng lịch hẹn – rủi ro lớn khi nhiều lễ tân, nhiều quầy cùng làm việc.

- Nhiều quy tắc nghiệp vụ trong đặc tả chưa được kiểm tra ở phía server: trùng SĐT/email/microchip/mã vật tư, lịch hẹn quá khứ, hủy lịch trước 1 tiếng, giờ và trùng ca trực, cộng dồn thanh toán.
- Một số thông báo lỗi hiển thị chuỗi JSON kỹ thuật hoặc không hiển thị, gây khó hiểu cho người dùng.

Hướng phát triển và đề xuất khắc phục:

- **Bảo mật:** bổ sung đăng nhập (JWT/session), middleware phân quyền theo vai trò cho toàn bộ API.
- **Toàn vẹn dữ liệu:** trừ tồn kho bằng câu lệnh UPDATE có điều kiện hoặc khóa bản ghi; ràng buộc UNIQUE cho lịch hẹn, SĐT, email, microchip, mã vật tư; cộng dồn số tiền đã thu khi xác định trạng thái hóa đơn.
- **Sửa lỗi mức Cao về giao diện:** gửi null thay cho chuỗi rỗng ở các trường không bắt buộc; không thay toàn bộ danh sách bằng trạng thái tải khi tìm kiếm (dùng placeholderData/keepPreviousData).
- **Hiệu năng khi mở rộng:** phân trang, cache cho API danh sách; chạy nhiều tiến trình Node; theo dõi bằng công cụ giám sát (APM).
- **Thông báo thân thiện:** chuyển lỗi Zod thành thông báo tiếng Việt theo từng trường.
- **Duy trì kiểm thử:** tích hợp unit test, bộ Selenium, kiểm thử toàn vẹn dữ liệu và kiểm thử tải vào CI để kiểm thử hồi quy sau mỗi lần sửa.

TÀI LIỆU THAM KHẢO

[1] International Software Testing Qualifications Board (ISTQB), *Certified Tester Foundation Level Syllabus v4.0.1*.
https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

[2] Selenium Project, *Selenium Documentation*.
https://www.selenium.dev/documentation/

[3] Mocha, *Mocha JavaScript Test Framework Documentation*.
https://mochajs.org/

[4] Matteo Collina, *autocannon – HTTP/1.1 benchmarking tool*. GitHub Repository.
https://github.com/mcollina/autocannon

[5] Oracle, *MySQL 8.0 Reference Manual*.
https://dev.mysql.com/doc/refman/8.0/en/

[6] Drizzle Team, *Drizzle ORM Documentation*.
https://orm.drizzle.team/docs/overview

[7] Meta Open Source, *React Documentation*.
https://react.dev/

[8] OpenJS Foundation, *Express – Node.js Web Application Framework*.
https://expressjs.com/